Confirmation of Candidature Report

Deakin University
Deakin Applied Artificial Intelligence Initiative

Leveraging Large Language Models for Sample-Efficient Imitation Learning

Candidate Suyog Khanal
Student identifier s226137394

Supervisors Associate Professor Santu Rana
Dr Arun Kumar Anjanapura Venkatesh

Candidature start date 13 November 2025
Confirmation of Candidature date 13 August 2026

Contents

Abstract

Imitation learning converts expert demonstrations into a policy, and the demonstration is the one input whose cost does not fall. Compute is bought and architectures are downloaded, while every trajectory still has to be produced by a person or a scripted oracle, one at a time. The binding constraint on a realistic deployment is a fixed budget of B demonstrations that has to be spent well. Interactive imitation learning, the family descended from dataset aggregation, spends that budget on a single decision, and its members differ only in the signal that decides when to hand control to the expert. Two further decisions are left unclaimed. Which failure to correct, and where the corrective demonstration should begin.

This programme claims that a large language model, given a structured description of how the policy is failing together with an explicit statement of what the environment permits, can make those two decisions, and that making them raises the information content of each demonstration under a restricted budget. The model is never placed in the robot's control loop. It reads a summary of the policy's own failures and returns a request for one specific demonstration. Aim 1 realises the claim as **D**emonstration d**I**stillation for **S**ample-**E**fficient **I**mitation **L**earning, DISEIL.

Each round, DISEIL perceives the round's failures by reducing every failed rollout to a six-dimensional geometric descriptor of the state at which the policy first became unreliable, partitions those descriptors into failure modes, prioritises one mode, and prescribes the demonstrations of the round inside it. A prescription is screened before any expert time is spent on it, once for feasibility against a store of explicit environmental constraints and once against the current policy, since a scenario the policy can already solve teaches it nothing. The learner is any policy that exposes a per-step loss, and the evaluation uses a multilayer perceptron, a convolutional network and a diffusion policy without changing the selection loop.

A setting is one task under one observation modality. DISEIL was evaluated on five tasks, a 5×5 grid-world, Push-T and the Lift, Wipe and Door manipulation tasks, each under state and image observations, which gives ten settings, and in every setting against five comparison methods: four query gates of the DAgger family throughout, with Diff-DAgger as the fifth on the robot tasks and a uniform-random allocation control as the fifth on the grid-world. Nine seeds were run on the grid-world and five on each robot task. DISEIL attains the best mean success rate in all ten settings, with a mean margin of 3.71 points over the strongest competing method in each. The two modalities of a task share the expert, the reward structure and the reset distribution, so the ten settings are not ten independent experiments, and the claim of record is the conservative one: collapsed to five task means the sweep holds at five wins from five, paired $t(4) = 4.15$, $p = 0.014$.

Ablations on three of the settings place the advantage in the allocation. Removing the partition costs 4.37 success-rate points as a mean over the three, while the per-demonstration information gain of the same runs does not fall (mean change $+0.02$), so a demonstration can be individually informative and jointly redundant with the one collected in the round before. The advantage is largest where the budget is smallest, the margin over the best baseline averaging 10.97 points at a budget of 10 demonstrations, 4.77 at 20 and 2.83 at 40.

Each foundation-model component is worth about a point, 1.33 for the prescription model and 1.33 for the vision-language model, and the cause is structural: the partition that decides which failure mode to correct is computed from the geometric descriptor and consumes no model output, so by the time a model is called the decision that carries the result has been taken. Aim 2 supplies the selector with the input it lacks, a record of what has already been taught. A captioner turns a trajectory's observations and its executed actions back into language, and the captions accumulate into a coverage record that a new failure is checked against before a demonstration is requested. Aim 3 turns that record outward, pricing a demonstration against the resource that is scarce outside simulation, a teacher's time, so that a generalist policy can ask a non-expert human for exactly the demonstrations it lacks.

1. Introduction

1.1 The demonstration budget

Imitation learning turns expert demonstrations into a policy. The earliest working system fitted a network to logged pairs of camera image and steering angle and drove a vehicle with the result [71], and the framework that generalises it fits a policy to logged expert state-action pairs [2, 5]. Model capacity and observation richness have grown since. The premise has not: somebody, at some point, produced the demonstrations.

Every other input to that pipeline has become cheap. Compute is bought, simulators run faster than real time, and architectures and pre-trained encoders are downloaded. The demonstration is the one input whose cost has not fallen, because it is produced by a person operating a robot one trajectory at a time, or by a scripted oracle written by an engineer who first had to solve the task by hand. Collecting demonstrations at scale is a logistics problem in its own right, and the projects that have done it read as such: a distributed teleoperation platform, and a year of coordinated collection across dozens of institutions [41, 58]. The number of demonstrations available to a given project is bounded by something other than the researcher's willingness to wait.

That bound would not matter if demonstrations did not help, and they do. Imitation-learning performance follows a scaling relationship in the number of demonstrations and in their diversity [49]. The demonstration is therefore both the input that most improves the policy and the input that is hardest to obtain, which is what makes it the binding cost of the enterprise. The constraint in a realistic deployment is a budget of B demonstrations that has to be spent well.

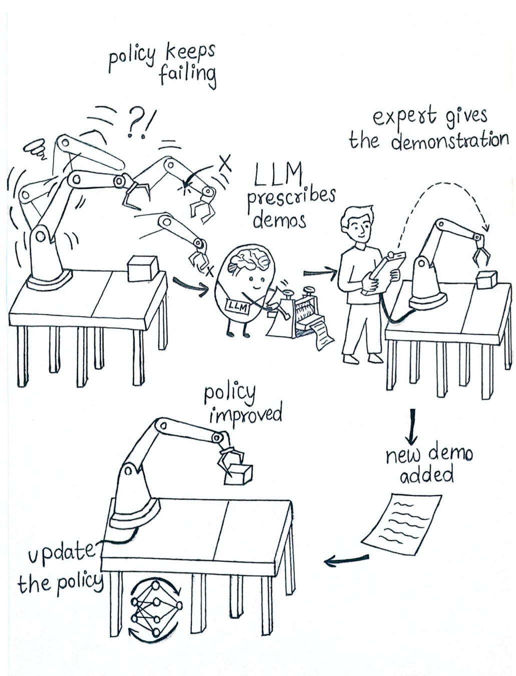


Figure 1. The demonstration-distillation loop. A policy trained on a small demonstration set fails repeatedly. The failures are summarised and read by a language model, which prescribes the configuration at which the next demonstration is to be collected. The expert supplies that demonstration, it is added to the training set, and the policy is retrained. Each turn of the loop spends one unit of the demonstration budget.

The budget is the object this thesis is built on, so its parts are named once here and the names are kept. A *budget* B is the total number of demonstrations the expert will supply over a run. Demonstrations are acquired in rounds, D of them per round, and the policy is retrained after each round. B and D are symbols of the framework and of its algorithm; the values validated in this work are stated once, in the experimental setup of Section 5.1.2, because the framework does not depend on them.

Under a fixed B, the quantity that can still be raised is the *information content of each demonstration*: how much of what the policy does not yet know is contained in the one trajectory the expert is about to record. The claim this programme develops and tests is that a large language model, given a structured description of how the policy is failing and an explicit statement of what the environment permits, can raise it.

1.2 Interactive imitation learning

A policy cloned offline from expert data is trained on the expert's state distribution and deployed on its own, and the two distributions come apart as soon as the learner's actions determine what it sees next [83]. Small action errors move the learner off the demonstrated manifold, where its errors are larger, and the error compounds over the horizon [76]. Dataset aggregation removes the compounding by labelling the states the learner actually visits: roll out the current policy, ask the expert what to do at the states it reached, add those labels to the training set, retrain, and repeat [77]. The interactive branch of imitation learning that grew out of this is now a field with its own taxonomy of feedback types [14].

Its members share one skeleton and differ in one component. The skeleton is to roll out, decide whether to hand control to the expert, aggregate the expert's labels, and retrain. The component that differs is the scalar signal that opens the gate. SafeDAgger trains a classifier to predict when the policy is about to deviate from the expert by more than a tolerance [99]. DropoutDAgger reads the spread of a Monte-Carlo dropout ensemble over the novice's action [63]. EnsembleDAgger reads the variance of an ensemble of independently trained policies and combines it with the discrepancy between the novice's action and the expert's [64]. ThriftyDAgger combines a novelty estimate with a learned risk estimate and calibrates the pair against a target switching rate [34]. Diff-DAgger reads a diffusion policy's own per-step training loss, which is available for free and requires no second model [46]. Section 2.3 gives the family as instances of one template.

Every one of those gates answers the same question, which is *when* to ask the expert for help. Three consequences follow from answering only that question, and they are the opening this programme works in.

A per-state gate cannot see a batch. It fires on the state in front of it. Given twenty rollouts that all failed, it has no representation in which two of them are the same mistake and a third is a different one, so it cannot tell a redundant correction from a novel one. Under an unbounded budget this costs nothing, because every failure is eventually labelled. Under a budget of twenty demonstrations it is the whole problem.

A per-state gate has no memory across rounds. Nothing in the signal records that the region it is firing on was already corrected in the previous round and in the round before that. One failure that persists can therefore absorb a large share of a small budget, while a failure that occurs less often is never reached.

A per-state gate inherits the state that tripped it. The corrective demonstration begins wherever the policy happened to be when the signal crossed the threshold, and that state is frequently one the policy has already ruined: the object has been knocked out of reach, or the gripper has closed on nothing. The expert then spends the demonstration recovering from a situation that would not arise under a competent policy, rather than teaching the behaviour that would have avoided it.

So *when* is one decision, and it is one of three. The other two are unclaimed: *which* failure to correct, and *where* the corrective demonstration begins.

Choosing a demonstration is also not the same problem as choosing a data point, and the distinction matters because a large literature already chooses data points. Active learning, coreset and diversity selection, and demonstration curation all select from a pool that has already been collected [3, 35, 62, 81, 82], and the word *distillation* in the title of the Aim-1 paper is not the dataset-distillation sense, in which a collected dataset is compressed into a smaller synthetic one that trains as well [13]. The framework proposed here prescribes a demonstration that does not exist yet, and then has an expert produce it. Section 2.9 treats the distinction in full.

1.3 Central idea and thesis statement

A language model is not a controller in this work. A large language model is a model over token sequences; it does not close a control loop here, it does not output torques, and it is never in the path between an observation and an action at execution time. It is the component that reads a structured summary of the policy's own failures, together with an explicit statement of what the environment permits, and returns a request for one specific demonstration.

The division of labour follows from what these models are measured to be good at. Language and vision-language models name causes reliably when they are handed structured evidence: a vision-language model can summarise a robot's experience and say why an episode failed, and can be trained specifically to reason over manipulation failures [21, 53]. They are unreliable at metric and spatial reasoning from pixels alone [15, 27], and their proposals must be grounded in what the robot can actually do before they are acted on [1]. The framework is built around both findings. The model is handed a low-dimensional geometric descriptor of each failure rather than raw pixels, the partition of failures into modes is computed geometrically and not by the model, and every prescription the model emits is checked against an explicit store of environmental constraints and revised until it is feasible. What is left to the model is the decision that structured evidence supports, which is what kind of correction the selected region of the failure distribution needs, and where the demonstration that supplies it should begin.

The thesis statement is one sentence. *Language models can raise the information content of each demonstration under a restricted budget, and raising it is what makes imitation learning sample-efficient.*

The programme tests that statement at three levels, one per aim, and each aim removes a limitation that the previous aim's own evaluation exposed. **Aim 1** is DISEIL, named after the title of the paper that reports it, *Demonstration Distillation for Sample-Efficient Imitation Learning*. It reasons over the current round's failures, partitioned into failure modes, and decides which mode to correct and where the corrective demonstration begins, so it raises the value of a demonstration within a round. Its selector knows nothing about the demonstrations already collected. **Aim 2** supplies that record. A captioner inverts the mapping that vision-language-action models learn, turning a trajectory's observations and its executed actions back into language, and the captions accumulate into a coverage record that a new failure is checked against before a demonstration is requested, which raises the value of a demonstration across the dataset. Aim 2's record is task-local, and its supplier is a scripted expert who is always available and identically priced. **Aim 3** turns the record outward. Demand for a skill is shared across tasks, embodiments and teachers, and a demonstration is priced against the resource that is actually scarce outside simulation, which is a teacher's time, so that a generalist policy can ask a non-expert human for exactly the demonstrations it lacks.

Aim 1 is complete and is reported in this document. Aims 2 and 3 are proposed. The gap the three aims close, and the three research questions they answer, are stated in Section 3, after the literature that establishes the gap.

2. Background and literature review

This chapter sets out the material the programme uses and locates the three aims in the literature they extend. The query gates of the DAgger family are both the baselines against which Aim 1 is measured and the point at which the programme departs from prior work, so they are treated at length and qualitatively. Everything the programme uses unmodified is named here, once, with attribution, and is flagged as standard practice: the behaviour-cloning objective, the aggregate-and-retrain skeleton, the query-gate template, the silhouette criterion, farthest-point selection, A* and breadth-first search. None of it is re-derived later. Abbreviations are expanded at first use in this chapter and used in short form thereafter.

2.1 Imitation learning and behaviour cloning

A task is modelled as a finite-horizon Markov decision process, or a partially observed one when the learner sees images instead of privileged simulator state,

$$\mathcal{M} = (\mathcal{S},\ \mathcal{A},\ P(s' \mid s, a),\ R(s,a),\ H), \qquad \pi_\theta : \mathcal{S} \to \Delta(\mathcal{A}),$$

with $\mathcal{S}$ the state space, $\mathcal{A}$ the action space, P the transition kernel, R the reward, H the horizon, and π_θ the learner's policy with parameters θ. An expert $\pi^\star$ supplies trajectories, and the demonstration set is the collection of state-action pairs those trajectories contain,

$$\mathcal{D} = \big\{(s_t, a_t)\ :\ a_t = \pi^\star(s_t)\big\}.$$

Behaviour cloning fits the policy to that set by supervised learning [5, 71]:

$$\theta^\star = \arg\min_\theta\ \mathbb{E}_{(s,a)\sim\mathcal{D}}\Big[\,\mathcal{L}_{\mathrm{BC}}(\pi_\theta(\cdot \mid s),\ a)\,\Big],$$

with $\mathcal{L}_{\mathrm{BC}} = -\log \pi_\theta(a \mid s)$ where the action space is discrete and $\mathcal{L}_{\mathrm{BC}} = \|\pi_\theta(s) - a\|_2^2$ where it is continuous. The reduction is old and it works: the first system of this kind steered a road vehicle from camera input through a three-layer network trained on logged human driving [71], and the formulation was later given as a named framework [5] and surveyed as one branch of learning from demonstration [2, 69].

The reduction carries one defect, and every method in this report exists because of it. Supervised learning assumes the training and test inputs are drawn from the same distribution. In imitation learning they are not. The policy is trained on the states the expert visits, and at deployment it visits the states its own actions produce. Any error moves the learner off the expert's state distribution, the next prediction is made on an input the training set under-represents, and the error grows. The statistics literature calls the mismatch covariate shift [83]; the imitation-learning consequence is quantitative. If the cloned policy incurs supervised loss ϵ under the expert's state distribution, its cost over a horizon H can grow as $O(H^2\epsilon)$, and the quadratic term is a property of the offline reduction and not an artefact of a loose bound. Allowing the learner to be corrected on the states it actually reaches removes it, leaving a bound linear in the horizon [76, 77].

The problem was visible in the first system. A policy trained only on a good driver's centred trajectory never observes a recovery from the road edge, because a good driver never produces one. Pomerleau's remedy was to synthesise the missing data: each camera image was shifted and rotated laterally, and the steering label was corrected to the command that would return the vehicle to the centre [72]. Noise injection into the expert's control stream is the modern off-policy version of the same idea [45].

The objective above is fixed for the whole programme, and it is fixed for every method compared. Neither Aim 1 nor either of the later aims changes the loss that is minimised, the optimiser, or the policy architecture. They change which demonstrations enter $\mathcal{D}$.

2.2 Dataset aggregation and the interactive loop

Dataset aggregation, introduced as DAgger, corrects covariate shift by moving the labelling effort onto the learner's own state distribution [77]. In round r the current policy π_{θ_r} is rolled out, the expert is asked for its action at the states the rollout visits, those labelled pairs are added to the dataset, and the policy is refitted to the whole aggregate:

$$\mathcal{D}_{r+1} = \mathcal{D}_r \,\cup\, \big\{(s,\, \pi^\star(s)) \;:\; s \sim d_{\pi_{\theta_r}}\big\}, \qquad \theta_{r+1} = \arg\min_\theta \, \mathbb{E}_{\mathcal{D}_{r+1}}\big[\mathcal{L}_{\text{BC}}\big],$$

where $d_{\pi_{\theta_r}}$ is the state distribution induced by rolling out the round-r policy. The analysis casts the loop as online learning against an adversarially chosen sequence of state distributions, so that a no-regret supervised learner attains a performance bound linear in the horizon rather than quadratic [77]. Variants replace action agreement with the expert's cost-to-go as the aggregation signal, which lets the learner be told how much a mistake costs and not only that it was a mistake [78, 86].

The loop has become a field rather than a single algorithm, with a taxonomy of feedback types and of who initiates the handover [14]. Human-gated variants give the decision to the person: the human watches the rollout and takes control when they judge it necessary, which removes the need for a machine-readable uncertainty signal at the cost of continuous human attention [40]. Data collected during an intervention has a different value from data collected on-policy, and can be reweighted accordingly, or used as the substrate of a deployment-time learning system in which the robot runs, a human intervenes, and the intervention becomes training data [52, 59].

One cost is the reason the rest of this chapter exists. The aggregation step as originally stated asks the expert to label every state the learner visits. An expert who must answer at every step of every rollout is an expert whose time scales with the number of rollouts, and outside simulation that expert is a person. The interactive loop trades the compounding-error problem for an expert-effort problem, and the query-efficient variants below are the field's answer to the second problem.

Aim 1 keeps this skeleton without modification, and so does every method it is compared against. The rollout, the aggregation and the retraining are shared. The only free variable is how the round's new demonstration is chosen, and holding everything else fixed is what makes the comparison a comparison.

2.3 The query gates of the DAgger family

Query-efficient interactive imitation learning replaces the ask-at-every-state rule with a gate. At each visited state the method computes a scalar score, compares it against a threshold, and hands control to the expert at the first state where the comparison fires:

$$\text{Query}(s_t) = \mathbf{1}\big[\,\text{score}(s_t) \;\gtrless\; \tau\,\big], \qquad t^\star = \min\{\,t \;:\; \text{Query}(s_t) = 1\,\}.$$

The expert then takes over from $t^\star$ and completes the episode, and the expert's segment is the round's new demonstration. The published methods differ in one place only: what they put in $\text{score}(\cdot)$. The template is stated explicitly because it makes the family's shared limitation legible.

SafeDAgger learns an auxiliary safety classifier that predicts, from the current observation, whether the policy's action will deviate from the expert's by more than a tolerance, and hands over when the classifier predicts a large deviation. The classifier is trained on the policy's own rollouts, so the gate is a learned model of where the policy is unsafe and not a direct measurement of it [99].

DropoutDAgger reads the spread of the policy's own action distribution under Monte-Carlo dropout. Several stochastic forward passes are drawn at the visited state, and the expert is called when the sampled actions stop concentrating near the expert's action [63]. The signal is imported from the Bayesian deep-learning literature, where dropout at inference time is interpreted as approximate posterior sampling [28].

EnsembleDAgger replaces the dropout samples with an ensemble of independently trained policies and reads their variance, which it calls doubt. The gate opens on high doubt or on a large discrepancy between the ensemble mean and the expert's action, so that the epistemic term and the safety term each have an arm [64]. Ensembles are the second canonical deep-uncertainty estimator and, like dropout, are used here as published [44].

ThriftyDAgger adds a second quantity to novelty. Alongside the ensemble doubt it trains an estimate of task risk, a value function predicting the probability that the episode will fail from the current state and action, and it opens the gate on either. Both thresholds are set as quantiles of the observed distributions, calibrated so that the method hands over at a target switching rate, which is what makes the method budget-aware [34]. A related design reduces the number of context switches by using asymmetric thresholds for handing over and handing back, so that control does not oscillate between learner and expert [33]; it is context here and not a baseline.

Diff-DAgger reads the learner's own training loss. For a diffusion policy the per-step denoising loss on a state-action pair is a usable score of how far that pair lies outside the training distribution, so the method thresholds it at a quantile of the training-loss distribution, recalibrated at each retrain, and hands over when the loss stays in the tail for a run of consecutive steps [46]. Using the diffusion loss as an uncertainty signal is Diff-DAgger's idea. Aim 1 uses that same per-step loss, both to localise a failure within a rollout and to measure the information content of an acquired demonstration, and it also compares against Diff-DAgger as a baseline. Both facts are stated plainly wherever the signal appears.

A uniform-random control completes the comparison, and it is a control on the *which* decision and not on the *when* decision. Referred to in this report as Stagger, it holds no gate, no score and no threshold. Each round it draws one of the round's recorded failures uniformly at random and has the expert correct it. It is not a published method: it is a floor implemented in this project, and it is never labelled as a member of the DAgger family. Its purpose is to establish what an uninformed allocation of the same expert effort buys, so that any margin a gated method reports can be read against a random one.

Gate	Scalar signal the gate reads	What opens the gate
SafeDAgger [99]	learned safety classifier predicting policy-expert deviation	predicted deviation exceeds tolerance
DropoutDAgger [63]	spread of Monte-Carlo dropout action samples	too few samples agree with the expert
EnsembleDAgger [64]	ensemble variance (doubt) and mean action discrepancy	either term exceeds its threshold
ThriftyDAgger [34]	ensemble novelty and a learned task-risk estimate	either term exceeds a budget-calibrated quantile
Diff-DAgger [46]	the policy's own per-step denoising loss	the loss stays in the tail of the training-loss distribution
Uniform-random allocation control (Stagger)	*none*	*no gate: one recorded failure of the round, drawn uniformly at random, is corrected*

Table 1. The five published query gates of the DAgger family as instances of one template: a scalar score, a threshold and a handover. The methods differ only in the score. The last row is the uniform-random allocation control implemented in this project, which has no gate and is not a member of the family.

Descriptions here are qualitative by design. The thresholds, ensemble sizes and calibration constants each published method specifies are not reproduced, and the choices made when these gates were re-implemented for the comparison are recorded with the experimental setup in the progress report.

The three properties of the template set out in Section 1.2 are properties of the form of score$(\cdot)$ and not of any one choice of it. The gate maps one visited state to one scalar, so it holds no representation in which two failures are the same mistake and a third is a different one. Nothing in score$(\cdot)$ carries across rounds, so a persistent failure mode can absorb the entire budget. And the gate inherits the state that tripped it, so the corrective demonstration starts wherever the score happened to cross the threshold. Each of the five published gates answers *when* to hand over, and answers it well. Neither *which* of a batch of failures to correct nor *where* the corrective demonstration should begin is answered by any of them.

2.4 Uncertainty estimation

The gates read their signals from the deep-uncertainty literature, and the three families of signal are worth separating because their limitations are shared. Monte-Carlo dropout treats dropout at inference as approximate posterior sampling and reads the spread of the resulting predictions [28]. Deep ensembles train several independent members and read their disagreement, which is simpler to implement and generally better calibrated [44]. Density-style signals score how far an input lies from the training distribution, and the per-step denoising loss of a diffusion policy is one such score: a state-action pair the model has not seen produces a high reconstruction error, which is what makes it usable as a query trigger [46].

All three produce a number attached to a state. A number attached to a state is enough to decide whether that state is a problem. It is not enough to decide whether that state's problem is the same problem as another state's, because two scalars can be equal for entirely different reasons and can differ while describing one underlying failure. The comparison the DAgger family cannot make is a comparison between failures, and no refinement of the scalar supplies it. What supplies it is a representation in which failures are points and not magnitudes, which is where clustering, and the standard machinery of Section 2.6, enter the programme.

2.5 Policy classes

Explicit regression onto expert actions is a poor fit for demonstration data whose action distribution has several peaks. Where two different actions are both correct at a state, a network trained under a squared-error loss learns their average, and the average may be correct under neither of the two actions. Energy-based and generative formulations of the policy fit such distributions instead of averaging them [25]. Denoising diffusion probabilistic models are the generative family that has proved most usable for this [32]. Applied to trajectories they give a planner [39], and applied to short action sequences conditioned on recent observations they give the visuomotor diffusion policy that is the learner for the robot tasks in this programme [17]. The conventions for training such policies on offline human-style manipulation data, including observation encoders and the treatment of action chunks, follow the empirical study that established them [60].

A diffusion policy is trained by noising the clean action target $x_0 = a$ over K steps and learning to reverse the corruption,

$$x_k = \sqrt{\bar{\alpha}_k}\, x_0 + \sqrt{1 - \bar{\alpha}_k}\, \epsilon, \qquad \epsilon \sim \mathcal{N}(0, I),$$

with the network trained to predict the noise, or an equivalent reparameterisation of it, and the resulting per-pair denoising loss written $L_{\text{dif}}(s, a)$. Evaluated at the state the policy visited and the action it executed, that loss gives a per-step signal along a rollout,

$$\ell_t = L_{\text{dif}}(s_t,\, a_t).$$

The quantity ℓ_t is the one Diff-DAgger thresholds [46], and the one Aim 1 uses to localise the step at which a rollout goes wrong.

The framework is stated over any policy f_θ that exposes a per-step loss at a visited state under the executed action. A multilayer perceptron trained with cross-entropy on a discrete grid exposes one. A convolutional network on grid images exposes one. A diffusion policy exposes one, in the form above. Nothing else about the policy enters the loop: not its architecture, not its action parameterisation, not whether the observation is a state vector or an image. The programme is therefore run on all three policy classes, and the diffusion policy is one instantiation of the requirement and not the requirement itself.

2.6 Standard machinery

Five routines are used unmodified. Each is named here, cited, and flagged as standard, so that the method chapter can use them without appearing to claim them.

The partition step is generic. Aim 1 groups a round's failures into failure modes with a clustering step $\mathcal{C}$, instantiated as agglomerative clustering under Ward's linkage [91]. The choice is one instantiation and not a commitment: k-means [54] or any other partition method that returns a labelling and a set of centroids would serve, and the framework is stated over $\mathcal{C}$ and not over the particular algorithm.

The number of clusters is chosen by the silhouette criterion, which scores a partition by comparing, for each point i, its mean distance $a(i)$ to the other members of its own cluster against its mean distance $b(i)$ to the members of the nearest other cluster,

$$s(i) = \frac{b(i) - a(i)}{\max\{a(i),\, b(i)\}},$$

and takes the mean over points [79]. The criterion is used exactly as published, and the number of clusters is the value that maximises the mean silhouette over a bounded range.

Diversity selection within a cluster is farthest-point, or k-centre, selection: given a set already chosen, add the candidate whose minimum distance to that set is largest, and repeat [23]. The greedy rule is standard and is used as such.

Path validity on the discrete grid task is checked with A* [30] and breadth-first search [19]. Their role must be stated precisely, because it is easy to misread. They verify that a prescribed grid configuration admits a valid path from the start cell to the goal cell around the obstacles, which is a feasibility check on the prescription. They are never the expert. The GridWorld expert is a human, and the demonstrations on that task are human trajectories.

Clustering, the silhouette computation and the feature standardisation that precedes them are taken from a standard library implementation [70].

One point of ownership is settled here and not left to the method chapter. Pre-trained visual representations for manipulation supply the visual encoder for the image-modality policies. R3M is the one used here [66], and masked visual pre-training [73] and value-implicit pre-training [55] are the alternatives that were available. Their role ends at the policy. They do not supply the features that are partitioned: clustering in this programme is geometric in every run, under both observation modalities, over a low-dimensional descriptor of the robot and object configuration at the flagged step.

2.7 Language and vision-language models

Large language models, which are autoregressive models over text, and vision-language models, which condition the same generation on images, have been used in robotics in several distinct roles, and the distinctions matter because they determine which capability this programme depends on.

As planners, language models decompose a natural-language instruction into a sequence of steps over a fixed repertoire of learned skills. The decomposition is only useful if it is grounded in what the robot can actually do, which is why the influential version of the idea scores each candidate skill by the product of the language model's likelihood that the skill is useful and a value function's estimate that the skill will succeed from the current state [1]. As programmers, they write executable code against a perception and control interface, so that the plan is a program with loops and conditionals and not a flat list [48, 85]. Multimodal variants take sensor observations directly into the language model's embedding space, so that the plan is conditioned on what the robot sees and not on a textual scene description produced by another module [20]. As designers of objectives, they write reward functions and cost maps that a downstream optimiser consumes, which is the setting in which a language model's output is a specification and never an action [37, 56, 96].

Failure reasoning and self-correction matter more directly to this programme. A vision-language model given a summary of a robot's execution, in the form of a small number of frames and a record of what happened, can name the cause of a failure, and the naming is accurate enough to drive a recovery [53]. A model trained specifically on manipulation failures does better than a general one, which is evidence that the capability is a learnable perceptual skill and not an emergent accident [21]. Closed-loop textual feedback improves an embodied planner, because a plan that fails can be revised when the failure is described back to the model that wrote it [36], and verbal self-critique with iterative refinement is by now an established pattern in its own right [57, 84]. A language model can also be calibrated to recognise when it does not know, and to ask a human instead of guessing [75], which is the direct precedent for the request interface proposed in Aim 3.

Against those capabilities stands a well-documented weakness. Vision-language models are unreliable at metric and spatial reasoning from pixels alone. They mistake relative depth, distance and size, and they fail on spatial-relation questions that a human answers instantly [15, 27]. The failure is a property of how these models are trained and not a matter of model scale, and it is measured as such on purpose-built benchmarks.

The two findings together dictate a design commitment, and the commitment is inherited by the Aim-1 method without further argument. The model is reliable at naming a cause when it is handed structured evidence, and unreliable at recovering geometry from an image. It is therefore handed the geometry, in the form of a low-dimensional numeric descriptor and an explicit store of what the environment permits, and it is asked for the thing it is good at. The model that reads frames and the model that writes the prescription are open-weight instruction-tuned models [4, 94]. Neither ever emits a robot action. In this programme a language model is a component that reads a structured summary of the policy's failures and returns a request for a demonstration, and it is not a controller.

2.8 Structured environmental knowledge and constraint grounding

Retrieval-augmented generation conditions a language model's output on passages fetched from an external store, so that the model's factual claims are anchored in retrievable text and not in its parameters [47]. Graph-structured retrieval organises the store instead of treating it as a flat pile of passages, which improves queries whose answer is spread across many documents [22]. Robot knowledge bases predate both and do something different again: they hold explicit, queryable, symbolic knowledge about objects, actions and the environment, and a robot's planner asks them questions instead of reading them as text [89].

The knowledge-augmented graph used in Aim 1 belongs to the third tradition. It stores explicit environmental constraints as structured key-value entries: workspace bounds, reachability limits, the ranges within which objects may be placed and spawned, and the limits of the controller. A prescription is checked against those entries during verification, and no passage of the store is retrieved into the model's context as evidence in the manner of retrieval-augmented generation.

The pattern in which it is used is established. A language model's proposal can be handed to an external checker and the checker's verdict returned to the model as feedback: a plan can be validated by a symbolic planner [51], and the propose-verify-revise cycle can be iterated until the proposal passes [16]. Aim 1's feasibility loop is an instance of that pattern, with the knowledge-augmented graph as the checker, so the feasibility check is not the novel part of the method.

A second check in Aim 1 has no precedent in the list above and must not be conflated with the first. Before expert time is spent, the prescribed configuration is rolled out under the current policy. If the current policy already solves it, the prescription carries no information and the configuration is revised. The nearest relatives in the literature are methods that choose start states by what the learner can and cannot yet do, in particular reverse curriculum generation, which grows the start-state distribution outward from states the learner already handles [26], and reset-learning work in which an auxiliary policy returns the system to states from which learning can continue [24]. They are the intellectual neighbours of the solvability check and they do not perform it. The two checks are separate mechanisms: one asks whether the environment permits the prescribed configuration, the other asks whether the prescribed configuration would teach the policy anything.

2.9 Demonstration selection, curation and active learning

Active learning studies which unlabelled point to send to an oracle. An acquisition function scores each candidate and the highest-scoring one is labelled, and the survey of acquisition functions is the standard entry point [82]. Two families are relevant here. Uncertainty-style acquisition scores a candidate by how unsure the model is about it, of which expected information gain, formalised as the mutual information between the label and the model parameters, is the Bayesian version [35]. Coverage-style acquisition ignores uncertainty and instead selects a subset that covers the representation space, on the argument that a model trained on a good cover of the input distribution generalises to the rest of it; the core-set formulation makes that argument precise and solves it with a greedy k-centre rule [81]. Batch acquisition needs both, because a batch of individually uncertain points can be a batch of near-duplicates, and combining an uncertainty term with a diversity term is the standard remedy [3]. The observation that individually informative selections can be jointly redundant recurs in this report as an empirical finding about the Aim-1 framework itself.

Within imitation learning, the corresponding literature is about curating demonstration data. Sub-trajectory retrieval selects segments from an existing corpus that resemble the target task and trains on them, which turns a large heterogeneous dataset into a task-relevant one at consumption time [62]. Data quality in imitation learning has been characterised directly, and the finding that demonstrations differ in value, so that more data is not automatically better data, is the premise the whole programme rests on [6]. Diverse and partly suboptimal demonstrations can be exploited and not discarded [97], and the mixture weights over data sources in large-scale training can be optimised and not set by hand [31]. Performance itself follows a scaling relationship in the number of demonstrations and in their diversity, which is what licenses the claim that a demonstration has a measurable marginal return [49].

Approach	What it selects over	Where the data comes from
Active learning acquisition [35, 82]	unlabelled points	an existing unlabelled pool
Core-set and diversity selection [3, 81]	points in a representation space	an existing unlabelled pool
Sub-trajectory retrieval [62]	segments of collected trajectories	an existing demonstration corpus
Data-mixture optimisation [31]	source datasets	existing datasets
Dataset distillation [13]	synthesised training examples	compressed from an existing dataset
DAgger-family gates [46, 77]	the timestep at which to hand over	the rollout the policy just produced
Aim 1	the configuration of a demonstration that does not exist	the expert, after the request is issued

Table 2. Selection methods by what they choose and where the data they choose from comes from. Every method above the last row selects from something that already exists. Aim 1 specifies a demonstration that has not been collected, and an expert then produces it.

The distinction in the table is the reason none of these methods is a baseline in Aim 1. An acquisition function ranks candidates in a pool. A retrieval method ranks segments in a corpus. Both presuppose that the data exists and that the only question is which of it to use. Aim 1 answers a different question: given that the policy is failing in a particular way, what demonstration should be collected next, from an expert who has not yet been asked. The demonstration does not exist until the request is made, so there is no pool to rank, and a coverage criterion over a pool cannot be evaluated.

One neighbour needs to be distinguished by name, because the title of the Aim-1 paper uses the word. Dataset distillation compresses a large training set into a small synthetic one that trains a model to comparable accuracy, by matching training trajectories or gradients [13]. The operation runs after collection and operates on data that is already held. Demonstration distillation, in the sense used in this programme, runs before collection and decides which demonstration to acquire next. The two share a word and share no mechanism.

2.10 Vision-language-action models

A vision-language-action model maps an image observation and a natural-language instruction to a robot action, and is trained end to end on demonstration data. The line began with a transformer trained on a large corpus of real robot episodes [10] and continued by initialising the same mapping from a vision-language model pre-trained on web data, so that semantic knowledge acquired from images and text transfers into control [11]. Open reproductions followed, trained on pooled cross-embodiment data and released with weights, which made the class of model available to laboratories that cannot collect at that scale [42, 67]. The pooling itself is a research object: a collaboration assembled demonstration data from many robots and many laboratories into a single corpus and showed that policies trained on the pool transfer across embodiments [68]. Later variants change the action decoder, replacing autoregressive token prediction with flow matching to emit continuous action chunks at high rate [8], and the generalist-agent line trains one network across tasks and embodiments with the same supervised recipe [9, 74].

Language has also been used as an intermediate representation inside the mapping. One design predicts a short language motion primitive from the instruction and the observation and then predicts the action from the primitive, which gives a hierarchy in which the middle layer is human-readable [7]. Another emits an explicit chain of embodied reasoning steps, including sub-tasks and object positions, before emitting the action [98]. In both, the language is produced on the way to an action and is consumed by the action decoder.

Every model in this section maps vision and language to an action. Aim 2 inverts the mapping.

2.11 Open problems

Three problems are left open by the sections above. They are named here and stated as the gap of this programme in Section 3.1, where the three research questions that answer them are also set out.

The first lies in the interactive loop. The published gates decide when to hand control to the expert, from a scalar attached to a single state, and the selection literatures decide which item to draw from a pool that has already been collected. Neither decides which of a round's failure modes should receive the next demonstration, or from which configuration that demonstration should begin.

The second lies in what a selector knows. A gate reads the current state and a failure-reasoning model reads the current episode, and neither holds a representation of the training set assembled so far, so neither can separate a genuinely novel failure from one the dataset already covers.

The third lies in what a demonstration is taken to cost. Demonstration supply has been pooled across embodiments [68], and cost-sensitive acquisition is standard where the queries are labels for data that already exists [82]. No framework holds demonstration demand: an explicit statement of which skills a policy is short of, priced against the time of the person who would have to produce them.

3. Gap and research questions

The literature of Chapter 2 leaves one decision unclaimed, and this chapter states it. Section 3.1 sets out the gap at the three levels at which it appears. Section 3.2 states the three research questions the programme answers, one per level. Section 3.3 states the design by which each answer is validated.

3.1 The gap

The programme asks one question. What is a single expert demonstration worth, and how can that worth be raised, given that the demonstration is the input whose cost does not fall when compute is bought or a simulator is downloaded. Under a fixed budget of B demonstrations, the only lever is the information content of each one. The gap in the literature is that no existing framework pulls that lever, and it is absent at three levels of resolution.

The first is a gap in the interactive loop. The published query gates decide when to hand control to the expert, and they decide it from a scalar attached to a single state. Selection methods from active learning and dataset curation decide which item to take from a pool that already exists [35, 81, 82]. Between the two lies a decision nobody makes. Given a round's worth of failures, which failure mode should receive this round's demonstration, and from which configuration should that demonstration begin. A per-state gate cannot answer either question, because it has no representation in which two failures are the same mistake and a third is a different one, and because it must begin the corrective demonstration at whatever state tripped the threshold, which is frequently a state the policy has already ruined. Under an unbounded budget this costs nothing. Under a budget of twenty demonstrations it is the whole problem.

The second is a gap in what the selector knows. A gate reads the current state. A failure-reasoning model reads the current episode. Neither holds a representation of the training set assembled so far, so neither can separate a genuinely novel failure from one the dataset already covers several times over. Under a restricted budget, a demonstration spent re-teaching material the policy has already been shown is a demonstration lost.

The third is a gap in what a demonstration is taken to cost. Counting demonstrations is the right accounting in simulation, where a demonstration is a call to a motion planner and every call costs the same. Cross-embodiment pooling has made demonstration *supply* a shared object [68], and cost-sensitive acquisition is standard where the query is a label for a datum that already exists [82]. The minutes a demonstration takes to produce are recorded by the large collection efforts [41]. No framework holds demonstration *demand*: an explicit statement of which skills a policy is short of, priced against the time of the person who would have to produce them, and shared across the tasks and embodiments over which those skills recur.

The three aims close the three levels of the gap in order, and each aim raises the value of a demonstration at the level the previous aim's evaluation could not reach. Table 3 states the three levels.

Aim	What the selector reasons over	What it decides	Level
Aim 1	this round's failures, partitioned into failure modes, and the environment's explicit constraints	which failure mode to correct, and where the corrective demonstration begins	within a round
Aim 2	the same failures, and a language index of what the policy has already been taught	whether a failure is a coverage gap or a re-teaching of material the dataset holds	across the dataset
Aim 3	the coverage of a skill inventory shared across tasks, embodiments and teachers, priced against human time	which demonstration to buy next, from whom, and what it is worth	across tasks and teachers

Table 3. The three aims and the level at which each raises the value of a demonstration.

3.2 Research questions

RQ1. Under a fixed budget B, does choosing which failure mode to correct and where the corrective demonstration begins yield a policy with a higher final success rate than choosing only when to intervene?

The DAgger family answers *when*, and *when* is one decision of three. The other two, *which* and *where*, are the two the family leaves fixed by default: the failure it corrects is whichever rollout tripped the gate first, and the demonstration begins at the state that rollout was already in. RQ1 asks whether making those two decisions deliberately is worth anything under a budget small enough that a redundant correction cannot be recovered from. The question is answerable because the interactive loop, the policy class, the expert and the evaluation protocol can all be held fixed while the demonstration-acquisition rule alone is varied, which is the design of the Aim-1 experiments. Section 4.1 states the aim and the framework that answers the question; Section 5.1 reports the experiments and the result.

RQ2. Can the demonstration selector be given a model of what it has already taught, by inverting the mapping that vision-language-action models learn so that an executed trajectory is turned into language rather than language into an action, and does selection driven by a coverage gap beat selection driven by the failure in front of the policy?

The forward mapping takes vision and language to an action [11, 42, 67]. Aim 2 inverts it, so that a trajectory's frames and its executed actions produce a language description, and the descriptions accumulate into a memory of what the dataset contains. The motivation is the second level of the gap. A selector that reads only the current round will spend a demonstration on a failure the training set already covers, and it has no signal that would tell it so. The question has a second half because a memory that is merely readable is not a memory that does causal work, and the difference is measurable: a selector given the same failures and a coverage memory can be compared against the same selector given the same failures and no memory. Section 4.2 states the method and names that comparison before it is run.

RQ3. Can demonstration demand be made explicit, priced against a teacher's time, and satisfied across tasks and embodiments, so that a generalist policy asks a non-expert human for exactly the demonstrations it lacks?

Generalist policies already pool demonstration supply across many robots and many tasks [68]. Cost-sensitive querying is standard where the query is a label for a datum that already exists [82]. Neither prices a demonstration that has not been collected, and neither carries a demand that transfers between tasks or between teachers. RQ3 asks whether the demand can be constructed as an object: a ledger of missing skills, a price in minutes of human time, and a request rendered so that a person who is not a robotics expert can satisfy it. It is the level at which the cost the thesis is about stops being counted in demonstrations and starts being counted in the time of the person supplying them. Section 4.3 proposes the construction and the human study that tests it.

The three questions compose. Aim 1's selector reasons about the failure in front of it and knows nothing about the dataset behind it, which is the limitation RQ2 exists to remove. Aim 2's memory is task-local and its supplier is a scripted expert who is always available and identically priced, which is the limitation RQ3 exists to remove. Each aim is the correction to the defect the previous aim's own evaluation exposed, and Table 6 records which components carry across all three.

3.3 Validation strategy

The method of validation is the matched comparison, and it is the same at every level. The interactive loop, the policy class, the expert, the retraining schedule and the frozen held-out evaluation set are held fixed, and the only quantity that varies between arms is the rule by which the round's demonstration is chosen. Holding everything else fixed is what licenses attributing a difference in final success rate to the acquisition rule, and it is why the published query gates are re-implemented inside this project's own loop and are not compared against their reported numbers.

Four commitments follow from that design and are honoured throughout the report.

Evidence is reported at the level at which it was measured. A setting is one task under one observation modality, and the two modalities of a task share the expert, the reward structure and the reset distribution, so the settings of a task are correlated by construction. Aggregate claims are therefore made on the collapsed task means, which is the conservative reading, and the setting-level result is given beside it and not instead of it.

A component is claimed only where an ablation supports it. The Aim-1 ablation programme was designed so that it could retire components of the framework, and Section 5.1.8 records what it changed: one component re-specified, and the claims made for three others reduced. The same discipline applies forward. Section 4.2.4 names, before the experiment is run, the single ablation that decides whether language does causal work in the Aim-2 selector, and pre-commits to the interpretation of a negative result.

A mechanism claim is made only where the measurement can discriminate. Where the framework and its baselines both sit at the ceiling of the success-rate metric, a null result cannot separate a component that does nothing from a component whose effect cannot be observed, so the ablation programme is run in settings that have headroom for an effect to appear in.

Symbols carry the framework and values carry the instance. The budget B, the per-round acquisition count D and the policy f_θ appear as symbols in the method and in the algorithm. The values at which they were validated appear once, in the experimental setup of Section 5.1.2, because the framework is defined for any fixed restricted budget and the reported instance is one point in that family.

4. Aims and approaches

The programme has three aims. Aim 1 raises the value of a demonstration within a round of interaction, and it is the aim on which work has been completed; its implementation, experiments and ablations are reported in Section 5. Aim 2 raises the value of a demonstration across the dataset the loop is building. Aim 3 raises it across tasks, embodiments and teachers, and prices it against the human time it consumes. Each aim is stated here as a problem, a formulation and a method. Each is the correction to the limitation the previous aim's own evaluation exposes.

4.1 Aim 1. Demonstration distillation under a fixed budget

4.1.1 Motivation and problem statement

A practitioner who holds a fixed allowance of demonstrations faces a question the scaling relationship of Section 1.1 does not answer. Policy performance rises with the number and the coverage of the demonstrations a policy is trained on [49], and the allowance cannot grow, so the only open question is what each demonstration in it should contain. That question is RQ1, stated in Section 3.2: under a fixed budget, does choosing which failure mode to correct and where the corrective demonstration begins yield a higher final success rate than choosing only when to intervene?

Write B for the number of demonstrations the expert will supply beyond an initial set, and D for the number acquired in one round of interaction, so that the loop runs for B/D rounds. Neither symbol carries a value in this section. The framework is defined for any fixed budget and any per-round acquisition count, and the values at which it was validated appear once, in the experimental setup of Section 5.1.2. What the framework maximises under that budget is the information content of each demonstration: informally, how much of the policy's remaining error the demonstration can remove, and formally, the policy's per-step loss on the demonstration measured before it is trained on it. Section 5.1.5 is the argument that this quantity means what it appears to mean.

The gap it addresses is the first of the three stated in Section 3.1, and its two halves are the decisions the DAgger family leaves fixed by default: which failure to correct, and where the corrective demonstration begins. The adjacent selection literatures do not claim them either, because every one of them chooses from data that has already been collected, which is the distinction Table 2 draws. This framework prescribes a demonstration that does not exist yet and then has an expert produce it, and that operation is what the word distillation names in the title of the Aim-1 paper.

The policy is any function f_θ that maps an observation to an action and exposes a per-step loss ℓ_t at a state-action pair. The requirement stops there. A multilayer perceptron on a discrete grid, a convolutional network on grid images and a diffusion policy on a manipulator [17] all satisfy it, and all three are used in the experiments. The framework is a way of spending a demonstration budget, and the only thing it asks of the learner is a loss it can read.

4.1.2 Problem formulation

The interactive loop skeleton is shared by every method compared in this report, it is stated in Section 2.2, and it is not re-derived here [77]. What follows fixes the notation and isolates the one component that differs between methods.

The policy f_θ is trained on an initial demonstration set $\mathcal{D}_0$ by behaviour cloning. Rounds are indexed by r. At the start of round r the policy is rolled out on a fresh pool of episodes drawn from the task's reset distribution, and the episodes it fails are collected into the round's failure set. The only requirement placed on the policy is that it expose a per-step loss at its own executed action,

$$\ell_t^{(i)} \;=\; \mathcal{L}(f_\theta,\; s_t^{(i)},\; a_t^{(i)}), \tag{1}$$

where i indexes an episode, s_t is the observation at step t and a_t is the action the policy itself executed. For a diffusion policy $\mathcal{L}$ is the denoising loss, which is what Diff-DAgger uses as its gate signal [46]; for a discrete policy it is the negative log-likelihood of the executed action. The failure set of the round is

$$\mathcal{F}_r \;=\; \{\, f_i \;=\; (\tau_i,\; \ell^{(i)}_{1:T_i}) \;:\; \tau_i \text{ is a failed rollout of } f_\theta \,\}, \qquad N \;=\; |\mathcal{F}_r|, \tag{2}$$

with τ_i the trajectory and T_i its length. Success is measured on a frozen held-out evaluation set that no method sees during acquisition, and the same set is used for every method.

Each round ends by acquiring D demonstrations from the expert and aggregating them,

$$\mathcal{D}_r \;=\; \mathcal{D}_{r-1} \cup \{d_{r,1}, \dots, d_{r,D}\}, \qquad \theta_r \;=\; \arg\min_\theta \; \mathbb{E}_{(s,a)\sim\mathcal{D}_r}[\mathcal{L}_{\text{BC}}], \tag{3}$$

and the loop stops when the budget is exhausted, that is when $|\mathcal{D}_r| - |\mathcal{D}_0| = B$. Retraining is from scratch, at a per-task cadence, and follows standard practice.

Every method compared in this report instantiates the same three-part acquisition rule. Writing A for the rule, a round's acquisition is fully specified by

$$A \;=\; (\; \underbrace{t^\star}_{\text{when}}\,,\; \underbrace{C_{\text{tgt}}}_{\text{which}},\; \underbrace{\xi}_{\text{where}} \;), \tag{4}$$

where $t^\star$ is the step at which the expert takes over, C_{tgt} is the failure mode the round is spent on, and ξ is the specification of the state from which the corrective demonstration begins. The DAgger family fixes the second and third components trivially: C_{tgt} is whichever rollout tripped the gate first, and ξ is the state the rollout was already in. DISEIL computes all three. The methodology below is a description of how the second and third are computed, and nothing else in the loop is changed, which is what makes the comparison a controlled one.

4.1.3 Methodology

DISEIL runs four stages once per round: perceive the round's failures, partition them into failure modes, prioritise one of those modes, and prescribe the D demonstrations the expert is asked to supply. B and D are symbols throughout this section and throughout Algorithm 1. Standard machinery is used inside three of the four stages and is flagged as standard where it appears. The contribution is the pairing of a partition of the round's failures into modes with a prescription that is verified against an explicit model of what the environment permits before an expert is asked to satisfy it. This section states the framework; the implementation and the evidence are in Section 5.

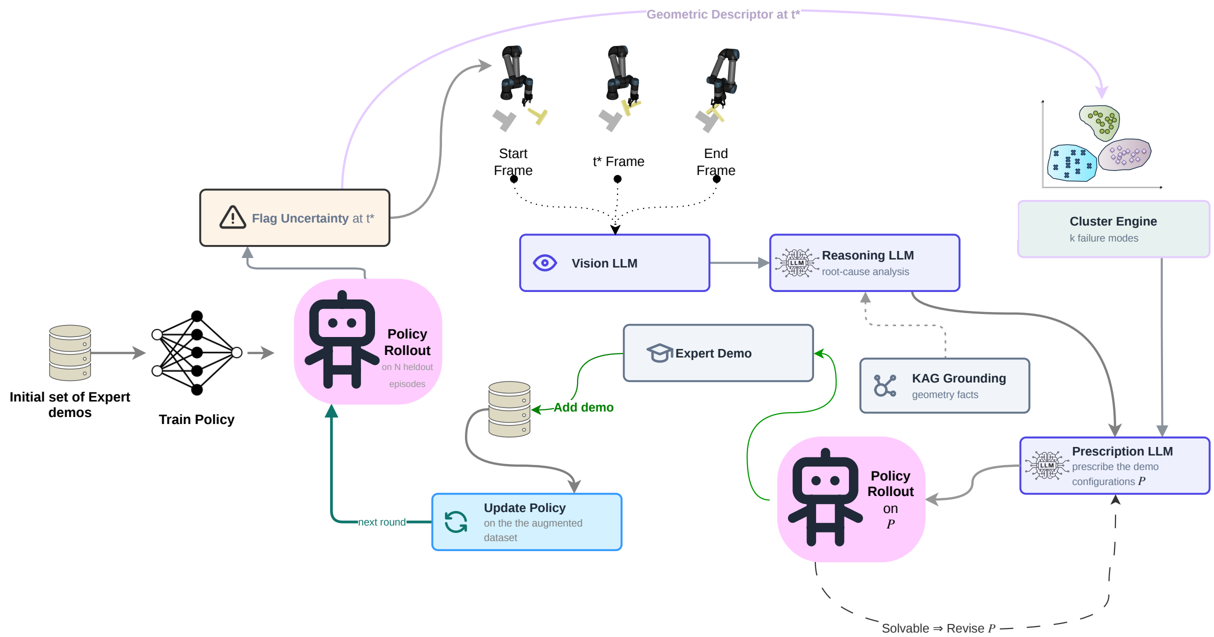


Figure 2. The DISEIL framework. A policy is trained on an initial demonstration set and rolled out; a query gate flags the step $t^\star$; a geometric branch and a perception branch describe that step; a prescription model proposes the configuration P of the next demonstration; the expert demonstrates it and the policy is retrained.

Figure 2 draws the framework as implemented, and the four stages below are the four bands of it. The reading of the figure is given at the end of this section, once the stages it draws have been stated.

Perceive. The first act of a round is to say where each failure went wrong, in two languages at once: a geometric one, which the partition consumes, and a natural one, which the prescription consumes. The two descriptions are computed from the same step of the same episode and they never mix.

Each failure is anchored at a single step $t_i^\star$, defined as the first step at which the per-step loss crosses an out-of-distribution threshold η and stays across it for K consecutive steps,

$$t_i^\star \;=\; \min\Big\{\, t \;:\; \ell_u^{(i)} > \eta \text{ for all } u \in [\,t,\, t+K\,] \Big\}, \qquad \text{with } t_i^\star \leftarrow \arg\max_t \ell_t^{(i)} \text{ if no crossing occurs.} \tag{5}$$

The threshold is a quantile of the training-loss distribution, recalibrated at every retrain, which is the Diff-DAgger construction used unchanged [46]. Taking the first crossing rather than the loss peak is a deliberate departure from the obvious definition. In a failing episode the peak arrives late, so an expert who takes over at the peak inherits a badly corrupted state and has almost no episode left in which to correct it. The first crossing is early, the state is less corrupted, and the expert has budget to work with.

Each failure is then reduced to a six-dimensional vector $\phi_i \in \mathbb{R}^6$ computed from the privileged simulator state at $t_i^\star$. For the manipulation tasks the canonical form is

$$\phi_i \;=\; [\,p_x,\; p_y,\; \sin\theta,\; \cos\theta,\; \rho_i,\; \delta_i\,], \qquad \rho_i = \frac{t_i^\star}{T_i}, \qquad \delta_i = \big\| p_i^{\text{tcp}} - p_i^{\text{obj}} \big\|_2, \tag{6}$$

where (p_x, p_y) is the planar position of the task-relevant object, θ its yaw, entered through its sine and cosine so that the wrap at $\pm\pi$ does not create a false distance, ρ_i the fraction of the episode completed before the failure was flagged, and δ_i the distance between the end-effector and the object at the flagged step. Where a task randomises no yaw, the two orientation slots carry the task's own state variables in their place. Table 4 gives the instantiation used for each task. The descriptor is geometric under both observation modalities, and clustering is geometric for every run: no output of any foundation model enters the partition.

Table 4. The six-dimensional geometric descriptor, per task. The descriptor is computed from the privileged simulator state at the flagged step and is the same under both observation modalities.

Task	The six components of ϕ
GridWorld	agent cell (2), signed offset to goal (2), progress, Manhattan distance to goal
Push-T	block planar position (2), sin θ, cos θ, progress, end-effector-to-block distance
Lift	cube planar position (2), progress, gripper-to-cube distance, gripper height, grasp indicator
Door	door-frame position (2), frame yaw, normalised hinge angle, end-effector-to-handle distance, progress
Wipe	remaining-dirt centroid (2), proportion wiped, end-effector-to-centroid distance, markers remaining, progress

The descriptor is small because the failure sets are small, and the two facts are connected by distance concentration: adding weakly informative dimensions to a distance computation over a few dozen points pushes all pairwise distances toward each other and makes the merge order of the clustering arbitrary. The width is a design choice and it is swept in Section 5.1.7.3.

In parallel with the descriptor, three rendered frames of the failing episode, at its start, at $t_i^\star$ and at its end, are passed to a vision-language model [4], which returns a short spatial account of what went wrong. A text-only reasoning model then converts that account into a root cause and a trajectory phase, drawn from a closed taxonomy stored in the task's knowledge-augmented graph rather than invented by the model. The literature supports this division of labour. Vision-language models are competent at naming a cause when they are given structured evidence [21, 53] and unreliable at metric and spatial reasoning from pixels alone [15, 27]. The framework therefore asks them for the cause and computes the geometry itself.

Partition. The round's failures are partitioned into failure modes by a generic clustering step $\mathcal{C}$ applied to the standardised descriptors,

$$\tilde{X}_i \;=\; \frac{\phi_i - \mu}{\sigma_\phi}, \qquad \{C_1, \dots, C_{k^\star}\} \;=\; \mathcal{C}(\tilde{X},\, k^\star), \qquad k^\star \;=\; \arg\max_{k \in [2,\, k_{\max}]} \mathrm{sil}(k), \tag{7}$$

with $k_{\max} = \max(2, \min(6, N-1))$. The step is generic by design: agglomerative clustering is the instantiation used here [91], k-means or any other partition method would serve [54], and the cluster count is selected by the silhouette criterion, which is standard and is used unmodified [70, 79]. The framework claims the presence of a partition step, not its implementation.

Each mode carries three quantities that the later stages consume: its centroid in the raw pose coordinates, its mean peak loss $\bar{L}_C$, and a representative $\mathrm{rep}(C)$, defined as the member nearest the cluster mean in the standardised feature space. The dominant mode $C^\star$ is the one with the most members, ties broken by mean peak loss. When fewer than four failures remain, the silhouette sweep is skipped and each failure becomes its own singleton, so in the late rounds of a budget the partition is inactive and the round is allocated by the fallback rule stated below. The modes are geometric, so they recover cause only to the extent that configuration determines cause, and the framework's claim about semantic modes is qualified by the purity measured in Section 5.1.7.4 wherever it is made.

A partition returns integers, and a method that reports a failure in mode 2 has said nothing. A mode's name is therefore the majority root cause among its members, taken from the per-failure labels the reasoning model assigned. The model may only choose from the categories enumerated in that task's knowledge-augmented graph, so the vocabulary of names is authored in the graph and the model's job is assignment rather than invention.

Prioritise. This stage makes the pair of decisions the framework owns: which mode the round's demonstrations are spent on, and which failures are shown to the prescription model as evidence.

The round is spent on the mode of highest mean peak loss among the modes that are near-dominant, which is to say within one member of the largest,

$$C_{\text{tgt}} \;=\; \arg \max_{C \,:\, |C| \geq |C^{\star}|-1} \bar{L}_C. \tag{8}$$

The size constraint $|C| \geq |C^{\star}| - 1$ keeps the target inside the bulk of the round's failures, so a mode that barely exists cannot capture the round's budget on the strength of one badly failed episode. Coverage-driven selection over a representation space [81], batch acquisition that mixes uncertainty with diversity [3] and the reweighting of intervention data [52, 59] are the nearest relatives of this decision, and none of them chooses among failure modes discovered inside the round it is allocating.

A rule of this shape can return the same mode round after round, because one demonstration rarely removes a mode outright. The framework therefore carries a cluster memory, and the memory is a configurable, task-dependent component rather than a part of the core rule. When it is switched on, it holds the centroids of the modes already corrected, tagged with the round in which the correction happened, and subtracts from each candidate's score a recency-discounted Gaussian penalty on the distance to those centroids, under a discount γ, a kernel width σ and a weight λ. Setting $\lambda = 0$ switches it off and returns Equation 8 exactly. It becomes active on a task whose failures form recurring clusters, where it rotates the budget away from modes that have already been corrected, and in an environment without that recurrence it costs negligible overhead and changes performance very little. The evidence says both halves of that plainly. Switching the memory off costs 0.6, 0.4 and 1.2 points on the three ablation settings of Section 5.1.7, the smallest of the seven knockouts; and the kernel is inert in most rounds whatever its constants are, because the candidate set of near-dominant modes is a singleton in 56 to 84 per cent of rounds on the settings with enough telemetry to measure it, and the dominant mode is then returned regardless of the penalty. The kernel width is a single global constant in this instantiation and the tasks do not share a spatial scale; Section 5.1.9 records the consequence and the identified fix. The memory is switched on in every run reported in Section 5, and it is a component of the instantiation and not part of the framework's contribution, which is the pairing of the partition with a feasibility-verified prescription.

The prescription model is not shown every failure in the target mode. It is shown a small set S of cited failures, capped at κ members and built by three rules,

$$S_0 \;=\; \big\{\text{rep}(C_{\text{tgt}})\big\} \cup \big\{ \arg\max_i \text{peak}_i \big\}, \qquad S \;\leftarrow\; S \cup \left\{ \arg\max_{i \notin S} \min_{j \in S} \big\|\tilde{X}_i - \tilde{X}_j\big\|_2 \right\} \text{ until } |S| = \kappa. \tag{9}$$

The representative of the target mode is forced into the set, because without it the model can be asked to fix a mode of which it has seen no example. The worst-loss failure is seeded next. The remaining slots are filled by farthest-point selection, which is standard [23] and is used here so that the cited failures span the mode rather than crowd its loss peak.

Prescribe. The prescription model [94] receives the target mode's anchor geometry, the cited failures in S with their root-cause labels, and the rendered constraints of the task, and returns the round's request for D demonstrations together with an integer confidence score and a one-line rationale. Each requested demonstration takes one of two forms. A targeted correction names one cited failure; that exact episode is re-instantiated, and the expert takes over at the flagged step and completes it. A bridging placement names two or three cited failures and asks for a new configuration positioned between them, from which the expert demonstrates a complete episode. Bridging changes the environment's configuration instead of selecting a recorded episode, and it is what allows a prescription to be easier than any failure it addresses: when a mode lies far outside anything the current policy can solve, a targeted correction is a large distributional jump and a bridged one is a step the policy can absorb. Which of the two arms exists is a property of the task, and the framework reads that property from the knowledge store instead of hard-coding it. Wipe randomises a path of dirt markers rather than the pose of a single object, so there is no object pose to place in a middle ground, and the task's graph declares the task targeted-only.

A prescription is a request for a configuration of the world, and a language model asked for a configuration will sometimes ask for one the world cannot produce: an object outside the reachable set, a pose outside the spawn range, a grid layout with no path from start to goal. The knowledge-augmented graph is the store that makes such a request checkable. It holds explicit environmental constraints as structured key-value knowledge rather than as prose: workspace bounds, object and spawn ranges, reachability, controller limits, the success predicate, and the task's failure-mode and phase vocabulary. It is not a document store to be retrieved from in the manner of retrieval-augmented generation [22, 47]; it is closer to the explicit, queryable environment and action knowledge of a robot knowledge base [89].

Verification is a loop. The prescription model proposes, the constraints are retrieved from the graph, a map g turns the proposal into a concrete reset specification ξ, the specification is checked against the retrieved constraints, and a violation is returned to the model as feedback so that it can propose again:

$$\begin{aligned}
\text{cmd}^{(j)} &= \text{LLM}\big(\,A,\ S,\ \mathcal{K},\ \text{violation}(\xi^{(j-1)})\,\big), \qquad \xi^{(j)} \ = \ g\big(\text{cmd}^{(j)}\big), \\
V(\xi) &= \mathbf{1}\big[\,\xi \in \mathcal{W}_{\mathcal{K}}\,\big] \ \wedge\ \mathbf{1}\big[\,\text{reachable}_{\mathcal{K}}(\xi)\,\big] \ \wedge\ \mathbf{1}\big[\,\text{valid-path}_{\mathcal{K}}(\xi)\,\big], \\
\xi^{\star} &= \xi^{(j)} \ \text{ for the first } j \le J_{\max} \text{ with } V\big(\xi^{(j)}\big) = 1, \qquad \text{else } \xi^{\star} = \text{nearest untried failure},
\end{aligned} \tag{10}$$

where $\mathcal{K}$ is the task's graph, $\mathcal{W}_{\mathcal{K}}$ its workspace bounds, and the conjuncts of V are the constraints the graph stores for that task. On the manipulation tasks the reachability and workspace conjuncts are box constraints on the object pose, padded from a measurement of the simulator's own reset sampler, so a prescribed configuration can never leave the task's native reset distribution. On the grid task the constraint is a path-validity predicate rather than a box: the prescribed layout must place start, goal and obstacles on distinct in-grid cells and must admit an obstacle-free path from start to goal, and that predicate is decided by breadth-first search [19]. The search is never the expert. A failed attempt consumes no budget, because the budget counts demonstrations collected and not prescriptions proposed, and after $J_{\max}$ attempts the round falls back to the deterministic rule of taking the nearest untried recorded failure, which is a correction the environment is guaranteed to be able to instantiate. The propose-verify-revise pattern is not new in itself [16, 51]. What the framework adds is the object being verified, which is a request for a training demonstration rather than a plan to be executed.

Feasibility asks whether the environment can instantiate the prescribed configuration. A second and separate question is whether the configuration is worth an expert's time at all. A prescription the current policy can already solve carries no information, and a unit of a restricted budget would be spent for nothing. The framework therefore contains a second screen. The prescribed configuration $P = \xi^{\star}$ is rolled out under the current policy, and

Algorithm 1 DISEIL

Require: initial demonstration set $\mathcal{D}_0$; policy f_θ; expert $\pi^\star$; budget B; demonstrations per round D; knowledge-augmented graph $\mathcal{K}$; context-set cap κ; re-prescription limit J_{max}

Ensure: the trained policy f_θ

train f_θ on $\mathcal{D}_0$ by behaviour cloning
for $r = 1$ to B/D **do**
  $\mathcal{F}_r \leftarrow$ the failed rollouts of f_θ on a fresh pool of episodes ▷ Eq. 2
  for all failures $f_i \in \mathcal{F}_r$ **do**
    $t_i^\star \leftarrow$ the flagged step of f_i ▷ Eq. 5
    $\phi_i \leftarrow$ the geometric descriptor of f_i at $t_i^\star$ ▷ Eq. 6
    assign f_i a root cause from the frames at $t_i^\star$ and the taxonomy in $\mathcal{K}$
  end for
  $\{C_1, \dots, C_{k^\star}\} \leftarrow$ partition $\{\phi_i\}$ into failure modes ▷ Eq. 7
  name each mode by the majority root cause of its members
  $C_{\text{tgt}} \leftarrow$ the near-dominant mode of highest mean peak loss ▷ Eq. 8
  $S \leftarrow$ the cited failures of C_{tgt}, at most κ of them ▷ Eq. 9
  repeat
    $\xi \leftarrow$ a configuration prescribed from C_{tgt}, S, $\mathcal{K}$ and the last violation
    $v \leftarrow$ the constraint of $\mathcal{K}$ that ξ violates, if any
  until $V(\xi) = 1$ or J_{max} attempts are spent ▷ Eq. 10
  if $V(\xi) = 0$ **then**
    $\xi \leftarrow$ the nearest untried failure in $\mathcal{F}_r$
  end if
  if f_θ already solves ξ **then**
    revise ξ ▷ Eq. 11
  end if
  $d_{r,1}, \dots, d_{r,D} \leftarrow D$ demonstrations from $\pi^\star$ at ξ
  $\mathcal{D}_r \leftarrow \mathcal{D}_{r-1} \cup \{d_{r,1}, \dots, d_{r,D}\}$ ▷ Eq. 3
  retrain f_θ from scratch on $\mathcal{D}_r$ ▷ Eq. 3
end for
return f_θ

$$\text{SR}_{f_\theta}(P) \;\geq\; \tau_{\text{solve}} \quad\Longrightarrow\quad \text{revise } P, \tag{11}$$

so that a solvable prescription is returned to the prescription model rather than to the expert. The nearest relatives are the reverse-curriculum and reset-state literatures, which choose start states by what the learner can and cannot yet do [24, 26]. The two screens are distinct mechanisms: the first rejects a configuration the world cannot produce, the second rejects a configuration the policy does not need. The solvability screen is a design element of the framework and no more than that. It is not exercised in the runs reported in Section 5, and no number in this report is attributable to it.

Algorithm 1 states the round. The loop header is symbolic, and a budget of any size runs the same algorithm.

Architecture. Figure 2 is read from left to right. The loop opens on a set of seed demonstrations and a behaviour-cloning fit, and the trained policy is rolled out on held-out episodes to produce the round's failure set. The query gate reads the per-step loss and returns the step at which the policy first becomes unreliable, which is the component the DAgger family consists of and the point at which DISEIL keeps going. Two branches leave that gate. The geometric branch carries the descriptor at $t^\star$ into the cluster engine, which emits the round's failure modes. The perception branch carries three rendered frames into the vision-language model and its output into the reasoning model, which is fed the geometry facts and the failure vocabulary of the knowledge-augmented graph. The two branches do not meet until the prescription model, which is the visual statement of the fact that the partition consumes no output of any foundation model. The prescription model emits the configuration P, which is screened for solvability by a rollout under the current policy before any expert time is spent on it. The expert then demonstrates the surviving configuration, the demonstration is added to the dataset, the policy is retrained, and one unit of the budget has been spent. Everything the framework does in a round bears on the choice of what the policy is trained on, and nothing in it touches how the policy is trained, which is what makes the comparison against the DAgger family a controlled one.

Two things the drawing understates are stated here. The feasibility loop of Equation 10 is a mechanism of the implemented framework, and the figure shows the knowledge-augmented graph only as a one-way input to the reasoning model. The solvability screen of Equation 11 is drawn as a return arrow, and it is the mechanism that is not exercised in the reported runs. The cluster memory is not drawn at all: it is a configurable, task-specific component that is active only when a task exhibits recurring failure clusters, it is the least damaging of the seven knockouts and not a headline contribution of the framework, and it is therefore left out of the framework figure and carried in the text as a feature of the loop that a task enables or disables.

4.2 Aim 2. Reverse vision-language-action, a coverage memory

4.2.1 The limitation Aim 1 leaves

DISEIL's selector reasons about the failure in front of it and knows nothing about the dataset behind it. Every input the language model receives in a round is drawn from that round: the flagged step of a failed rollout, three frames around it, a six-dimensional geometric descriptor, the partition of the round's failures into modes, and the constraints the environment imposes. The model can say that the policy failed at a particular configuration and that the cause is a grasp that closes before the gripper is aligned. It cannot say that the training set already holds six demonstrations of that cause, so the failure is not a gap in what the policy has been taught but a gap in how well it has been taught. Under a fixed budget, a demonstration spent re-teaching material the dataset already covers is a demonstration lost, which inverts the objective the programme sets out to serve.

Section 5 measures that limitation rather than asserting it, and three of its findings carry into the design of Aim 2. The knockouts of Section 5.1.7.2 show that each language-model component is worth about one success-rate point, and it is worth that little because the allocation decision is taken by a geometric partition that consumes no output from any foundation model. The one piece of cross-round state DISEIL holds is the cluster memory, which records where corrections have been placed and not what the training set contains; its kernel width is mis-scaled for the narrow-reset tasks (Section 5.1.9), and a per-task width would repair the scaling but not the representation, because a correctly scaled geometric memory still answers whether a demonstration has been placed near a location and not whether the dataset already contains a behaviour. The purity diagnostic of Section 5.1.7.4 measures where those two questions come apart: geometry mixes causes exactly where it separates them poorly, and no measurement inside the system tells the two failures apart. A memory indexed on coordinates cannot represent the difference between a failure mode it has covered and one it has merely visited.

The gap is therefore that no demonstration selector holds a model of the dataset it is building. Interactive imitation learning selects by local uncertainty and consults no training set [34, 46, 64, 77]. Selection and curation reason about a dataset but choose from a pool that already exists [3, 6, 62, 81, 97]. Language is used in robot learning on the way to an action [8, 11, 42, 67] or as an intermediate en route to a motor command [7, 98], and trajectory captioning exists as a primitive scored as description [43, 87, 92, 93], but none of that work keeps a cross-episode record of what has been taught or uses the description to decide what to collect next. Aim 2 addresses RQ2, stated in Section 3.2.

4.2.2 Core idea and proposed method

Core idea. A vision-language-action model consumes an instruction and an image and emits motor commands. Aim 2 runs the mapping backwards. A captioner C_ϕ consumes a trajectory's visual observations together with its executed action sequence and emits language:

$$C_\phi : \tau = \{(o_t, a_t)\}_{t=0}^{H-1} \longmapsto (\ell_{\text{traj}},\ \ell_{\text{act}},\ \ell_{\text{fail}}). \tag{12}$$

The three outputs are at three granularities. The trajectory caption states the intent the trajectory realises. The action caption segments the trajectory into sub-skill spans and names each span. The failure caption, emitted only for a failed rollout, states the root cause and is anchored at the flagged step $t^\star$, which is the same localisation signal Aim 1 uses [46]. The executed action sequence is the input that makes the inversion something other than video captioning: two trajectories can look nearly identical in pixels and differ in what the robot did, and proprioceptive and action signals are known to improve this class of caption and segmentation [87]. The captioner primitive is borrowed [87] and is not the contribution. What the captions are used for is the contribution. Every demonstration that enters the training set is captioned and its captions are stored, so the system accumulates a persistent, language-indexed record of what the policy has been taught, and the selector reads the coverage of that record before it prescribes.

The captioner. The captioner is a small vision-language model with an action channel. Keyframes are sampled from the trajectory, with the first frame, the flagged step and the final frame forced into the sample, and are encoded with a frozen pre-trained visual encoder of the kind already used for the image-modality policies [55, 66, 73]. The executed actions are quantised into tokens, interleaved with the visual tokens, and projected into a language backbone. The three caption heads are selected by a query token, so one forward pass serves all three. The failure head is where the two aims meet most directly: Aim 1's closed root-cause taxonomy was a compression forced by the absence of a memory, because a stateless pipeline can compare across rounds only by identity of labels. With a memory, comparison is by embedding and the vocabulary can be open.

Training the captioner. Manual captioning at the required scale is not affordable, so the training signal is constructed. Privileged simulator state supplies programmatic predicates, which a grammar renders into correct but stiff captions; a larger vision-language teacher paraphrases them into fluent language with decoding constrained to the template's facts; sub-skill spans come from change-point detection on the action, velocity and contact signals; and failure captions are built contrastively against the nearest successful demonstration of the same intent. The training objective adds to the language-modelling term a span-segmentation term, an alignment term that draws a caption and its trajectory together in a shared embedding space, and a fact-consistency term that re-parses the generated caption into predicates and penalises disagreement with the simulator's own. The alignment term is what makes the memory possible, because coverage is computed in the space it builds, and the fact-consistency term is what keeps a hallucinated cause out of it.

The coverage memory. The memory $M = \{(e_i, \ell_i, m_i)\}$ stores, for every demonstration in the training set, its caption embeddings e_i, its captions ℓ_i, and metadata m_i recording the task, the round of acquisition and the sub-skill spans it contains. Storage is at the trajectory level and at the span level, because a demonstration collected for one purpose usually teaches more than one thing. Coverage of a query embedding e_q is the mean similarity of the query to the memory under a kernel k,

$$\rho(e_q) \;=\; \frac{1}{|M|}\sum_{i=1}^{|M|} w_i\, k(e_q, e_i), \qquad w_i \;=\; 1 - \widehat{\mathrm{SR}}(e_i), \tag{13}$$

and the competence weight w_i is the part that matters. A skill can be present in the dataset and still not learned, so each stored skill is weighted by one minus the policy's measured success rate on it, and a skill that is present but taught badly still reads as a gap. A memory that conflated presence with competence would refuse to re-teach precisely the material that has been taught badly, and the data-quality literature is the reason to expect the two to come apart [6]. The coverage gap is the low-ρ region in the neighbourhood of the current failure's embedding, and it is what the selector is asked to fill. Aim 1's geometric descriptor is retained as one channel of the index rather than discarded, because Section 5.1.7.3 shows it produces the best-separated failure clusters and Section 5.1.7.4 shows where it stops working, which is where language is expected to earn its place. The coverage memory subsumes Aim 1's cluster memory as its degenerate case, and the mis-scaled kernel width does not survive the change, because coverage in an embedding space is not parameterised by a metric width that must be re-tuned against each task's reset range.

The memory-conditioned selector. One model does the reasoning that Aim 1 splits across three calls:

$$(P, R) \;=\; \mathrm{LLM}_\theta\big(\ell_{\text{fail}},\ \varphi,\ \text{Retrieve}(M, e_{\text{fail}})\big). \tag{14}$$

It consumes the failure caption, the geometric descriptor φ, and what retrieval over the memory returns for the failure's embedding: the nearest stored skills, their support counts and their competence weights. Retrieval over a structured store is standard and is used as such [22, 47]. The model emits the prescription P and a rationale R in language, which states why that demonstration and not another, and which the person who holds the budget can read. The claim is not that one model is better than three. The claim is that the selector's reasoning is stateful and coverage-aware, and the ablation that decides between the two readings is stated in advance in Section 4.2.4.

The outer protocol is Aim 1's, which is the point of the design: the same query gate, the same budget, the same retraining step. Both screens of Aim 1 are retained. Feasibility verification against the constraint store runs unchanged, because Section 5.1.7.2 measures what happens without it, and a prescription must still be checkable against what the environment permits before an expert is asked to satisfy it. The naming function of the knowledge-augmented graph is absorbed into the memory, because the failure-mode vocabulary becomes open language; its verification function is not absorbed. Policy solvability is the second screen and is unchanged.

4.2.3 Architecture

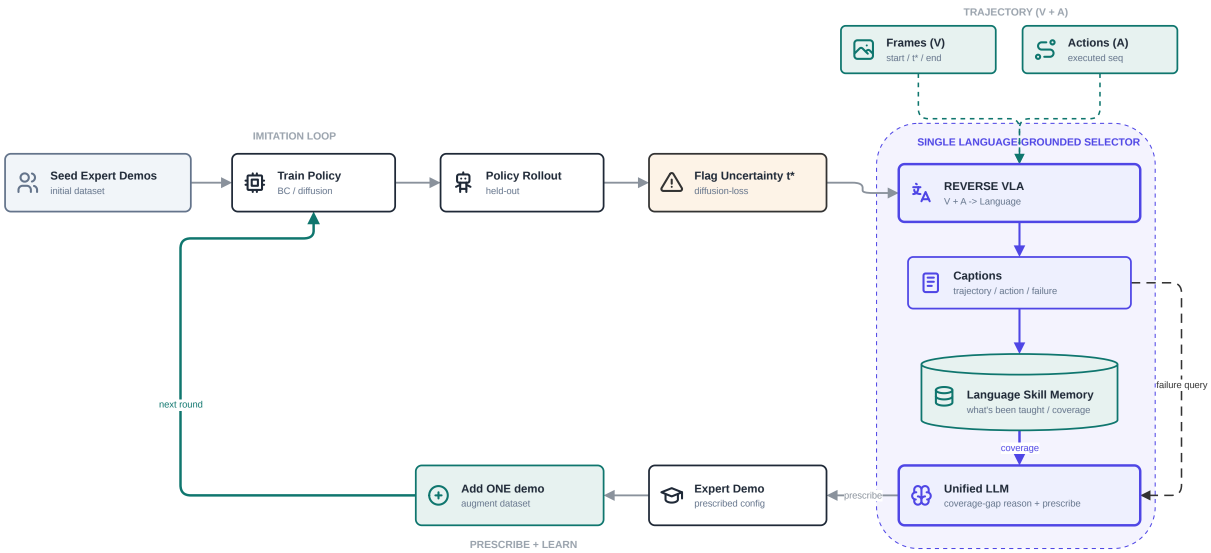


Figure 3. Proposed architecture for Aim 2. The imitation loop of Aim 1 is retained along the top left; the trajectory band supplies the frames and the executed actions the inversion consumes; the dashed enclosure holds the single language-grounded selector, in which the reverse vision-language-action model emits captions, the captions enter the language skill memory, and the unified model reasons over the coverage the memory returns together with the current failure.

The figure is drawn in three bands. The imitation loop along the top left is inherited from Aim 1 without change, and the figure keeps its four blocks to make visible that the protocol is unchanged. The trajectory band supplies the two inputs to the inversion, the frames and the executed action sequence, and the presence of the second is the whole difference between this and a video captioner. The selector, drawn as a dashed enclosure, holds the contribution: the reverse vision-language-action model maps frames and actions to language, the captions enter the language skill memory, the memory returns coverage, and the unified model holds two things at once, what has been taught and what has just gone wrong. The prescribe-and-learn band closes the loop along the bottom. The drawing does not yet carry the constraint store or the geometric channel of the fused index, both of which the method retains; the revision adds two edges and changes no block.

4.2.4 Evaluation strategy

The protocol is Aim 1's, unchanged, so that the head-to-head is a comparison of selectors and of nothing else. Push-T is carried forward as the continuity benchmark, because the Aim-1 numbers on it are directly comparable and a regression would be visible immediately. A graded manipulation suite [60, 100] is the sample-efficiency testbed, LIBERO [50] ships language instructions and a defined skill taxonomy that supply ground truth for caption scoring at no annotation cost, and Meta-World [95] provides a named skill inventory against which a claim of complementary rather than redundant selection can be checked. The primary metrics are demonstrations-to-threshold and the area under the success-versus-demonstrations curve, reported against Aim 1 over at least five seeds with confidence intervals. Secondary metrics are the final success rate, caption grounding scored as agreement between the generated caption's predicates and the simulator's, the redundant-demonstration rate, and the faithfulness of the rationale, tested by removing the coverage gap the rationale cites and checking that the selection changes. Baselines hold the policy and the retraining loop fixed and vary only the acquisition rule: passive behaviour cloning, random selection, the DAgger-family gates carried over from Aim 1 [34, 46, 64, 77, 99], and full DISEIL.

The decisive experiment is a matched-information ablation, and it is designed to be the first results figure of the Aim-2 paper. The selection loop is frozen and only the representation the selector consumes is varied, at equal capacity.

Table 5. The matched-information ablation. The selection loop is frozen and only the representation the selector consumes varies, at equal capacity.

Arm	Representation the selector consumes	What it tests
Generated captions	the captioner's output, indexed in language	the proposed method
Geometric descriptors	Aim 1's descriptor and its cluster signatures	whether language adds anything over Aim 1's representation
Learned trajectory embedding	a trajectory encoder trained at the same budget, no language	whether the gain is coverage in some space rather than in language
Scrambled captions	captions with their content permuted across trajectories	whether the selector uses caption content at all
Oracle captions	human-written captions	the ceiling the generated captions are measured against

The interpretation is committed before the experiment is run. The contribution holds only if generated captions beat both the learned embedding and the scrambled placebo, and trend toward the oracle. If the learned embedding matches the captions, the contribution is interpretability and cross-task composability rather than sample efficiency, Aim 2 is written as such, and Aim 3 proceeds on an embedding index. Three further ablations isolate the components: memory on against memory off, which should show its effect in the redundant-demonstration rate before it shows it in the success rate; a language-indexed memory against a raw-embedding memory, which separates the index from the memory; and the memory bolted onto three separate calls against the single unified model, which settles whether the unification is a mechanism or an engineering convenience.

The limitation Aim 2 will leave is stated here, because it is what makes Aim 3 the next step. The coverage memory is task-local: it records what one policy has been taught about one task, and a skill shared between tasks, such as the reach-and-align that precedes both a door pull and an insertion, cannot be credited across them. The supplier is a scripted expert who is always available and identically priced. Outside simulation the budget is not a count of demonstrations, it is a person's time, and demonstrations differ by an order of magnitude in what they cost that person to produce. The selector spends but does not price.

4.3 Aim 3. Demonstration demand across tasks, embodiments and teachers

4.3.1 The limitation Aim 2 leaves

Aim 3 is future work. Nothing in this section has been implemented and no number in it is a measurement.

Aim 2 gives the selector a memory of what it has been taught, and the memory is task-local. Skills are shared across tasks in a way that record cannot express: the reach-and-align that precedes a door pull is the reach-and-align that precedes an insertion, and under a task-local memory a demonstration collected for the first cannot be credited against the shortfall of the second. Generalist policies are trained on data pooled from many tasks and many robots precisely because that sharing exists [9, 42, 67, 68, 74]. What is pooled in that literature is supply, and the demand side has no representation at all.

The second limitation is the supplier. Aims 1 and 2 address a scripted or planner-based expert that answers instantly and charges the same for every question, and under that assumption the demonstration budget is a counter. Outside simulation the assumption fails on every clause. Demonstrations are produced by people, at a cost measured in minutes, and the cost varies by an order of magnitude with what is being asked: a short push and a long contact-rich insertion are one demonstration each and are not one price each. Large-scale collection efforts are budgeted in human hours and in operator interfaces, not in trajectory counts [41, 58]. A framework whose purpose is to spend a scarce resource well is therefore measuring the wrong resource. The Aim-2 selector spends but does not price: it can say which demonstration is most informative, and it cannot say what that demonstration is worth relative to what it costs.

The components of the answer exist and the pairing does not. Cross-embodiment data pooling trains one policy on many robots and holds no ledger of what the policy is short of [8, 42, 67, 68]. Cost-sensitive active learning weighs the value of a query against its labelling cost, for a datum that already exists [82]. Sub-trajectory retrieval shows that one collected trajectory serves several tasks, and it performs that crediting at consumption time, over a corpus that is already fixed [62]. Aim 3 addresses RQ3, stated in Section 3.2.

4.3.2 Proposed method

Aim 3 makes demonstration demand a priced object that transfers across tasks. Four components extend the components of Aim 2 rather than replacing them.

A cross-task skill inventory. Aim 2's captions are aggregated into a shared skill space, annotated with the embodiment on which each instance was demonstrated. A skill is a language-indexed cluster of sub-trajectory captions, for example aligning a gripper with a vertical handle and pulling along the hinge arc. It is neither a task nor a trajectory. Coverage is measured over the inventory, so the question of whether the policy can align with a handle is answerable without reference to the task in which the handle appeared. Open-ended skill libraries built by a language model are the nearest existing object [90], and benchmark suites with a named skill taxonomy supply the ground truth against which an inventory can be scored [50, 95]. The inventory is where the language index must earn the claim Aim 2 makes for it, because Aim 1's geometric descriptor does not compose across tasks: its coordinates are defined against one task's objects and one task's reset distribution.

A demand model with a price. For each skill in the inventory the demand model maintains a shortfall, the distance between the policy's competence on that skill and what the task family requires, weighted by how often the skill lies on the critical path of a task the policy is currently failing. Every candidate request then carries two numbers. The first is an expected information gain. Aim 1 measures information gain after a demonstration has been collected, as the per-step loss on it before retraining, and Aim 3 predicts the same quantity before collection, from the current coverage of the requested skill and the policy's measured competence on it. Aim 1's measurements are the training data for that predictor, which is the most direct link between the three aims: one quantity, measured in Aim 1, contextualised in Aim 2 and predicted in Aim 3. Expected information gain is the standard way to price a query before it is answered [35, 82], and what is new is the object being priced. The second number is an expected human cost, in minutes of teacher time, estimated from the length and difficulty of comparable demonstrations already collected. Selection maximises expected information gain per unit of teacher time, and the budget stops being a count of demonstrations and becomes a time budget, which is what it always was outside simulation.

A non-expert teaching interface. A demand is rendered as a request a person can act on: a natural-language instruction, a scene specification, and the reason the demonstration is being asked for. The scene specification has already passed the feasibility check, which is the propose-verify-revise loop Aim 1 runs against the knowledge-augmented graph and Aim 2 retains for exactly this moment [16, 51, 89], so no request is issued that the environment cannot instantiate or the robot cannot reach. Aim 1's solvability screen is retained and acquires an economic reading: a request the current policy can already satisfy wastes a person's time, and a framework that prices human time cannot afford to issue one. A non-expert demonstration also breaks the second half of Aim 1's information-gain argument, because a high pre-retrain loss on a non-expert demonstration is ambiguous between novelty and incompetence, so the loop requires a quality filter and the demand model must be able to reject a satisfied request. Learning from suboptimal and preference-based human input is the starting point for that filter [12, 18, 97].

Transfer credit. An arriving demonstration is captioned by the Aim-2 captioner, decomposed into its sub-skill spans, and credited against the outstanding shortfall of every task in the inventory that those spans partially satisfy. A demonstration requested to close a door-opening shortfall reduces the alignment shortfall of an insertion task, and the ledger records the reduction. Crediting at collection time is a different operation from retrieval at consumption time, because it can change what is collected next. Transfer credit is the mechanism by which a fixed number of human hours buys more competence than the same hours spent one task at a time.

The loop, for the algorithm float that will accompany the Aim-3 paper: roll out the generalist policy across the task family; caption the failures; update the competence estimate for every skill in the inventory; compute each skill's shortfall; price every candidate request by predicted information gain per unit of teacher time; verify the highest-value request against the constraint store and against policy solvability; issue the request to a teacher; caption the returned demonstration; credit it against every task whose shortfall it reduces; aggregate and retrain.

4.3.3 Component lineage across the three aims

Table 6 is the lineage the panel should be able to check. Each row is one object followed through the programme, and no row begins in Aim 3.

Table 6. The lineage of each component across the three aims.

Component	Aim 1	Aim 2	Aim 3
Index of failures	6-D geometric descriptor, one task, one round	caption embedding fused with the geometric descriptor	skill inventory, shared across tasks and embodiments
Memory	configurable penalty over corrected cluster centroids, active where failures recur	coverage of what the dataset contains, weighted by competence	demand ledger: shortfall per skill, settled by transfer credit
Value of a demonstration	measured after the fact as the pre-retrain per-step loss	conditioned on what the dataset already holds	predicted before collection, divided by expected teacher time
Screening	feasibility against the knowledge graph, and policy solvability	both retained	both retained; solvability becomes a cost argument
Supplier	scripted or planner-based expert	scripted or planner-based expert	non-expert human, whose time is the budget

4.3.4 Evaluation strategy

Benchmarks are multi-task suites with a defined skill taxonomy, so that coverage is measured against a ground-truth inventory instead of against the system's own captions [38, 50, 61, 95], together with cross-embodiment evaluation on pooled multi-robot data, to test whether a skill demanded on one embodiment can be satisfied on another [68]. The primary metric is teacher-time-to-threshold, the number of minutes of human demonstration time required to bring the policy family above a target success rate. Demonstrations-to-threshold is reported beside it, and the gap between the two curves is itself a result, because a framework that reduces the demonstration count while raising the cost per demonstration has achieved nothing. Three further quantities are reported: transfer credit, in tasks advanced per demonstration; the rate at which a non-expert can act on a generated request; and the calibration of the price, as predicted against realised information gain, which is the successor of Aim 1's prescription-confidence measurement in Section 5.1.6. Controls remove one component of the proposal each: per-task demand with no transfer credit, uniform demand across skills, demand without a price, and Aim-2 single-task selection, which keeps the chain of comparisons unbroken from Aim 1 through Aim 3. The DAgger-family gates remain the outer reference point [46, 77].

The human study is the evaluation the Aim-3 claim depends on and the first point in the programme at which humans enter. Non-expert participants satisfy generated requests in simulation, using a teleoperation interface of the kind established for crowdsourced demonstration collection [52, 58]. Ethics approval will be sought from the Deakin human-research ethics committee before any participant is recruited, and the arrangements are set out in Section 7.2. The study makes the programme's central claim checkable from outside it: a person who has never read the thesis is handed a request, satisfies it, and the policy improves by approximately the amount the demand model predicted. If it does not, the calibration curve says so.

Two of Aim 1's components carry all the way through, and they are the reason the programme is one programme rather than three papers. The constraint store, which Aim 1 uses to verify a prescription before an expert is called, is what makes it possible to hand a request to a person without wasting their time. The geometric descriptor, which the purity diagnostic shows is semantically blunt where configuration does not determine cause, is what keeps language from collapsing kinematically distinct skills into one entry of the inventory. The value of one demonstration is measured in Aim 1, contextualised in Aim 2, and priced in Aim 3.

5. Progress report

5.1 Progress on M1

Section 4.1 states the framework. This section reports what was built from it, what the first evaluation returned, and what was added to the system afterwards. The order of the account is the order of the work: the framework was implemented on one task, extended to five, compared against six demonstration-acquisition methods, and then taken apart one component at a time to find out which part of it carried the result. The ablation programme was designed to be able to retire components, it recast one as an optional feature and reduced the claims made for three others, and it located the advantage in a component the framework had treated as an implementation detail. All of that is reported below, including the parts that did not survive.

5.1.1 Implementation

The implementation runs the four stages of Section 4.1.3 as a loop around an otherwise standard interactive imitation-learning harness. A round rolls out the current policy on the held-out evaluation set, records the failures, computes the geometric descriptor of each one, partitions the failures, applies the cluster memory, calls a vision-language model on the frames of each cited failure, calls a reasoning model to assign a root cause, calls a prescription model to turn the selected mode into a demonstration request, checks the request against the task's knowledge-augmented graph, and hands the surviving request to the expert. Every run stores its exact prompts and replies, so the material quoted below is a record and not a reconstruction.

The three model calls. The perception call takes a system instruction and three frames of a cited failure: the start of the episode, the flagged step of Equation 5, and the final step.

```
SYSTEM:  You are analysing a robot manipulation failure from rendered frames. Be
         concrete and spatial; describe what you actually see, not generic advice.

USER:    You are analysing a robot manipulation failure. The attached frames are, in
      order: start, high_loss, end (the peak-loss frame is the policy's most-uncertain
         step, t*=146).
         Task: A UR5e robot with a wiping pad (no fingers) must WIPE a trail of dirt
        markers off the table by pressing down and sweeping along the dirt path until
         all markers are cleared (coverage).
       Describe what went wrong. Focus on: where in the trajectory the failure occurs,
        the robot/gripper configuration at peak loss, and what object or contact state
         caused it. ~120 words, concrete and spatial.
         [start frame] [high_loss frame] [end frame]
```

The model's reply on that instance, from a Wipe run, reads: *"The failure occurs during the sweep at the beginning of the trajectory, around step t*=146. At peak loss, the robot's wiping pad is positioned at the start of the dirt trail, but the pad is tilted and not fully in contact with the table surface. The gripper is misaligned, causing the pad to lift slightly and only partially engage the dirt. This lack of stable contact prevents effective wiping, leaving the initial segment of the trail untouched."* One defect in the template is visible in the quotation and is recorded here. The template names the anchor the "peak-loss frame", while the frame actually passed is the first-crossing step of Equation 5. The wording is stale and the frame is correct.

The reasoning call is constrained to the vocabulary of the task's graph and to strict JSON.

```
SYSTEM:  You are a robot-manipulation failure analyst. Classify the root cause and
         trajectory phase using ONLY the provided categories and the KAG facts.
         Output strict JSON, no prose, no code fences.

USER:    TASK: {task_description}
         {kag_text}
         VLM FAILURE DESCRIPTION (the only visual evidence): {vlm_report}
         Identify the root cause category and the trajectory phase where the failure
         occurred.
         root_cause in [grasp_failure, approach_failure, placement_error,
                        contact_instability, pose_mismatch, timeout]
         phase      in [pre_grasp, grasp, transport, placement, insertion]
         Output ONLY this JSON:
         {"root_cause": "<one of the categories>", "phase": "<one of the phases>",
        "rationale": "<one sentence grounded in the VLM description and a KAG fact>"}
```

Its reply on the same failure is the grounding requirement in miniature, because the rationale cites both the visual evidence and a named node of the graph.

```
{"root_cause": "contact_instability", "phase": "transport",
 "rationale": "The wiping pad tilts and lifts during the sweep (transport phase) as
 described in the VLM evidence, causing intermittent contact with the table surface and
 aligning with the KAG-reasoned 'contact_instability' failure due to loss of downward
 pressure/alignment."}
```

The prescription call states the budget rule to the model, offers the two arms of Section 4.1.3, and requires a confidence line.

```
SYSTEM:  You are a demonstration coach for an interactive imitation-learning loop. Each
         round you spend ONE expert demonstration to fix the dominant failure mode. You
         decide HOW to spend it, grounded in the KAG facts and the per-failure analyses.
         Reason briefly, then end with EXACTLY two lines: (1) a decision line in the exact
            required format, and (2) a confidence line 'CONFIDENCE: <integer 0-100> -
<one-line
         rationale>' reporting how confident you are that this demonstration will improve
            the policy.

USER:    (A) SELECT ep<ID> - one recorded failure represents the whole mode. That exact
            scene is re-run and the expert corrects it on-policy from the divergence point
                t*. Use when the cluster is TIGHT or one failure clearly dominates.
         (B) BRIDGE ep<ID>,ep<ID> - no single failure covers the mode. Prescribe ONE new
            object placement in the MIDDLE GROUND between 2-3 cited failures (e.g. failures
             at (1,1) and (5,5) -> a demo near (3,3)); the expert demonstrates from there.
                Use when the members are geometrically SPREAD but share a root cause.
```

The cited failures are handed to the model one line each, and the model answers in two lines. The round below is taken verbatim from a Wipe log.

```
DOMINANT FAILURE CLUSTER (members with their VLM+analysis findings):
  - ep3000001: object_xy=(0.221,-0.027) progress=74/500  peak_loss=0.0434
               root_cause=contact_instability phase=transport
  - ep3000003: object_xy=(0.165,-0.068) progress=146/500 peak_loss=0.0181
               root_cause=contact_instability phase=transport
  - ep3000006: object_xy=(0.123,-0.157) progress=234/500 peak_loss=0.0186
               root_cause=pose_mismatch         phase=placement
===== RESPONSE =====
SELECT ep3000001
CONFIDENCE: 85 - Ep3000001's contact_instability root cause (higher peak_loss) at early
progress best represents the dominant failure mode, ensuring the expert demonstration
directly addresses unstable wiping pressure causing missed coverage.
```

That record also shows the imperfection of geometric partition from the inside. Two of the three cited members carry the label `contact_instability` and the third carries `pose_mismatch`, so this mode has a purity of two thirds, and the purity measured across the ablation settings is an average over instances like it. The confidence line is not decoration: its correlation with the improvement that the prescribed demonstration actually produces is reported in Section 5.1.6.

The environmental constraints. The knowledge-augmented graph of a task is a JSON document with a fixed schema: metadata, typed nodes with key-value properties, relations between them, and a block of reasoning implications, one per failure mode plus a workspace constraint and a non-emptiness rule. A renderer turns the document into the text block injected into the reasoning and prescription prompts. The graph is authored once per task, and its constraints are measurements of the environment rather than opinions about it.

Push-T stores its bounds as typed workspace nodes and its controller as a node in its own right.

```
{"id":"ws_tee","type":"Workspace","label":"Reliable tee init range",
 "properties":{"x":[-0.20,0.20],"y":[-0.25,0.05],"z":0.021}},
{"id":"ws_tcp","type":"Workspace","label":"Reliable tcp range",
 "properties":{"x":[-0.35,0.35],"y":[-0.35,0.35],"z":[0.02,0.08]}},
{"id":"ctrl","type":"Controller","label":"pd_joint_pos / rel_joint_pos",
 "properties":{"policy_action":"7 joint deltas (rel_joint_pos)",
               "expert_action":"PPO -> joint_delta_pos (same 7-joint space)"}}
```

The predicate that Equation 10 checks is stored as an implication and is written in the imperative, because it is addressed to the model as much as to the checker.

```
"workspace_constraint": "Every prescribed config MUST keep tee_xyz within
x[-0.20,0.20] y[-0.25,0.05] z=0.021 and tcp_xyz within x[-0.35,0.35] y[-
0.35,0.35] z[0.02,0.08]; out-of-range poses are dropped (the PPO expert is
unreliable there) and waste the round."
```

Door's constraint is tighter by an order of magnitude, and the numbers are a padded empirical measurement of the simulator's own reset sampler, so that a prescribed configuration cannot leave the task's native reset distribution.

```
{"id": "ws_door", "type": "Workspace", "label": "Reliable door-frame range",
 "properties": {"x": [-0.135, -0.108], "y": [-0.366, -0.340], "z": 1.10,
                "yaw_rad": [-1.82, -1.57]}},
{"id": "succ", "type": "SuccessCondition", "label": "Door open",
 "properties": {"metric": "hinge_qpos > 0.3 rad", "info_key": "success"}}
```

For a discrete task the environmental constraint is not a bounding box but a reachability predicate, and GridWorld's graph states it as one: a prescribed layout must place the start cell, the goal cell and the three obstacle cells as distinct in-grid cells, with a minimum separation between start and goal and a breadth-first path from one to the other that avoids the obstacles, and a layout that fails the predicate is rejected before it reaches the expert.

Wipe shows the store doing something a bounding box cannot do at all. Its graph carries an implication that removes an entire arm of the prescription from the task.

```
"select_only": "Wipe randomizes a whole marker PATH, not a single object
pose, so BRIDGE is infeasible - always choose SELECT of the most represen-
tative failed episode."
```

The planner reads that implication structurally and the prompt omits the bridging option. The graph therefore does two jobs. It constrains where a demonstration may be placed, and it determines which prescription arms exist for a task at all. Both jobs are done by knowledge that is written down and checkable, which is the sense in which the framework's model of the environment is explicit rather than implicit in a network's weights.

5.1.2 Experimental setup

Every concrete value in the Aim-1 evaluation is fixed in this section and appears nowhere else. The framework is stated over symbols: a budget B of expert demonstrations, D demonstrations acquired per round, a policy f_θ with a per-step loss ℓ_t. What follows is the instance of that framework which was run.

Tasks, observation modalities and settings. A *setting* is one task under one observation modality. The evaluation covers five tasks under two modalities, state and image, which gives ten settings. The word *mode* is used in this report only for a failure mode, which is a cluster of failures that the framework discovers; an observation modality is never called a mode.

GridWorld 5x5 is a discrete navigation task on a five-by-five grid with three obstacle cells, in which an agent must reach a goal cell from a start cell. A* search and breadth-first search enter the task only as the feasibility and path-validity checker that decides whether a prescribed grid configuration admits an obstacle-free route from start to goal [19, 30]; they are never used as the expert, and no policy is trained on their output. Push-T [17, 25] is a planar pushing task, in which a manipulator must push a T-shaped block into a fixed goal pose. The task originates in the implicit-behaviour-cloning work that introduced it [25] and was popularised by the diffusion policy [17]; the implementation used here is ManiSkill3's PushT-v1 [88], which the benchmark documents as the simulated version of that task. ManiSkill3 is the third release of a benchmark line whose earlier releases carry a different task suite and do not contain Push-T [29, 65], so the simulator and the task are cited separately. Lift, Wipe and Door are RoboSuite manipulation tasks on a UR5/UR5e arm [100]: lifting a cube from a table, wiping a randomised trail of dirt markers from a surface, and pulling a door open past a hinge threshold. The three RoboSuite tasks and Push-T supply the continuous-action half of the evaluation, and their reset distributions differ by an order of magnitude in width.

The expert differs by task, and the report states which it is in each case, because the claim that the demonstrations are correct by construction depends on it. On GridWorld the expert is a human. On Lift, Wipe and Door the expert is a scripted oracle: an open-loop motion-planner routine on Lift, a closed-loop routine reading the hinge angle on Door, and a scripted wiping routine over the sampled marker path on Wipe. On Push-T the expert is a policy trained by proximal policy optimisation [80], a standard reinforcement-learning algorithm used here without modification, so the Push-T expert is learned rather than scripted. It is an expert in the sense the framework requires, namely a demonstrator whose trajectories are the target the policy is fitted to, and it is not uniformly competent: the trained policy pushes in one rotational direction only, so configurations that require the opposite rotation lie outside what it can demonstrate. Those configurations are excluded by the workspace constraints stored in the task's knowledge-augmented graph.

Policy instantiations. The framework requires only that the policy expose a per-step loss, and it is instantiated with three policy classes to make that requirement visible. GridWorld under the image modality uses a convolutional network. GridWorld under the state modality uses a multilayer perceptron. The four robot tasks use diffusion policies under both modalities, with an R3M visual encoder supplying the image branch [17, 66]. R3M supplies the policy's visual representation and nothing else. It does not supply the clustering features, which are geometric in every run, state and image alike.

Budget, rounds and seeds. The validated instance is $B = 20$ and $D = 1$. Each round rolls out the current policy on a frozen held-out evaluation set, analyses the failures, and acquires one expert demonstration, which is added to the training set. Twenty rounds therefore consume twenty demonstrations. Study A11, in Section 5.1.7.3, sweeps B over 10, 20 and 40 and is the evidence that $B = 20$ is an instance and not a requirement.

The retraining cadence is a property of the task and not of the framework, and the runs reported here do not use one cadence everywhere. On GridWorld the policy is retrained from scratch after every round, so at $D = 1$ the framework analyses a freshly retrained policy twenty times. On the four robot tasks, where a diffusion policy is expensive to fit, retraining runs once every fourth acquired demonstration, so at $D = 1$ the policy is refreshed five times over the budget and the twenty rounds are twenty fresh rollout pools analysed against a policy that changes every fourth round. The retraining step of Algorithm 1 therefore reads "at the per-task cadence", and the two cadences hold for every arm of the comparison, DISEIL and baselines alike, so no arm is advantaged by the schedule.

Seed counts are not uniform, and the asymmetry is stated. GridWorld is run with nine seeds; the four robot tasks are run with five. The round accounting confirms both counts independently: clustered rounds plus skipped rounds total 180 for each GridWorld setting and 100 for each robot setting, which is seeds times B in both cases.

Initial demonstrations and starting performance. Before the first round, the policy is trained on an initial demonstration set that is excluded from the budget. That set is per task, not uniform: it holds twenty demonstrations on GridWorld and Push-T, twelve on Wipe, eight on Lift and four on Door, the counts N_i reported in Table 7. The initial demonstrations were chosen so that each task's round-zero success rate, the held-out success rate before any DISEIL round, exceeds roughly 45 per cent: 47.0 on GridWorld, 46.2 on Push-T, 45.2 on Wipe, 67.2 on Lift and 56.8 on Door, which is the Init SR column of Table 7.

The count is not a free parameter, and the reasoning behind it is the first half of the information-gain argument of Section 5.1.5. A policy's starting success rate has to sit inside a band for the experiment to mean anything. If the initial policy is too weak, its rollouts fail everywhere, every configuration is a failure, the failure set carries no structure for the descriptor to separate, and there is no allocation problem to solve. If the initial policy is too strong, the failure set is empty or nearly so, the budget has nothing to allocate, and every method converges to the same place. The band between those two conditions is the regime in which a fixed budget of demonstrations can be spent well or badly, which is the regime the framework exists for. Demonstration count and coverage are known to govern imitation-learning performance directly [49], so the count is the lever that places a task inside the band.

The principle is implemented as a behaviour-cloning data-scaling sweep. A pool of expert demonstrations is collected, behaviour cloning is trained on each nested prefix of the pool, each prefix is evaluated on the frozen held-out set, and the prefix whose round-zero success rate is closest to a target of roughly 50 per cent is selected. That sweep sets the initial counts N_i used in every run that produces Table 7: twenty demonstrations for GridWorld and Push-T, twelve for Wipe, eight for Lift and four for Door, with round-zero success rates of 47.0, 46.2, 45.2, 67.2 and 56.8 per cent respectively. Every task therefore begins the budget above the roughly-45-per-cent floor, with enough competence to produce meaningful rollout failures and enough headroom for the budget to matter. Lift begins the furthest above the target, at 67.2 per cent, because the smallest prefix that trains a stable policy on that task already clears the band, which is why Lift is the task in Table 7 that begins the budget closest to a perfect success rate.

Baselines. Six comparison methods are run, described qualitatively; their hyperparameters are not reproduced here, and the mechanics of each belong to Chapter 2.

Five of them are published interactive imitation-learning methods and share one skeleton: roll out the current policy, read a scalar signal, hand control to the expert when the signal crosses a threshold, aggregate the expert's labels and retrain [77]. They differ only in the signal. SafeDAgger learns a classifier that predicts when the policy is about to deviate from the expert [99]. DropoutDAgger reads the spread of a Monte-Carlo dropout ensemble of the learner's action distribution [28, 63]. EnsembleDAgger reads the variance of an explicit ensemble, combined with an action-discrepancy term [44, 64]. ThriftyDAgger combines a novelty estimate with a learned risk estimate under a target switching rate [34]. Diff-DAgger uses a diffusion policy's own per-step training loss as the uncertainty signal [46], and is run on the robot tasks only, where the policy is a diffusion policy. Those five are the DAgger family, and they are labelled as such in every comparison table. Diff-DAgger's use of the per-step diffusion loss as an uncertainty signal is its own contribution; DISEIL uses that signal for failure localisation and also compares against it as a baseline, and both facts are stated plainly.

The sixth comparison method, Stagger, is not a published system. It is a uniform-random control implemented in this project: each round corrects one uniformly chosen recorded failure, with no gate, no descriptor and no allocation. It carries no citation, it is never labelled as a member of the DAgger family, and it is reported on GridWorld in the main table. Its extension to the robot tasks is study A2 of Section 5.1.7.2, where it answers the most damaging objection available against this work, which is that any failure replay would do.

Metrics. Three quantities are reported. The primary metric is the final success rate on the frozen held-out evaluation set after the budget is exhausted, in per cent, averaged over seeds.

Per-demonstration information gain is the policy's per-step loss on a newly acquired demonstration, measured before the policy is retrained on that demonstration. The quantity is the same per-step diffusion loss that Diff-DAgger uses as its gate signal [46], evaluated on a datum and not on a rollout step. The retraining cadence enters the measurement: on GridWorld the scoring policy is the policy of the round, and on the robot tasks it can be up to three demonstrations stale, because retraining runs every fourth demonstration there. Each cell of Table 8 pools between 168 and 184 loss records. A GridWorld setting acquires nine seeds times twenty demonstrations, which is 180, and one loss record per demonstration reproduces that range; a robot setting acquires 100 demonstrations, so a robot demonstration must contribute more than one record, and the source does not record the decomposition. The discrepancy is carried as an outstanding item in Section 6.1 and is not resolved by an assumption here.

ΔSR is the change in the policy's success rate on the round-level rollout evaluation, measured before and after a round. It is a per-round quantity, not a final one, and it is the outcome against which a prescription's reported confidence is scored.

5.1.3 Discovered failure modes

The partition step produces the object the whole framework allocates over, so the first thing the evaluation had to show was what that object actually is on a real task. Figure 4 shows the three failure modes discovered on Push-T (image), with three sampled members of each. The modes are behaviourally distinct: the block is brought to the goal region but left almost fully inverted; the arm never establishes a working contact; the block is pushed but abandoned at a moderate orientation error and far from the end-effector. The clusters are found from geometry alone, and the naming pipeline of Section 4.1.3 supplies their labels from the task graph's own vocabulary.

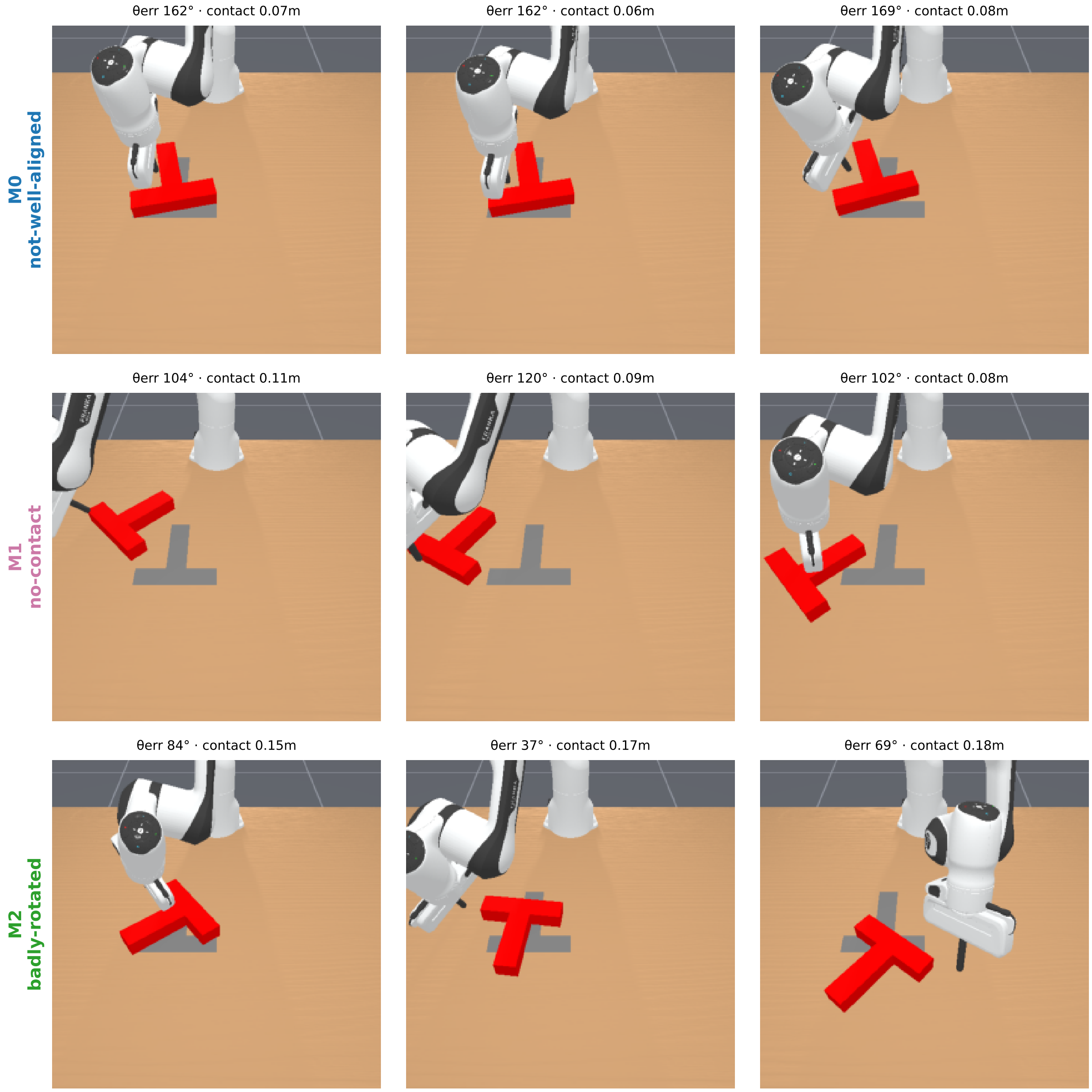


Figure 4. Failure modes discovered on Push-T (image). Each row is one geometric cluster of the round's recorded failures, with three sampled members. Labels are assigned by the naming pipeline from the task graph's vocabulary.

Three is not the number of failure modes on these tasks. It is the count most often selected when the silhouette criterion runs, and the count varies by round; the distribution is reported as study A15 in Section 5.1.7.3. How well the geometric partition agrees with the reasoning model's root-cause labels is measured as study A14 in Section 5.1.7.4, and the agreement is high but not perfect.

5.1.4 Main comparison

Table 7 gives the final held-out success rate in all ten settings. DISEIL attains the highest mean in every one of them.

Table 7. Final held-out success rate (per cent), mean ± standard error over 5 seeds (robot tasks), 9 seeds (GridWorld); Ni = initial demonstrations; Init SR = round-0 held-out success rate; best per row in bold. The budget is twenty expert demonstrations in every setting. Safe, Dropout, Ensemble, Thrifty and Diff-DAgger are the five published query-gated methods of the DAgger family. Stagger is a uniform-random allocation control implemented in this project and is not a DAgger-family method. Diff-DAgger is run on the robot tasks only; Stagger is reported on GridWorld only.

Task	Obs	Ni	Init SR	Safe	Dropout	Ensemble	Thrifty	Stagger	Diff-DAgger	DISEIL (ours)
GridWorld 5x5	state	20	47.0	85.3±0.9	84.9±0.8	86.2±0.7	86.8±0.7	85.7±0.5	—	**92.4±0.4**
GridWorld 5x5	image	20	47.0	88.8±0.9	88.4±0.7	88.8±0.9	88.7±0.6	89.1±0.8	—	**91.3±0.6**
Push-T	state	20	46.2	82.0±3.0	84.8±2.7	85.9±2.6	83.2±3.2	—	94.1±2.0	**96.1±1.6**
Push-T	image	20	46.2	78.1±3.5	82.1±3.1	83.2±3.0	79.3±3.6	—	89.0±2.1	**92.6±2.2**
Lift	state	8	67.2	99.2±0.7	99.2±0.4	99.2±0.4	100.0±0.0	—	99.2±0.4	**100.0±0.0**
Lift	image	8	67.2	99.6±0.4	97.2±1.6	98.8±0.7	99.6±0.4	—	99.6±0.4	**100.0±0.0**
Wipe	state	12	45.2	88.0±1.1	88.6±1.8	86.8±1.9	89.0±1.1	—	90.4±2.7	**93.1±1.3**
Wipe	image	12	45.2	69.6±2.4	83.2±3.0	84.4±3.2	69.2±4.0	—	88.6±1.4	**92.3±1.4**
Door	state	4	56.8	91.8±2.1	92.5±1.2	88.8±3.1	89.6±1.7	—	93.2±1.9	**96.6±1.9**
Door	image	4	56.8	82.4±1.4	81.8±1.5	83.0±4.9	82.8±1.2	—	84.2±1.6	**88.6±1.5**

The margin over the strongest baseline in each setting averages 2.80 points, with a standard deviation of 1.73 and a range from 0.0 to +5.6. Which baseline is strongest varies: Diff-DAgger on Push-T, Wipe and Door under both modalities; ThriftyDAgger on GridWorld (state); the Stagger control on GridWorld (image); and a tie on the two Lift settings, at 100.0 with ThriftyDAgger on the state setting and at 99.6 on the image setting. The comparison is therefore against a moving target, and DISEIL is ahead of whichever method happens to be best in each setting.

Ten wins from ten is a pattern, and the pattern, and not any individual comparison, is what the aggregate test converts into a number. Treating the ten settings as paired observations, a sign test and a Wilcoxon signed-rank test both reject a coin-flip ranking at two-sided $p = 0.002$, which is the smallest p-value attainable with ten pairs and is therefore the floor of what this design can produce.

That figure should not be led with, and the reason is a defect in the design. The ten settings are not ten independent experiments. They are five tasks under two observation modalities, and the two modalities of a task share the expert, the reward structure and the reset distribution, so they are correlated by construction and the effective sample size is nearer five than ten. Collapsing to the five task means, the paired differences are +3.90 (GridWorld), +2.80 (Push-T), +0.20 (Lift), +3.20 (Wipe) and +3.90 (Door). The sweep holds at five from five. A one-sided sign test gives $p = 0.031$, which is its floor at $n = 5$, and a paired t-test over the same five means rejects at $t(4) = 4.10$, $p = 0.015$ two-sided. The two-sided nonparametric test does not reject at $n = 5$ ($p = 0.063$), and it cannot, because 0.063 is the smallest value it can return with five pairs.

The claim of record is the collapsed one. DISEIL attains the best mean success rate in all ten settings, and the aggregate advantage is significant under the conservative task-level analysis. What carries that claim is the sign of the margin and not the size of any one of them. Table 7 shows the seed standard errors of the two arms overlapping in several rows, so those rows would not support the claim on their own, and the rows are not independent of one another either. The systematic direction across the rows is the whole of the evidence, and the pooled tests above are what convert it into a number.

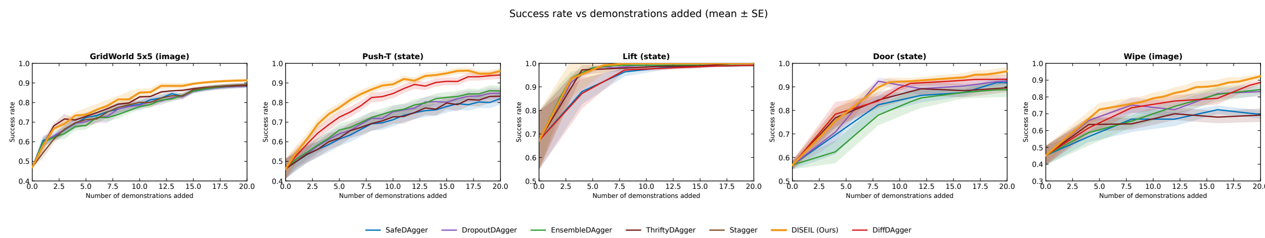


Figure 5. Success rate against the number of demonstrations added, on five tasks. One setting per task, with the observation modality printed in each panel title: GridWorld (image), Push-T (state), Lift (state), Door (state) and Wipe (image). Lines are means over seeds and shaded bands are one standard error (5 seeds on the robot tasks, 9 on GridWorld).

The learning curves say two things the final numbers do not. The separation between DISEIL and the DAgger family opens early on Push-T, from about the fifth demonstration, and holds thereafter, which is consistent with the budget sweep of Section 5.1.7.3: the advantage is a coverage-rate advantage and it is paid out at the front of the budget. On GridWorld every method rises together and finishes bunched, and Table 7 puts the whole image column between 88.4 and 91.3, which is why the GridWorld (image) margin, +2.2 points, is among the smallest in the table. The task is small enough that twenty demonstrations approach what any allocation rule can extract from it.

The third observation is a limitation and is reported as one. On Wipe (image), DISEIL and the strongest baseline are both still rising at the twentieth demonstration. Neither curve has plateaued inside the budget, so the +3.7-point claim on that setting rests on the final gap, 92.3 against 88.6, and not on a demonstrated asymptote. A longer budget could close it, and it has not been shown to vanish.

5.1.5 Per-demonstration information gain

The comparison establishes that the framework wins. The next question the evaluation had to answer is whether the demonstrations it buys are different from the demonstrations the query gates buy, and per-demonstration information gain is the measurement that answers it. It is the current policy's per-step loss on a newly acquired demonstration, evaluated before the policy has been retrained on it. The intuition is the standard one from active learning, where a datum on which the current model incurs a large loss is the datum whose acquisition is expected to change the model most [35, 82]. What is new here is not the measure. It is what the measure licenses once the acquisition pipeline is known.

Table 8 gives the mean gain per setting against Diff-DAgger, which is the reference the comparison has to be made against: its gate signal is the same per-step diffusion loss that this metric is computed from, so it is the one baseline that selects on the quantity being reported and the one that could be expected to lead on it. It does not. DISEIL acquires demonstrations of higher pre-retrain loss than Diff-DAgger in every one of the eight settings in which Diff-DAgger runs. On the two GridWorld settings the policy is not a diffusion policy, so Diff-DAgger does not run and its cells are empty. The four other query gates and the uniform-random control are not reproduced as columns, because the comparison against them adds no argument that the Diff-DAgger column does not already make: DISEIL's gain is above every one of them in every one of the ten settings, GridWorld included.

Table 8. Per-demonstration information gain (mean ± standard error). The policy's per-step loss on each newly acquired demonstration, measured before retraining on it; the error is the standard error over 5 seeds (robot tasks) and 9 seeds (GridWorld), and Figure 6 shows the full per-demonstration distribution. Diff is Diff-DAgger, which is run on the robot tasks only, so its GridWorld cells are empty. The three ablation settings, GridWorld (image), Push-T (state) and Door (image), are rows of this table.

Task	Obs	Diff	DISEIL
GridWorld 5x5	state	—	**3.55±0.84**
GridWorld 5x5	image	—	**3.21±0.78**
Push-T	state	1.57±0.49	**2.81±0.93**
Push-T	image	1.80±0.49	**2.82±0.77**
Lift	state	1.61±0.50	**2.64±0.74**
Lift	image	1.36±0.38	**2.93±0.75**
Wipe	state	1.43±0.36	**2.91±0.90**
Wipe	image	1.95±0.52	**3.62±0.98**
Door	state	1.84±0.50	**3.43±0.95**
Door	image	1.58±0.41	**3.00±0.89**

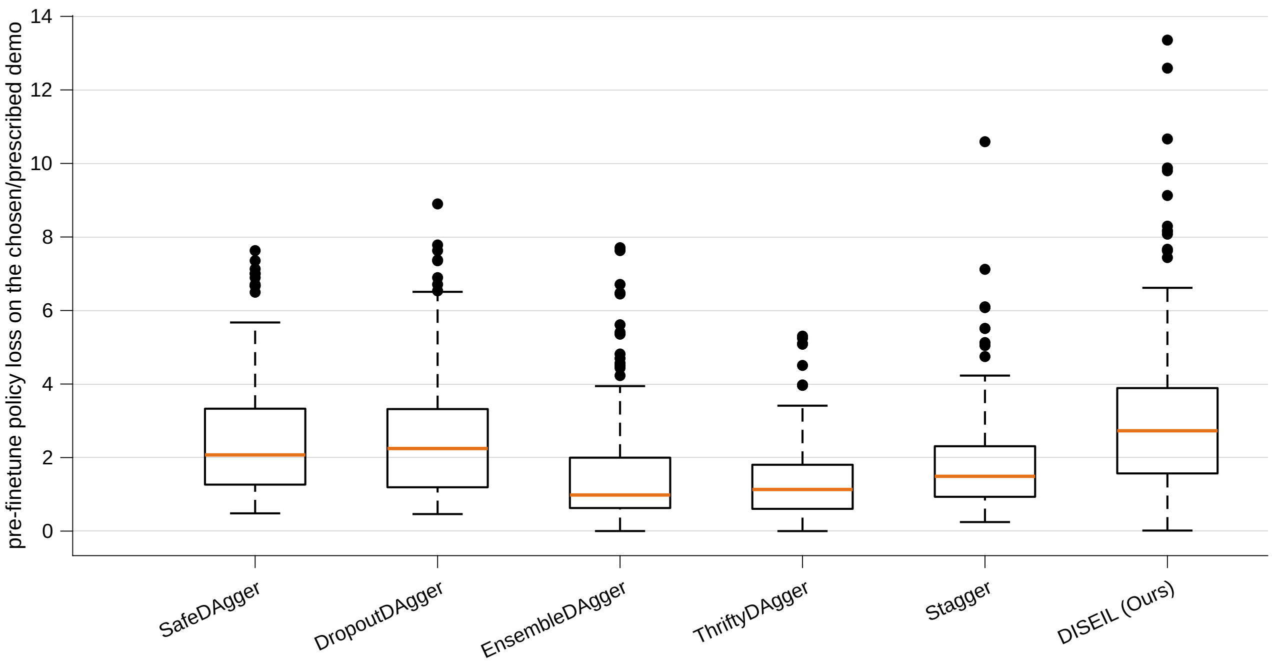


Figure 6. Per-demonstration information gain on GridWorld (image). Each box is the pre-retrain per-step loss of the current policy on the demonstration that method acquired in that round, over nine seeds and twenty rounds. Diff-DAgger is a robot-task baseline and does not appear on a GridWorld setting.

What a high pre-retrain loss means. A high pre-retrain loss on a demonstration admits exactly two readings. Either the demonstration covers a region of the state space that the current training set underrepresents, so that the policy has never had to fit anything like it, or the demonstration is itself poor, in the sense of being suboptimal or invalid, so that no policy could fit it and the loss is a statement about the datum's incoherence rather than about the policy's ignorance. The second reading is the one that would destroy the measure, and any method that reports information gain without addressing it is reporting a number that could mean either.

In DISEIL the second reading is ruled out by construction, and by two independent constructions. A prescription reaches the expert only after it has passed the feasibility check against the knowledge-augmented graph: the prescribed configuration lies inside the workspace bounds, inside the object's spawn range and inside the reachable set, because a violation is returned to the prescription model as feedback and a revised prescription is demanded until a feasible one is produced. An infeasible scenario therefore never becomes a demonstration. And the demonstration itself comes from the expert, whose trajectories are the target the policy is being fitted to, so a demonstration that survives the feasibility check cannot be suboptimal with respect to that target. Neither an infeasible scenario nor a bad action survives into the dataset.

The first reading is therefore the only one left. High pre-retrain loss on a demonstration acquired by this framework identifies genuinely novel, underrepresented data. That is a claim with an argument behind it and not a hypothesis awaiting a test, and it is why Table 8 is a statement about coverage rather than about noise. Starting performance is what makes the argument interpretable in the first place, and it is the reason the initial demonstration count was set as it was. Loss is measured relative to a policy, and a policy that fails uniformly produces a high pre-retrain loss on any demonstration whatsoever, including a redundant one. The measure only discriminates when the policy is competent enough that its failures are localised.

Where the measure stops licensing the claim. High information gain per demonstration is necessary and it is not sufficient, and the evidence for that is one of the project's own ablations, reported in full as A3 in Section 5.1.7.2. Removing the clustering step, so that each round greedily corrects the single highest-loss failure, leaves information gain essentially unchanged on the three ablation settings: it rises by 0.02 on GridWorld (image) and by 0.16 on Push-T (state) and falls by 0.13 on Door (image), a mean change of +0.02. The final success rate on the same three settings falls by 2.2, 4.1 and 6.8 points, a mean of 4.37. Greedy worst-loss selection collects demonstrations that are individually informative and jointly redundant, because information gain measured on one demonstration carries no term for its overlap with the demonstration collected in the previous round. Allocation across failure modes is precisely the term that supplies it.

Table 8 must therefore never be read as the source of the framework's advantage on its own. It is the evidence that the demonstrations DISEIL asks for are novel to the policy. The evidence that they are novel *to each other* is the allocation ablation.

5.1.6 Prescription confidence

At the moment it issues a prescription, the prescription model also emits an integer confidence between 0 and 100, together with a one-line rationale, reporting how likely it believes the resulting demonstration is to improve the policy. That number was added to the prompt as an instrument, to find out whether the model has any usable forecast of the value of the round it is about to spend. It is scored against ΔSR, the change in the policy's success rate on the round-level rollout evaluation across that round.

The Pearson correlation between the reported confidence and the realised ΔSR runs from 0.82 to 0.89 across the ten settings. Quoting each task as state then image: GridWorld 0.88 and 0.82, Push-T 0.87 and 0.88, Lift 0.88 and 0.89, Wipe 0.82 and 0.86, Door 0.83 and 0.82. Figure 7 shows the GridWorld (image) scatter, where $r = 0.82$ over 180 prescriptions, which is the full count that nine seeds at a budget of twenty supply.

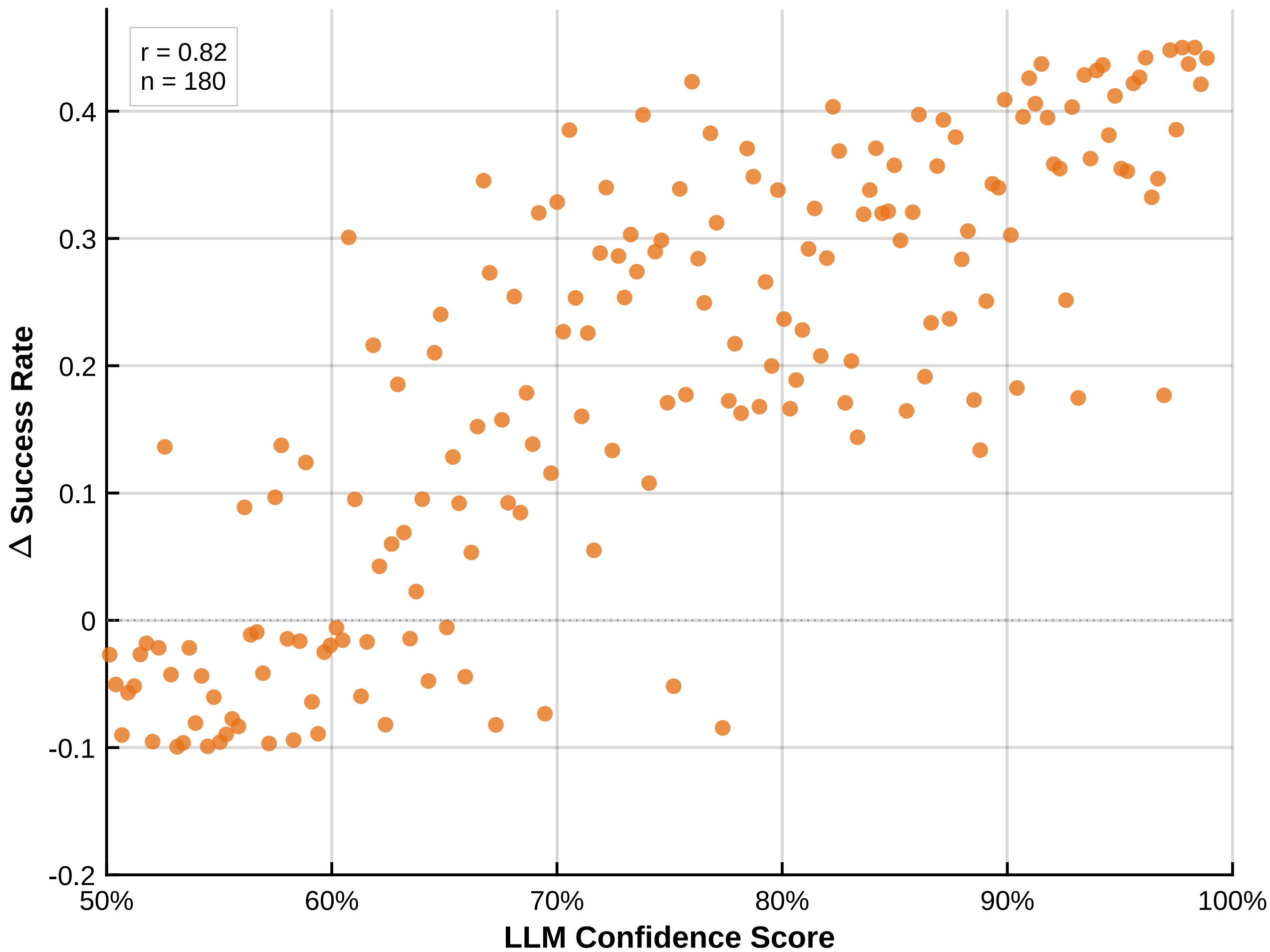


Figure 7. Reported prescription confidence against realised improvement. The confidence the prescription model reports at prescription time, against the change in the policy's success rate on the round-level rollout evaluation that the resulting demonstration produced. GridWorld (image), 180 prescriptions, Pearson $r = 0.82$.

What makes this number usable, and not a rationalisation after the fact, is the order in which the two quantities become available. The confidence is reported blind, at prescription time. At that moment the demonstration has not been collected, the expert has not been called, the policy has not been retrained, and the re-rollout that produces ΔSR has not been run. The success-rate signal arrives only after all three of those steps have completed, by which point the round's unit of budget has already been spent. The model is therefore forecasting an outcome it cannot observe, and a correlation of 0.82 to 0.89 is the accuracy of that forecast rather than a description of an outcome the model was shown. A signal available before the expenditure and correlated with what the expenditure returns is what an allocation framework under a restricted budget needs, and it is not available to any query gate, whose scalar signal is a property of a state and carries no forecast about a demonstration that does not yet exist.

The number carries two limits. The correlation is measured on DISEIL runs only, so it says nothing about whether a baseline's gate signal would predict its own ΔSR equally well. And no experiment in this project gates on the confidence: nothing is skipped, deferred or re-prescribed on the basis of a low confidence score, so the correlation is an observed property of the system and not yet a mechanism inside it. Turning it into one, by declining to spend a demonstration on a prescription the model itself does not believe in, is the obvious next step and it has not been run.

5.1.7 Ablation studies

The comparison establishes that DISEIL wins. It does not establish *what* wins, and a framework assembled from a geometric descriptor, a clustering step, a memory term, a reasoning model, a prescription model, a vision-language model, a constraint store and a fallback rule can win for reasons unrelated to the components the method presents as its contribution. The ablation programme exists to find out which of those components carries the result.

5.1.7.1 Scope and conventions

The programme comprises eighteen studies, A1 to A18. A1 to A13 remove or vary one component at a time. A14 to A18 are diagnostics: they measure a property of the running system rather than knock a component out of it.

The studies are run and reported on three settings, chosen to span the three policy classes and both observation modalities: GridWorld (image), where the policy is a convolutional network; Push-T (state), where it is a state diffusion policy; and Door (image), where it is an image diffusion policy. Every per-setting number below is one of those three, and every aggregate is the mean over those three and is labelled as such. The unit of analysis is the setting, and three paired settings are below the resolution of a Wilcoxon, sign or Friedman test, so no aggregate p-value is reported in this section. What is reported instead is the three per-setting values and the sign they share, which is what three matched blocks can support.

Two derived quantities recur. ΔSR is the change in the policy's success rate on the round-level rollout evaluation. *Margin retained* is the fraction of DISEIL's advantage over the strongest baseline that survives an ablation, $(\text{ablated} - \text{best baseline})/(\text{full} - \text{best baseline})$, expressed as a percentage. Margin retained is reported alongside the raw damage because a component whose removal costs two points where the margin is three points is a different object from a component whose removal costs two points where the margin is ten. A value near 100 per cent means the component is decorative. A value near zero means it carries the result. A negative value means the ablated system has fallen beneath the baseline it was built to beat.

5.1.7.2 Knockouts

The knockouts were run from the bottom of the system upwards, and they are reported in that order: first the two controls that establish what unstructured and unreasoning allocation can already do, then the components the method claims for itself. Figure 8 is the resulting ladder.

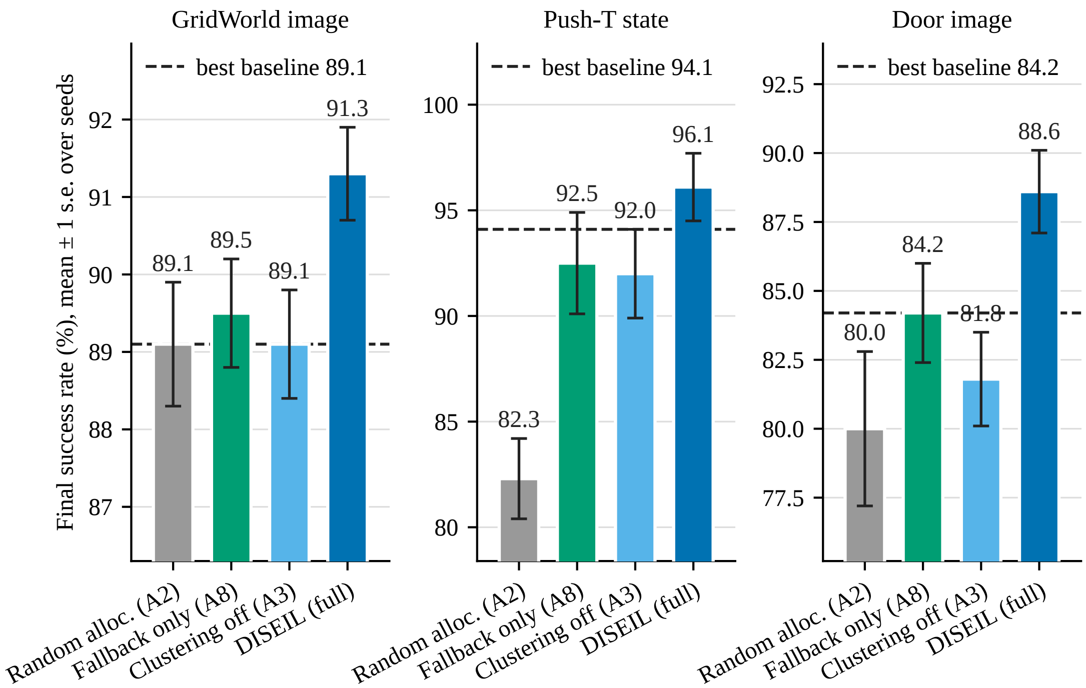


Figure 8. The allocation ladder, on the three ablation settings. Bars are the mean final success rate over seeds, with one standard error (SE $=$ std$/\sqrt{n}$, $n = 9$ on GridWorld and $n = 5$ on the robot settings), for uniform-random allocation over recorded failures (A2), the deterministic nearest-untried fallback rule promoted to the whole method (A8), clustering removed in favour of greedy worst-loss selection (A3), and full DISEIL. The dashed line is the strongest DAgger-family baseline in that setting: EnsembleDAgger on GridWorld (image), Diff-DAgger on Push-T (state) and Door (image). Budget $B = 20$, one demonstration per round.

A2, uniform-random allocation. The first rung corrects one uniformly chosen recorded failure each round, with no descriptor, no partition, no memory and no reasoning model. On GridWorld this arm is the Stagger control of Table 7; on the robot settings it is a separate run. It reaches 89.1 on GridWorld (image), 82.3 on Push-T (state) and 80.0 on Door (image), against DISEIL's 91.3, 96.1 and 88.6. On the two robot settings it lands well below the strongest gated baseline; on GridWorld (image), where it is the Stagger control, it finishes level with the gated baselines and marginally above the best of them, and still below DISEIL. Sampling failures uniformly reproduces the frequency of failure modes in the current policy's failure set, so the dominant mode is corrected in proportion to how often it occurs and a rare, persistent mode is almost never touched inside twenty rounds. The premise of the whole framework is that the value of a demonstration is not proportional to the frequency of the corresponding failure, and A2 measures that premise directly. It also settles the obvious objection, that the advantage is nothing more than failure replay. On the robot settings random replay of recorded failures is worse than an uncertainty gate, and an uncertainty gate is worse than allocation; on the small GridWorld task the gates and random replay finish level, and allocation still leads.

A8, the deterministic fallback rule. When the prescription model cannot produce a feasible prescription after five attempts, DISEIL falls back on the nearest untried recorded failure in descriptor space. A8 promotes that rule to the entire method. It reaches 89.5, 92.5 and 84.2 on the three settings, costs 1.8, 3.6 and 4.4 points against the full system, and retains 18.2, −80.0 and 0.0 per cent of the margin. It beats the strongest baseline only on GridWorld (image); on Push-T (state) it falls below the strongest baseline, and on Door (image) it lands exactly on it. Nearest-untried is a spatial heuristic that implicitly spreads the budget, since a failure adjacent to one already corrected is less likely to be chosen than a distant one, and that is a crude version of the coverage pressure the partition and the memory supply deliberately. It captures a fraction of the margin at best, and it turns negative on Push-T (state), the setting whose strongest baseline is itself the strongest in the table. It cannot do better, because nearest-untried has no notion of a failure mode: it will select a chain of adjacent failures that all belong to the same mode, and it has no mechanism for concluding that a mode has been addressed. A8 is the fallback-only floor against which the rest of the framework is calibrated, and every claim below about the language and vision-language components is measured against the strongest baseline and against A8, not against the baselines alone.

A3, the clustering step. The third rung removes the clustering step and targets, each round, the single failure with the highest peak per-step loss. The loss signal is kept; the failure-mode structure is removed. Success falls by 2.2 points on GridWorld (image), 4.1 on Push-T (state) and 6.8 on Door (image), a mean of 4.37, and the margin retained collapses to 0.0, −105.0 and −54.5 per cent, a mean of −53.2. On Push-T (state) and Door (image) the ablated system falls beneath its own best baseline, 92.0 against 94.1 and 81.8 against 84.2, and on GridWorld (image) it lands exactly on it. Removing the clustering step erases the whole margin over the baselines and on two of the three settings turns it negative, the largest damage of any knockout in the programme.

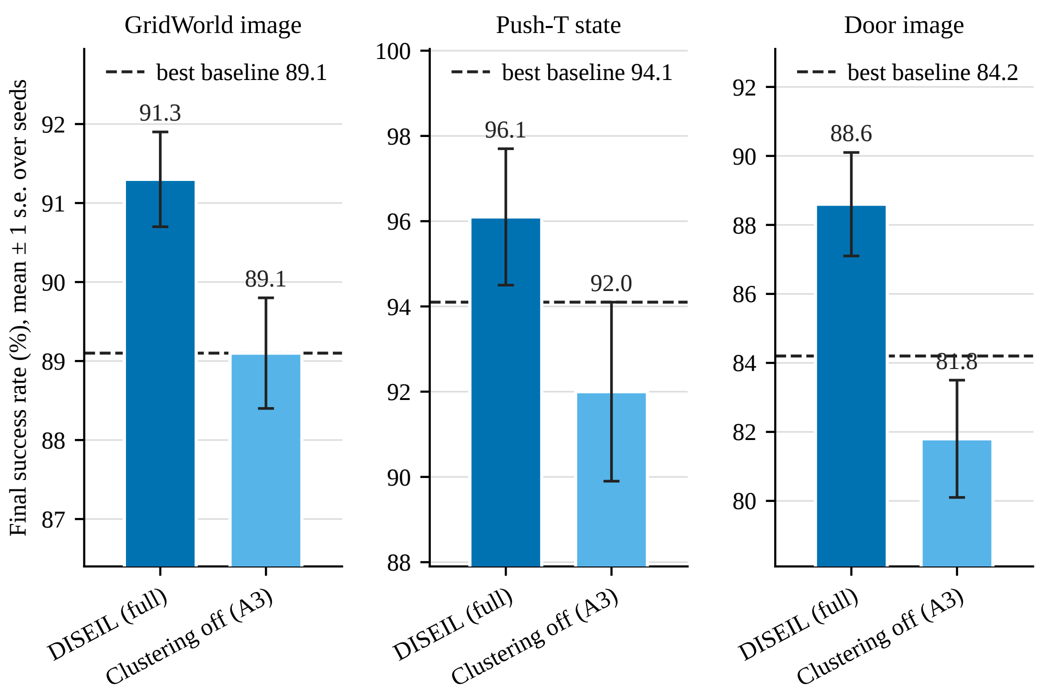


Figure 9. Success rate with clustering removed. Final success rate for full DISEIL and for greedy worst-loss selection (A3) on the three ablation settings, with the strongest DAgger-family baseline as a dashed line.

What makes A3 the argument for the framework, and not one knockout among seven, is what happens to the information gain while the success rate is falling by those amounts. Per-demonstration information gain does not fall when clustering is removed. It rises on two of the three settings, by 0.02 and 0.16, and falls on the third by 0.13, for a mean change of +0.02. Greedy worst-loss selection is as good as DISEIL, or marginally better, at collecting demonstrations on which the current policy has high pre-retrain loss, which is unsurprising, since maximising that quantity is exactly what it does by construction, whereas DISEIL sacrifices some of it to spread the budget across failure modes. The two quantities move in opposite directions, and the dissociation is the finding.

The mechanism is straightforward once stated. Peak loss is a property of one failure trajectory. It is not a property of the *set* of failures the policy is still producing. Under greedy worst-loss, the highest-loss failure in round r very likely belongs to the same failure mode as the highest-loss failure in round $r-1$, because the modes that generate the largest loss spikes are the ones the policy has learned least about, and one demonstration does not close a mode. The budget is spent repeatedly inside one region of the state space. Each demonstration in that stream is genuinely informative in isolation, which is why the gain does not fall, and each is largely redundant with the demonstration collected in the round before, which is why the success rate does not move.

The consequence for the rest of the report is a restriction on what may be claimed. High information gain is necessary for a demonstration to be worth collecting and it is not sufficient, and Table 8 cannot be presented on its own as evidence of the method's advantage, because A3 exhibits an ablated system with equal or higher gain and a four-point lower success rate. A3 also has a limitation worth stating: it removes the descriptor and the memory along with the clustering, because those components have nothing to operate on once modes are gone, so it is a knockout of the allocation *stack* and not of the partition in isolation. The cleaner variant, which keeps the descriptor and replaces agglomerative clustering [91] with a random partition of the failures into k groups, would separate grouping by geometry from grouping at all. It was not run, and it is future work rather than a claim.

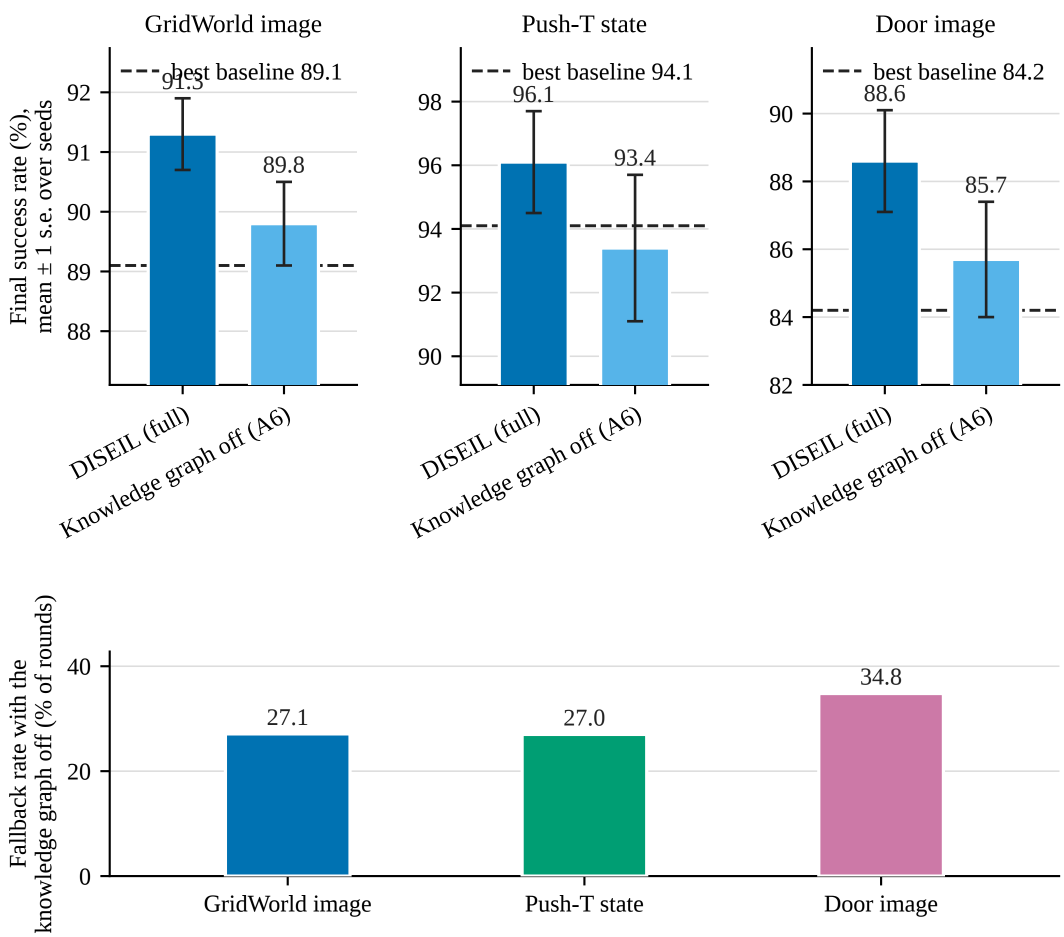


Figure 10. Grounding and feasibility. Top row: final success rate for full DISEIL and with the knowledge-augmented graph removed from the prompts (A6), against the strongest baseline. Bars are means over seeds with one standard error ($\mathrm{SE} = \mathrm{std}/\sqrt{n}$), including the A6 bar. Bottom row: the share of rounds that fall back to the deterministic rule when the graph is removed.

A6, the constraints the prescription is checked against. Equation 10 is a feasibility-verification loop. The prescription model proposes a prescription; constraints are retrieved from the knowledge-augmented graph, which stores workspace bounds, reachability, object and spawn ranges and controller limits as structured key-value knowledge; the prescription is checked against them; if a constraint is violated, the violation is returned to the model as feedback and a revised prescription is requested, until a feasible one is produced. A6 removes the graph from both the vision-language and the reasoning prompts, so the loop has nothing to verify against.

The cost is 1.5, 2.7 and 2.9 points on the three settings, a mean of 2.37, and 31.8, −35.0 and 34.1 per cent of the margin is retained, a mean of 10.3. A6 is the third most damaging knockout, and it costs nearly twice what the prescription model itself is worth. The fallback rate rises to 27.1 per cent of rounds on GridWorld (image), 27.0 on Push-T (state) and 34.8 on Door (image), which means that roughly five to seven of the twenty rounds are spent on a fallback correction and not on a prescribed one, a direct loss of a quarter to a third of the budget. The relation between the fallback rate and the damage is loose, and that is what A8 explains: a fallback round is not an empty round, it is the nearest untried failure, and a lost round costs the difference between a prescribed correction and a fallback correction, not the whole round, which is why A6 costs two and a half points and not six.

What the graph buys follows from that mechanism. It stops the prescription model from producing prescriptions the environment cannot instantiate, and it does not make the model a better reasoner. Two limitations bound the claim. The per-task graph is authored by hand, and the study does not decompose it, so it cannot be said whether the workspace bounds alone would recover most of the damage or whether the failure taxonomy and the per-mode rules matter as well. And A6 knocks out only the first of the architecture's two screens. The second, the policy-solvability check, in which the prescribed scenario is rolled out under the current policy and revised if the policy already solves it, is not ablated anywhere in the programme. That is a gap in the study and it is named as one.

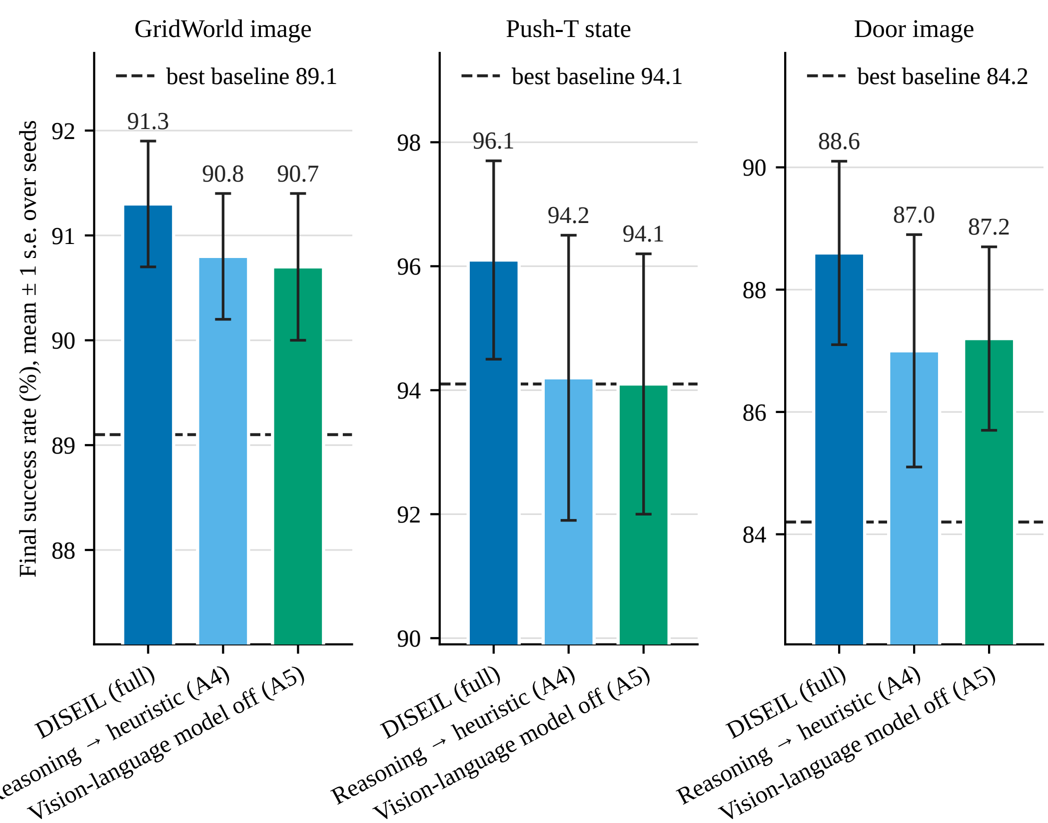


Figure 11. The prescription model and the vision-language model. Final success rate for full DISEIL, for

the prescription model replaced by the dominant-representative heuristic (A4), and for the vision-language model removed (A5), with one standard error over seeds and the strongest baseline as a dashed line.

A4 and A5, the two model calls. Two model calls are defined in Section 4.1.3 and only one of them is knocked out here. The *reasoning model* is the text-only call that assigns a root cause and a trajectory phase to each failure from the taxonomy stored in the task's graph. The *prescription model* is the call that turns the selected mode and its context set into the round's demonstration request. A4 replaces the second of those with the deterministic rule "always target the dominant cluster representative", operating on the same geometric clusters, so that the comparison isolates the prescription decision and not the partition; the root-cause reasoning call is retained in A4 and still runs. A5 removes the vision-language model, which reads three frames of each cited failure and supplies a description of what went wrong, leaving the reasoning model with the geometric descriptor and the root-cause taxonomy. The two calls are removed together nowhere in the programme, and the phrase *reasoning stack*, used below and in Section 5.1.7.5, names the two model calls plus the vision-language call taken together.

A4 costs 0.5, 1.9 and 1.6 points on the three settings and A5 costs 0.6, 2.0 and 1.4, so both average 1.33 points and both retain roughly half the margin, 48.6 per cent for A4 and 47.0 per cent for A5. Every individual gap in both studies is comparable to the seed standard error of the corresponding full run, and the two are close enough in magnitude that the study cannot rank them. Both are largest on Push-T (state) and smallest on GridWorld (image).

The explanation is structural. Clustering is geometric in every run, state and image alike, and it consumes no output from any foundation model. By the time either model is called, the decision that matters, which region of the failure distribution receives this round's demonstration, has already been made by the descriptor and the memory. The prescription model chooses the form of the correction inside a region that was selected without it. A component acting downstream of the decisive step cannot produce a large effect, and the measurement agrees. Read the other way, the same fact is a practical result: a deployment that cannot afford the reasoning stack can delete it, keep the geometric clustering, the memory and the deterministic heuristic, and still beat every baseline on every setting.

Both knockouts carry a limitation that cuts against the framework's interest. The A4 heuristic is a strong one. "Always target the dominant cluster representative" is itself an allocation rule, and it inherits the memory's rotation, because the dominant cluster changes as the memory penalises recently corrected regions. A weaker heuristic would have produced a larger gap. The heuristic also cannot bridge, so A4 and the bridging knockout below are not independent, and the study cannot separate what the prescription model buys as a reasoner from what it buys as the only component that can place a demonstration between two failures. A5, for its part, does not test whether a better visual model would help more. Its largest gap is on Push-T (state), which is a state-modality setting and therefore counterintuitive, and the reading the architecture supports is that the frames are not there to compensate for a missing state vector: they are there to let the model see why the block ended where it did, and Push-T is the task on which one terminal geometry can be reached by several distinct failure processes, such as pushing on the wrong face, losing contact, or over-rotating. On Door, where the geometry of a failure largely determines its cause, the visual channel adds less.

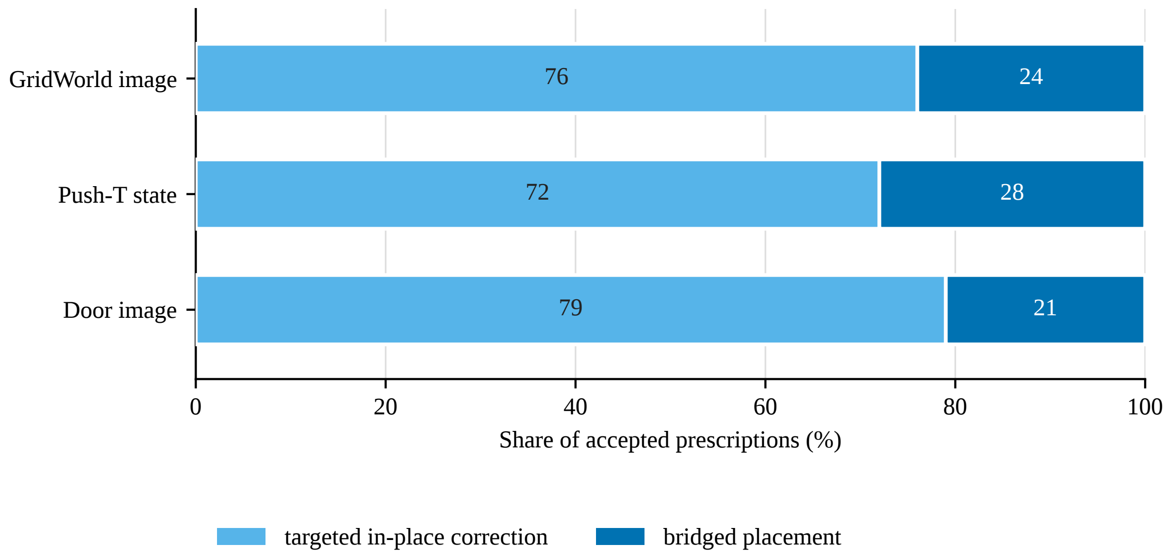


Figure 12. Targeted and bridged prescriptions (study A16). Share of accepted prescriptions on each of the three ablation settings that are a targeted in-place correction and that are a bridged placement.

A7, bridging placement. Bridging is the only part of the prescription that changes the environment configuration rather than selecting an episode, and it is the mechanism by which a prescription can be made easier than the failure it addresses. Disabling it costs 1.3, 1.1 and 1.4 points on the three settings, a mean of 1.27, and retains 40.9, 45.0 and 68.2 per cent of the margin. That is proportionate to how often the arm is used: the bridged share of accepted prescriptions, measured as diagnostic A16 and shown in Figure 12, is 24 per cent on GridWorld (image), 28 on Push-T (state) and 21 on Door (image). A component used in a quarter of rounds cannot produce a large aggregate effect unless the rounds in which it is used are the decisive ones, and the three settings do not order the damage the way they order the share: the largest damage is on the setting with the middle share. What matters is which rounds bridge, not how many, and the coherent reading is that bridging pays when the target cluster lies far outside anything the current policy solves, so that a targeted demonstration would be a large distributional jump and a bridged one is a step the policy can absorb. That reading is not itself tested here.

One discrepancy in the record is stated rather than smoothed over. The method's precondition for bridging is pose randomisation, from which it follows that bridging should be inapplicable on GridWorld, which is a discrete grid. The prescription logs disagree: 24 per cent of accepted prescriptions on GridWorld (image) are marked bridged, and A7 records a measurable effect there. Either the implemented mechanism is broader than the method text claims, or the runs did something the method text did not intend. The data are the source of truth, so bridging is reported as active on that setting, and the resolution of the discrepancy, by inspection of the prescription logs, is an outstanding item in Section 6.1.

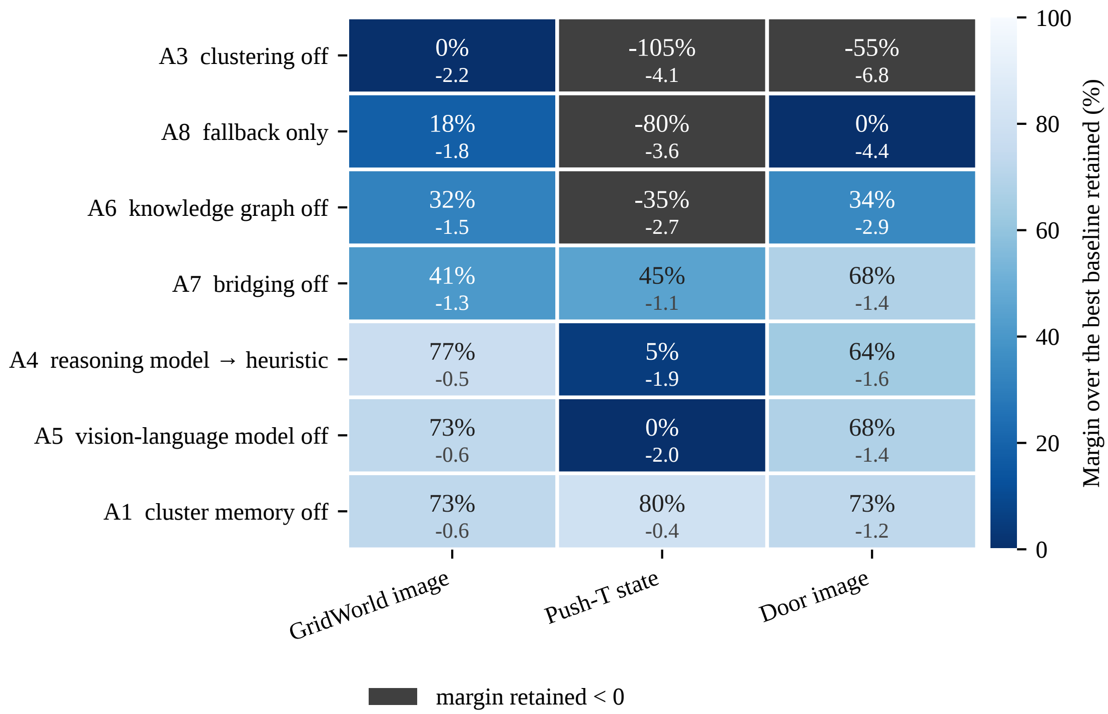


Figure 13. Every knockout, on the three ablation settings. Cells give the percentage of the margin over the strongest DAgger-family baseline that survives the ablation, with the change in success rate in points beneath. Rows are ordered by mean damage. The single cell below zero is A3 on Door (image), where the ablated system falls beneath its baseline.

A1, the cluster memory. Switching the memory off ($\lambda = 0$) costs 0.6, 0.4 and 1.2 points on the three ablation settings, a mean of 0.73, and retains 72.7, 80.0 and 72.7 per cent of the margin, a mean of 75.1. It is the smallest effect of the seven knockouts, and every individual gap is no larger than the seed standard error of the corresponding full run. The memory suppresses a cluster that has just received a demonstration, so that the following round is pushed onto a different one, and A1 prices that suppression once the partition is already in place. The price is task-dependent. It is 0.4 points on Push-T (state) and 1.2 on Door (image), a factor of three across two of the three settings, so no single number describes what the memory is worth and the study does not supply one. The framework therefore carries the memory as a configurable component of the loop, switched on per task, and not as a headline contribution, and this study is the evidence for that description. What the framework has to defend is the partition the memory operates over, and A3 is where it is defended.

The ordering. Ranked by mean damage over the three ablation settings, the seven knockouts are: clustering (−4.37 points, −53.2 per cent of the margin retained), the fallback rule promoted to the whole method (−3.27, −20.6 per cent), the knowledge-augmented graph (−2.37, 10.3 per cent), the prescription model and the vision-language model (−1.33 each, 48.6 and 47.0 per cent), bridging placement (−1.27, 51.4 per cent) and the cluster memory (−0.73, 75.1 per cent). Figure 13 gives the whole grid. The ordering shaped the rest of the programme in two ways. The clustering step, which the framework treats as an implementation detail of a generic partition, is the component that carries the result. The cluster memory, which an earlier draft of the method advanced as one of its two contributions, is the least damaging of the seven, and it is reported as a configurable, task-dependent feature on that evidence.

5.1.7.3 Design choices

Given that allocation is the mechanism, the next family of studies asks whether *this* descriptor, *this* cluster count, *this* context set and *this* budget are the right way to allocate. These studies were added after the knockouts, because until the knockouts had run there was no reason to interrogate the partition in such detail.

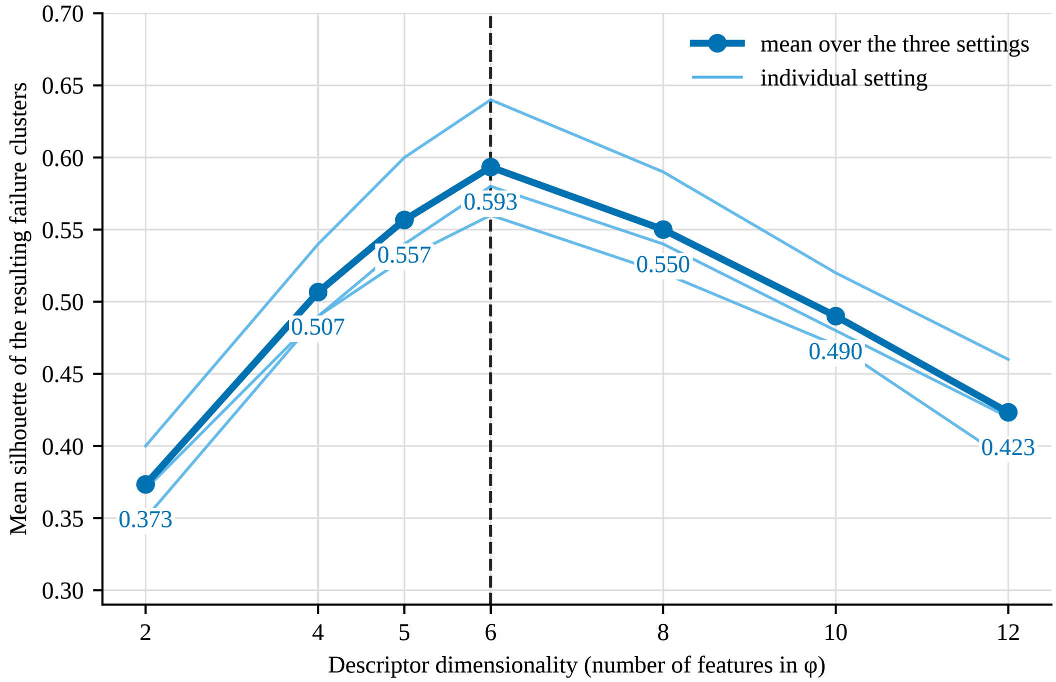


Figure 14. Mean silhouette of the failure clusters against the dimensionality of the geometric descriptor. One line per ablation setting and the mean over the three. The dashed vertical line marks the six-dimensional descriptor the framework uses. Silhouette scores geometric separation only and is independent of the success rate.

A10, the width of the geometric descriptor. The descriptor is designed, not learned, which invites the suspicion that the feature set was fitted to the reported result. A10 answers by scoring the descriptor on what it is for, the separation of failure modes, using a criterion with no relationship to success rate: the mean silhouette of the resulting clusters [79]. Features are removed from and added to the descriptor, and the silhouette is measured over every clustering round. If the feature set had been over-specified, the curve would be flat and the choice of six dimensions arbitrary.

The curve is a clean inverted U with a single interior maximum, and the six-dimensional descriptor is the highest-scoring variant in each of the three settings. Averaged over the three, the mean silhouette runs 0.373 at two dimensions, 0.507 at four, 0.557 at five, 0.593 at six, 0.550 at eight, 0.490 at ten and 0.423 at twelve. Below six dimensions the descriptor discards information that distinguishes modes, and the largest single step in the whole sweep, +0.133 in mean silhouette, is the step from two dimensions to four, which adds orientation. Position alone cannot separate a Push-T failure in which the block is in the right place at the wrong angle from one in which it is in the right place at the right angle and the pusher lost contact, so those failures collapse into one cluster and the cluster's silhouette is poor. Adding task progress, worth +0.050, and contact distance, worth +0.037, each buy less, which is consistent with orientation being the dominant discriminator on these tasks.

Above six dimensions the fall is not an information loss, since every added feature carries some signal. It is geometric. As dimensionality rises, the ratio of the nearest to the farthest pairwise distance approaches one, all failures come to look equidistant, the agglomerative merge order becomes arbitrary, and the silhouette falls although nothing was removed. End-effector velocity adds two dimensions of mostly noise to a distance computation over a few dozen points, and by twelve dimensions the joint-angle summary has added six more. The number of failures being clustered is small, forty-two in round 1 and two by round 20 on the instrumented setting, and distance concentration bites hardest at small sample sizes. The descriptor is small because the failure sets are small, and the two facts are linked by the geometry of the distance computation and not by a design preference.

A10 also settles a point of record. Clustering is geometric for every run, state modality and image modality alike, and the descriptor is the same six-dimensional vector in both: $[p_x, p_y, \sin\theta, \cos\theta, \rho, \delta]$ for the robot tasks, where ρ is task progress and δ is contact distance, and, for GridWorld, the agent cell, the signed offset to the goal, the progress and the Manhattan distance to the goal. The two columns for one task under the two modalities differ because an image policy fails in different places, not because the features differ. The visual channel feeds root-cause reasoning and it does not feed the partition.

The limitation is the one silhouette cannot address. Silhouette scores geometric separation, not semantic correctness, and a descriptor could produce well-separated clusters that correspond to no distinct root causes at all. The purity diagnostic of Section 5.1.7.4 is the measurement that bears on that hole, and it does not close it. A10 also shows only that this family of descriptors peaks at six dimensions. It says nothing about whether a learned descriptor would do better.

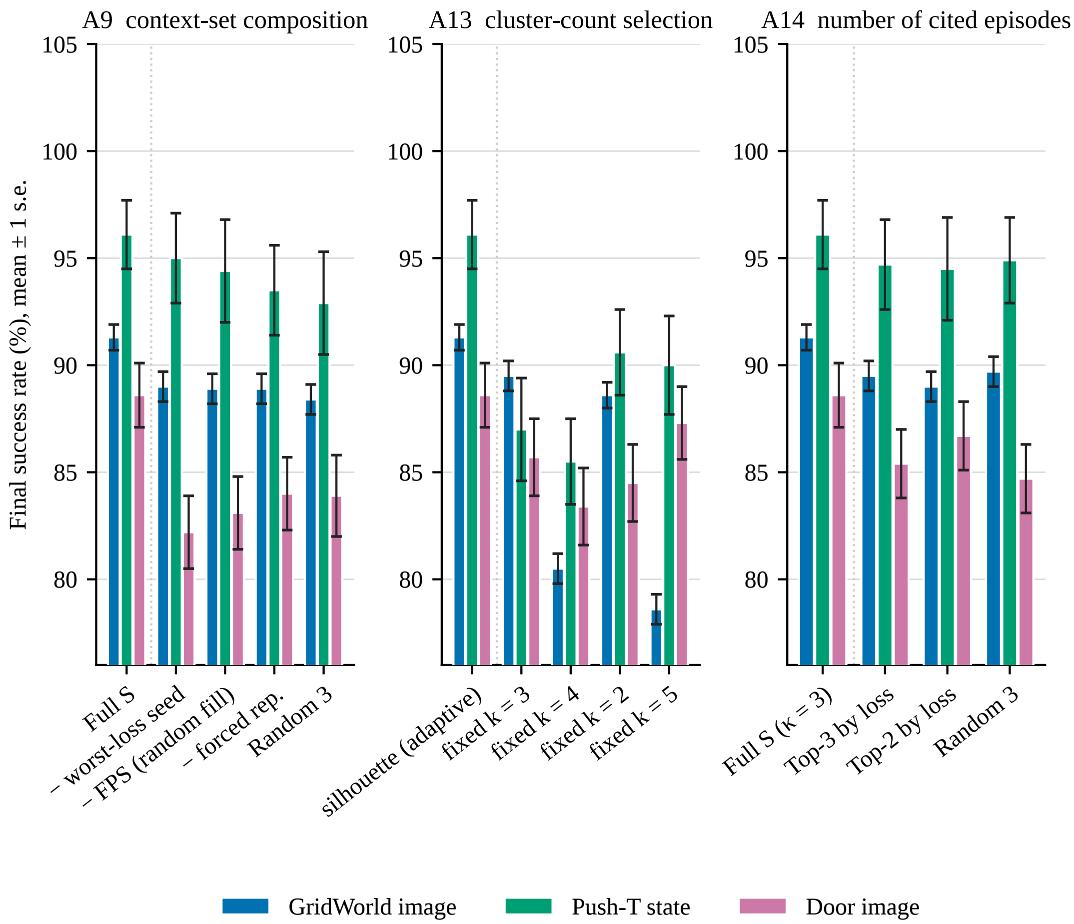


Figure 15. Three ablations of the machinery inside individual steps, with arms ordered by effect. Left: the composition of the context set S (A9). Middle: silhouette-based selection of the cluster count against

fixed alternatives (A12). Right: the number of cited episodes and the selection rule (A13). Bars are means over seeds with one standard error, on the three ablation settings.

A9 and A13, the context set. The context set given to the prescription model contains three cited failure episodes chosen by three rules: the forced representative of the target cluster, the worst-peak-loss seed, and a farthest-point-sampling fill for diversity [23]. A9 removes each rule in turn and adds a floor control of three episodes drawn at random from the cluster, with the target cluster fixed by the memory in every arm so that only the composition of S varies. All figures in this paragraph are means over the three ablation settings, against a full-system reference of 92.0.

The three single-rule removals cost almost the same. Dropping the forced representative costs 3.2 points, dropping the farthest-point diversity fill costs 3.2, and dropping the worst-loss seed costs 3.27, so on this run the study cannot rank the three rules against one another. Each still hurts, and each for a reason the others do not cover: the forced representative guarantees the prescription model sees an example of the mode it has been instructed to fix, without which the cited episodes can all come from the cluster's periphery; the diversity fill keeps a context set of three episodes from all crowding near the loss peak, where it would describe the mode narrowly and draw a narrow correction; and the worst-loss seed sharpens the description of the mode. Random selection of three episodes from the cluster costs 3.6 points, more than any single-rule removal, so the three rules together beat an unstructured draw and are complementary rather than redundant.

The magnitudes discipline the claim. The whole spread from the full context set to random selection is 3.6 points, larger than the knowledge graph and still less than the clustering. A13 sharpens the point by varying the *number* of cited episodes jointly with the selection rule, and it contains the one comparison that could have deleted a component of the method: citing the top three episodes by plain peak-loss rank, against the three-rule construction, at the same number of citations. The gap is 2.13 points. It is a real gap in the paired sense, since the full construction wins on each of the three settings, and it is a substantial one: the three-rule context set clearly beats a plain top-three-by-loss citation, so the farthest-point construction earns its place rather than barely surviving. Citing only two episodes costs 1.93 points against the same reference, so fewer citations are worse; citing five gives no measurable gain over three, and citing every failure in the target cluster only raises the prompt length without adding discriminative detail, since the early rounds carry about forty failures and a context set of that size buries the target mode in detail the model cannot weigh, so the cap at three is justified from both sides. The single-citation arm was not run, and the reason is that it is confounded by construction: bridging requires at least two cited failures in order to define a placement between a failing region and a solved one. It is not a null result and must not be reported as one.

A12 and A15, the cluster count. The cluster count k is chosen per round by maximum mean silhouette, which is standard practice, is used as such, and is not claimed as a contribution [70, 79]. A12 replaces the adaptive choice with a fixed $k \in \{2, 3, 4, 5\}$. Silhouette selection wins on each of the three settings, and it beats the best fixed alternative, averaged over the three, by 4.1 points: a fixed $k = 2$ costs 4.1 points against the adaptive rule, $k = 3$ costs 4.6, $k = 4$ costs 7.4 and $k = 5$ costs 6.7. The effect is well outside the seed standard error of the three settings, which runs from 0.6 to 1.6 points, so the adaptive rule is defended by the size of its effect and not only by the consistency of its sign. No single fixed k is best across the three settings either: the best fixed value is $k = 3$ on GridWorld (image), $k = 2$ on Push-T (state) and $k = 5$ on Door (image), which is the point. Too few clusters merges distinct failure modes, so the memory penalises a merged cluster and suppresses correction of a mode that was never addressed. Too many splits one mode across several clusters, so the memory cannot recognise that the mode has been corrected and rotation is diluted across fragments of the same region. The right number varies by setting and by round, and only the adaptive rule tracks it.

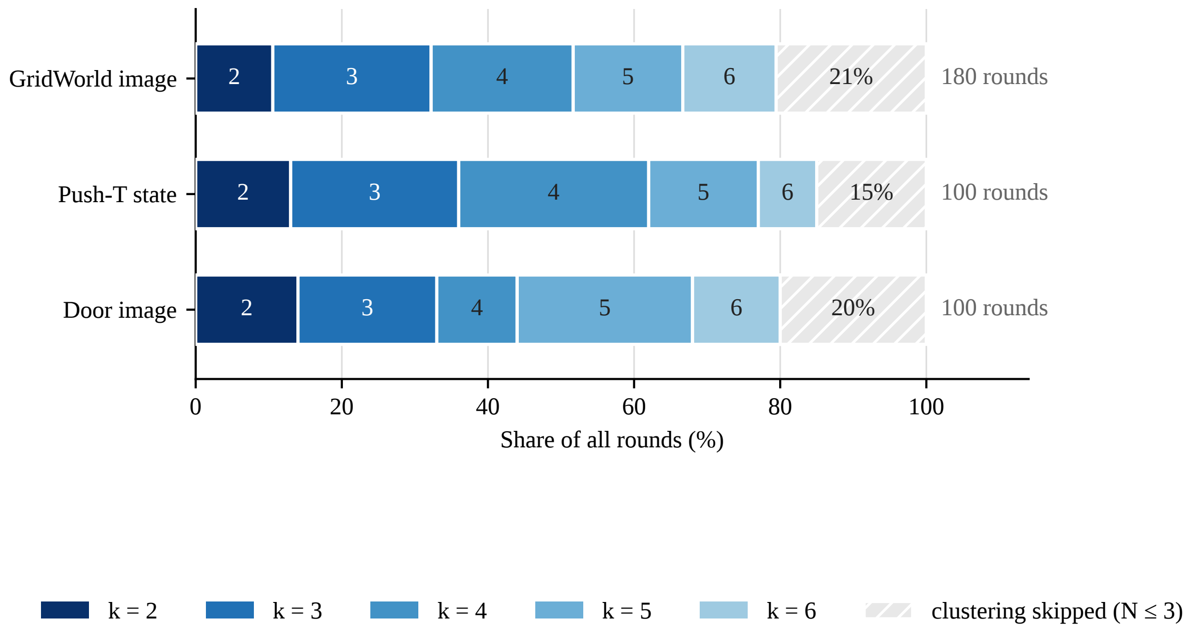


Figure 16. Cluster count selected per round (study A15). Share of all rounds selecting each cluster count k, with the rounds that skip clustering entirely, because fewer than four failures remain, shown hatched.

A15 shows that the adaptivity is not a disguised constant. Pooled over the 308 clustered rounds of the three ablation settings, $k = 3$ is the most frequently selected count at 26.3 per cent, with $k = 4$ at 23.4, $k = 5$ at 21.4, $k = 2$ at 14.9 and $k = 6$ at 14.0. No value from two to six falls below 14 per cent. The claim the framework may make is that the number of discovered failure modes varies by round and is most often three or four, and not that there are three failure modes. A15 also carries a seam: 21 per cent of GridWorld (image) rounds, 15 per cent of Push-T (state) rounds and 20 per cent of Door (image) rounds never cluster at all, because fewer than four failures remain, and in those rounds each failure becomes its own cluster and the budget is allocated by the fallback rule. That is the same fact the failure-count curve of Figure 18 shows in time.

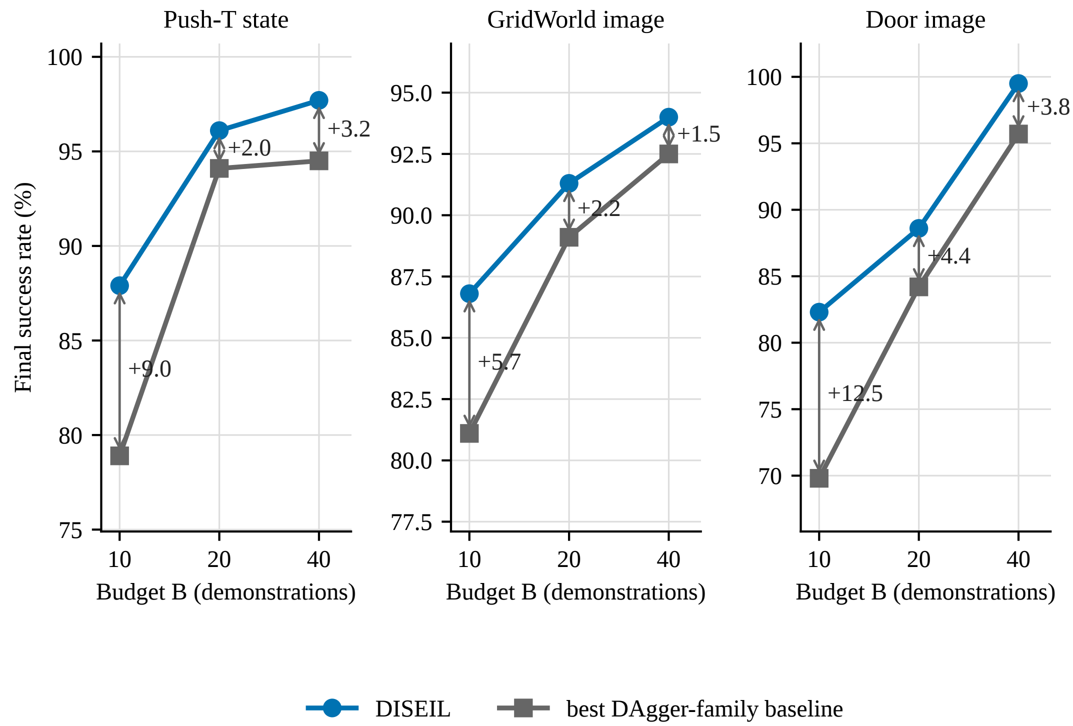


Figure 17. Final success rate against the budget B (study A11). DISEIL against the strongest DAgger-family baseline at $B = 10$, 20 and 40, on the three ablation settings, with the margin printed between the two series.

A11, the budget. The framework is claimed to operate under any fixed budget, with $B = 20$ as the validated instance, and A11 is the experiment that supports or refutes the claim. The margin is far larger at the smallest budget than at the two larger ones: it averages +9.07 points at $B = 10$, +2.87 at $B = 20$ and +2.83 at $B = 40$, with per-setting values of +5.7, +9.0 and +12.5 at the smallest budget and +1.5, +3.2 and +3.8 at the largest. The decline with budget is clean on GridWorld (image) and Door (image); on Push-T (state) the margin is +9.0 at $B = 10$ and then sits near +2 to +3 at the two larger budgets, so the trend is not strictly monotonic there.

The margin shrinks because the baseline catches up and not because DISEIL degrades, and Figure 17 is the evidence: DISEIL's own success rate rises with the budget on every one of the three settings, from 86.8 to 94.0 on GridWorld (image), from 87.9 to 97.7 on Push-T (state) and from 82.3 to 99.5 on Door (image), while the strongest baseline rises faster from a lower start. Allocation buys the *rate* at which the failure distribution is covered, not the asymptote, which is why a large budget closes the gap: with enough draws, even a poorly allocated stream eventually covers the failure distribution, because coverage is a coupon-collector problem and the collector wins if it draws long enough. A11 is therefore the evidence for the separation the framework insists on: B is a symbol in the method and in the algorithm, whose loop header reads "for $r = 1$ to B", and the value 20 appears only in the experimental setup.

One claim the sweep was expected to support does not hold, and it is retracted here. The study was designed around the headline that DISEIL at $B = 10$ matches the strongest baseline at $B = 20$, which would be the same policy for half the expert labour. It does not. On the three ablation settings, DISEIL at $B = 10$ reaches 86.8 against the baseline's 89.1 at $B = 20$ on GridWorld (image), 87.9 against 94.1 on Push-T (state) and 82.3 against 84.2 on Door (image), so it falls short of the strongest baseline's twenty-demonstration result on every one. The halved-labour claim is false and does not appear in this report. What survives is the claim the data do support, which is the one that matters for the framework: the advantage of allocation grows as the budget shrinks, and the method stops paying when demonstrations stop being scarce.

5.1.7.4 Diagnostics

The last family of studies removes nothing. It asks whether the clusters the framework discovers mean what the method says they mean, and whether the machinery that discovers them stays active for as long as the budget runs. Both answers bound the claims made above rather than adding to them.

Table 9. Root-cause label purity per geometric cluster (study A14). Mean cluster purity is the fraction of a geometric cluster's failures that share the dominant root cause assigned by the reasoning model. Mean root causes per cluster counts the distinct root causes present in a cluster. Mean silhouette measures the geometric separation of the clusters. Purity is scored against the reasoning model's own labels, so it records agreement between two components of the same system and not agreement with ground truth.

Setting	Mean cluster purity	Mean root causes per cluster	Mean silhouette
GridWorld 5x5 (image)	0.89	1.62	0.58
Push-T (state)	0.91	1.35	0.64
Door (image)	0.84	1.86	0.56

A14, and the limit of a geometric descriptor. A10 shows that the descriptor produces well-separated clusters. A well-separated cluster need not correspond to a single root cause, and A14 is the check that stops the failure-mode claim from being a statement about geometry alone. Purity runs from 0.84 to 0.91 across the three ablation settings, and the number of distinct root causes per cluster runs from 1.35 to 1.86.

The ordering is the informative part, and it is the same ordering on both columns. Push-T (state) has the best silhouette, 0.64, and the highest purity, 0.91. Door (image) has the lowest silhouette, 0.56, the lowest purity, 0.84, and the most root causes per cluster, 1.86. Geometric separation and semantic purity rise and fall together on these three settings, so A10 and A14 are not independent audits of the descriptor: a descriptor that separates well geometrically also tends to produce clusters that are semantically clean, and neither measurement checks the other. What A14 establishes is the qualification, not an independent guarantee. The descriptor separates failures by where and how they occur, and it recovers root cause only to the extent that configuration determines cause. Where several causes share one end-effector position, as they do wherever a cluster carries close to two distinct root causes, geometry cannot tell them apart.

The circularity in the measurement is admitted. Purity is scored against the reasoning model's own root-cause labels, so it records agreement between two components of the same system. There is no human-labelled root-cause set, and there should be, because purity and silhouette covary and a human-labelled set is the only measurement that would audit either of them from outside.

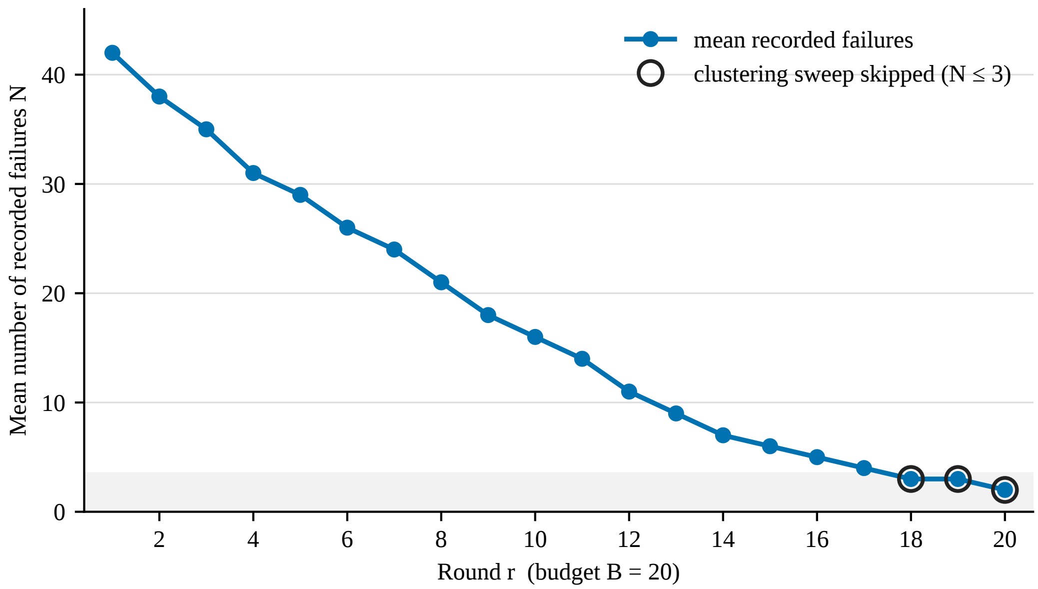


Figure 18. Mean number of recorded failures per round over the budget (study A17). The setting is Push-T (image), averaged over five seeds. Below four remaining failures the clustering sweep is skipped and each failure becomes its own cluster; the shaded band marks that region.

A17, failures per round. Failures per round fall from forty-two to two over the budget, halving by round 8 and falling by an order of magnitude by round 17. The decline is the system working as intended, and it also bounds the system's own mechanism. Clustering forty failures into three or four failure modes is a meaningful operation. Clustering five failures into three is barely one. On the instrumented setting the descriptor, the clustering and the memory do their work through round 17, and the last three rounds run the fallback rule on a handful of remaining failures. A15 confirms the pattern on the three ablation settings, where 15 to 21 per cent of all rounds skip clustering altogether.

This diagnostic is instrumented on one setting, Push-T (image), which is the only setting for which the failure count was logged per round. It is a diagnostic of the loop rather than an arm of an ablation, and the instrumentation should be extended to the three ablation settings before the front-loading behaviour is claimed as general, because the shape may differ where the initial success rate is higher. That is an outstanding item and is listed in Section 6.1.

Read alongside the budget sweep, where the margin doubles as the budget is halved, A17 says that DISEIL front-loads the value of a small budget, and the two facts are consistent: the budget's marginal value is highest early and the machinery is most active early. It also names an extension that is not tested here and is not claimed, which is to stop the reasoning stack once the failure count drops below the clustering threshold and spend the remaining rounds on the fallback, saving most of the per-round reasoning cost at no measured loss.

5.1.7.5 Computational cost

The per-round cost of the reasoning pipeline is a limitation of this framework and it is the first quantity a reviewer will ask for once the success rates are accepted. Study A18 measures it directly, so that the limitation is stated with a number attached.

The measurement. Each of five settings runs a matched pair of runs, DISEIL against SafeDAgger, on the same task, the same observation modality and the same hardware, and the wall-clock and token cost of every round is read out of each run's own telemetry. SafeDAgger is the baseline arm throughout. Two protocols are reported and they must not be mixed. Protocol P1 reports the first round of each run. It is the only protocol available in all five settings, and the first round holds the weakest policy and therefore the most failures and the most model calls, so P1 is an upper bound on steady-state cost rather than an average of it. Protocol P5 reports the mean and the sample standard deviation over the language-model-active rounds of the runs that carried a longer budget, matched arm for arm against the baseline's same-indexed rounds. The spread quoted under P5 is the round-to-round spread inside a run. A P1 row and a P5 row measure different objects and no reader should read one against the other.

Table 10. Per-round wall-clock and token cost, DISEIL against SafeDAgger (study A18). *Shared* is the part of the round that both arms pay: a from-scratch policy retrain and a held-out evaluation. *Add-on* is every DISEIL-specific stage of the round, which is the failure-screening rollout together with clustering, the vision-language call, the reasoning call, the prescription and the feasibility check, less the query-gate rollout the baseline runs in its place. *Tokens* is the sum of the vision-language and language-model tokens drawn in the round. Token counts are not comparable across rows, because the backends differ; see the note below.

Protocol	Setting	Baseline (s)	DISEIL (s)	Shared (s)	Add-on (s)	Tokens
P1	Door (state)	737.0	1,054.0	783.0	+270.0	11,511
P1	Door (image)	1,247.0	1,474.0	1,180.0	+293.0	11,379
P1	Wipe (image)	1,468.0	2,195.0	1,491.0	+700.0	9,560
P1	Push-T (image)	688.0	1,891.0	652.5	+1,232.1	82,116
P1	GridWorld (image)	54.6	118.0	51.1	+62.6	9,735
P5	Door (state)	532.6±145.5	782.6±183.5	547.8	+232.8	10,929
P5	Door (image)	1,034.2±150.2	1,168.8±194.8	901.6	+266.2	10,787
P5	Wipe (image)	1,303.3±144.1	1,991.0±179.2	1,332.7	+655.7	10,118

What the measurement says. A round's wall-clock is dominated by the from-scratch policy retrain and the held-out evaluation, and both arms pay all of it. Under P1 that shared cost is 783 to 1,491 seconds per round on the three RoboSuite settings, against a reasoning add-on of 270 to 700 seconds, and under P5 it is 548 to 1,333 seconds against an add-on of 233 to 656. The ratio of a DISEIL round to a baseline round inherits the large shared denominator and is diluted by it: across the two protocols it runs from 1.13 to 2.75, and a ratio near one is not evidence that the reasoning pipeline is cheap. It is evidence that the retrain is expensive. The quantity that characterises the pipeline is the add-on, the seconds and the tokens the baseline never spends, which is 63 seconds per round on GridWorld and 1,232 seconds on Push-T, at 9,560 to 82,116 tokens per round. Both numbers belong in the report, and neither on its own is honest.

Push-T is the outlier and its reasoning is not the reason. Its shared cost is the second smallest in the sweep, and its baseline arm is unusually cheap, because SafeDAgger's loop runs until the intervention budget is spent and therefore halts after its first intervened episode, which is one episode and 6.3 seconds, while DISEIL screens a fixed sixty. At a longer budget the baseline would roll out many episodes per retrain and the ratio would fall. The token draw is the second reason: Push-T issues twenty language-model calls against Door's seven. GridWorld sits at the other end in seconds and not in tokens. Its whole round is 118 seconds, so 63 seconds of reasoning is more than double a 55-second baseline round, and it still draws 9,735 tokens, of which the knowledge-graph block is 54 per cent of the prompt budget, the largest share of any setting. A cheap task in seconds is not a cheap task in tokens. Token counts are not comparable across rows in any case, because the settings were served by different backends, and the hidden reasoning tokens are read directly from the usage record on some and recovered from the billed completion length on others.

What this means for the framework. The language and vision-language models run only at demonstration-selection time. They are never in the control loop and they do not run at execution, so their cost is paid once per round, and on the three RoboSuite settings it is amortised over a retrain and an evaluation that together cost more than the reasoning does. The resource being traded is model inference against expert demonstration time, and on the settings measured here a round buys one demonstration for an extra 63 to 1,232 seconds of inference. Whether that trade is worth making depends on what a demonstration costs, which in simulation is nothing and outside simulation is a person's time. The limitation is now quantified and not removed. The pipeline costs real seconds and real tokens per round, the cost is highest exactly where the failure count is highest, and the graceful degradation that A4 measures, in which a deterministic heuristic over the same geometric clusters costs 1.33 points and still beats every baseline in every setting, is available to anyone for whom the trade does not clear.

5.1.8 Changes made to the framework

The ablation programme changed the framework in three ways, and the changes are recorded here rather than presented as if the method had always said them.

The cluster memory is re-specified as a configurable component. A1 prices it at 0.6, 0.4 and 1.2 points on the three ablation settings, which is the smallest of the seven knockouts and is no larger than the seed standard error of every full run it is measured against, and the price varies by a factor of three across the three settings. The method therefore carries the memory as a feature of the loop that a task enables or disables, with the evidence for its value stated per task, and not as one of the framework's two headline contributions, which is what an earlier draft claimed for it.

Three components survived with their claims reduced. The clustering step is the non-negotiable core, and it is presented in the method as a generic partition step instantiated here with agglomerative clustering [91], where k-means [54] or another partition would also serve; the descriptor it operates on is geometric in every run and A10 fixes its width at six dimensions. The prescription model and the vision-language model stay, and each is now described as worth about one and a third points, with the prescription model presented as the component that turns an allocation into an executable prescription and not as the source of the advantage. The knowledge-augmented graph stays, and its function is stated precisely: it does not make the reasoning model a better reasoner, it stops the reasoning model from proposing prescriptions the environment cannot instantiate, and the fallback rate is the mechanism by which that shows up in the success rate.

The largest thing the ablations changed is the framework's account of itself. The advantage is carried by an allocation mechanism operating over a hand-designed geometric descriptor of the current round's failures, and the language model's marginal contribution is small because the language model is never told anything the descriptor does not already encode. Its entire input is a description of three frames and a handful of geometric coordinates from the round that has just finished, and it is given no information about the demonstrations already collected. Supplying that information is the object of Aim 2.

5.1.9 Limitations

Each limitation below was measured by a study reported above, and the two that matter most define Aim 2. They are ordered by how much they constrain the claim.

The selector reasons about one round and knows nothing about the dataset. The selector consumes the current policy's failures, partitions them into failure modes, and prescribes one demonstration. Its only representation of history is the recency-discounted penalty applied to the clusters already corrected, which records *where* corrections have been placed in the descriptor space and holds no representation of *what the training set contains*. The consequence bounds the framework's central objective. The selector can determine that the policy failed at a particular configuration and that the root cause is, say, a grasp failure. It cannot determine that the training set already holds six demonstrations of that cause, and that the failure is therefore a symptom of under-fitting and not of under-coverage. Under a restricted budget, a demonstration spent re-teaching material the dataset already contains is a demonstration lost, which is the opposite of what the budget exists to buy. Nothing in the current instantiation prevents that expenditure, and nothing in it measures how often it occurs. The gap is not closed by the cluster memory even in principle: a memory indexed on geometry answers "have we placed a demonstration near here?" and not "does the dataset already contain this behaviour?", and the two questions come apart wherever geometry does not determine cause, which A14 measures directly. Dataset-level curation and retrieval methods index a corpus by embedding distance and select from it [6, 31, 62]; this framework has no such index, and its selector was never given one.

The descriptor is designed by hand, and geometry recovers cause only where configuration determines cause. The six-dimensional descriptor is authored, not learned. A10 defends its *width* on a criterion independent of the reported success rates, and what it establishes is that within this family of geometric features six dimensions is the right number. It says nothing about whether a learned descriptor, of the kind that frozen visual encoders supply for the image-modality policies [66], would separate failure modes better. That experiment has not been run. A14 locates the descriptor's ceiling: purity runs from 0.84 to 0.91 on the ablation settings, and a cluster carries between 1.35 and 1.86 distinct root causes. Purity is measured against the reasoning model's own labels, so it records agreement between two components of the same system, and it covaries with silhouette, so neither measurement audits the other. The claim that the discovered failure modes are semantically meaningful must be qualified by those numbers, and a human-labelled root-cause set is the measurement that is missing.

The cluster memory is worth about three quarters of a point, and how much it is worth depends on the task. Switching it off costs 0.6, 0.4 and 1.2 points on the three ablation settings, a mean of 0.73 and about a quarter of the margin over the best baseline, and every individual gap is no larger than the seed standard error of the corresponding full run. It is the least damaging of the seven knockouts. The framework carries it as a configurable component on that evidence, and the limitation is that no study in this programme establishes when a task should switch it on: the three settings disagree with each other by a factor of three, three settings cannot support a rule, and the rule is not offered. Aim 2 replaces this component with a memory indexed on the dataset rather than on the descriptor space, which is a different object and is evaluated as one.

Each language-model component is worth about one point, and the reason is structural. Replacing the prescription model [94] with a deterministic rule costs 1.33 points on the ablation settings, and removing the vision-language model [4] costs the same, with every individual gap comparable to the seed standard error of the corresponding full run. The partition is geometric and uses no output from any foundation model, so by the time either model is called, the decision that determines the round has already been made by the descriptor and the memory. A component that acts downstream of the decisive step cannot produce a large effect, and the measurement agrees with the architecture. That result admits two readings, and the choice between them is the argument of Aim 2. Either language models add little to demonstration selection, or the language model here was given too little to reason over. The evidence supports the second, and the comparison that sharpens the point is the grounding knockout: removing the prescription model costs 1.33 points, and removing the environmental constraints against which its proposals are verified costs 2.37 points and drives the fallback rate to between 27 and 35 per cent of rounds. The literature says why this should be expected, since vision-language models are reliable at naming a cause given structured evidence and unreliable at metric geometry from pixels alone [15, 27]. The framework hands the model a low-dimensional geometric descriptor and a constraint store for exactly that reason, and it then does not hand it anything else.

The allocation machinery is active early and idle late. Failures per round fall from forty-two to two over the budget on the instrumented setting, and below four remaining failures the clustering sweep is skipped and each failure becomes its own cluster. The descriptor, the partition and the memory therefore do their work in the early and middle rounds, and the last few rounds are allocated by the deterministic fallback rule, which on its own retains about a third of the margin. Across the three ablation settings the skipped rounds are 15 to 21 per cent of all rounds. The allocation account is an account of the early and middle rounds, and the failure-count curve itself is instrumented on one setting only.

The reasoning pipeline costs seconds and tokens. Each round calls a vision-language model on three frames per cited failure and a reasoning model at least once, and possibly several times if the feasibility check returns a violation. A18 prices that pipeline against a matched baseline round on the same task and the same hardware, and the add-on the baseline never pays is 63 seconds per round at the cheapest setting and 1,232 at the most expensive, at 9,560 to 82,116 tokens. The ratio of a DISEIL round to a baseline round, 1.13 to 2.75, is the smaller of the two facts, because the retrain and the evaluation that both arms pay make up most of the round and dilute it. Degrading to the deterministic heuristic is a real deployment option for a user who cannot pay the inference, at a cost of about one and a third success-rate points.

The experiments are in simulation and no expert is a person. Every result above comes from simulation, with an expert that answers instantly, answers correctly, and answers any prescription at the same price. On Lift, Wipe and Door that expert is a scripted oracle. On Push-T it is a policy trained by proximal policy optimisation [80], competent on the configurations the task's constraint store admits and unreliable outside them, which is why those configurations are excluded before a prescription is issued. On GridWorld it is a person. The uniformity assumption is what allows the budget to be counted in demonstrations. Outside simulation the budget is a person's time, demonstrations differ by an order of magnitude in what they cost to produce, and human demonstrators are not uniformly correct [41, 52, 58]. The information-gain argument of Section 5.1.5 depends on that assumption, because high pre-retrain loss identifies novel data only if invalid demonstrations are ruled out by construction. The second half of that argument fails the moment the demonstrator is a person, and Aim 3 carries the consequence.

5.2 M1 conclusion

Aim 1 asked whether a language model can raise the information content of each demonstration bought under a fixed budget, for the decision that sits inside a single round of interactive imitation learning. The answer is on record and it is qualified by its own ablations.

What the evidence established. The DAgger family and its descendants decide when to hand control to the expert [77]. DISEIL decides which failure mode receives the round's demonstration and where that demonstration begins. Across ten settings, DISEIL attains the best mean success rate in every one, with a mean margin of 2.80 points over the strongest baseline in each setting. The claim of record is the conservative one, because the two modalities of a task are not independent experiments: collapsed to five task means, the sweep holds at five wins from five and a paired t-test gives $t(4) = 4.10, p = 0.015$. The ablations say where the advantage lives, and they say it more sharply than the comparison table does. Removing the partition over failure modes costs 4.37 points on the three ablation settings, turns the margin negative at −53.2 per cent retained, and drops the system below its own best baseline on Push-T (state) and Door (image), while per-demonstration information gain does not fall. Greedy worst-loss selection collects demonstrations that are individually informative and jointly redundant, and allocation is the term that supplies what information gain, measured per demonstration, cannot. Two controls bracket the result: uniform-random replay of a recorded failure lands below the strongest gated baseline on the two robot settings and level with it on GridWorld (image), and the deterministic nearest-untried rule promoted to a whole method beats the strongest baseline on only one of the three settings and turns the margin negative on another. The advantage is largest where the budget is smallest, averaging +9.07 points at $B = 10$ against +2.83 at $B = 40$, which is the behaviour a sample-efficiency method should show.

What the evidence did not support. Two of the framework's own design claims did not survive its own ablations, and they are carried as findings and not as caveats. Each language-model component is worth about one and a third points, and every individual gap is comparable to the seed standard error of the corresponding full run. The reason is structural: the partition is geometric and consumes no output from any foundation model, so by the time the language model is called, the decision that matters has already been taken. And the cluster memory, which an earlier draft of the method advanced as the second of its two contributions, is the least damaging of the seven knockouts, at 0.73 points on average, with a price that varies by a factor of three across the three ablation settings. It is carried as a configurable, task-dependent component of the loop, its value is reported per task rather than as a headline, and the memory Aim 2 needs is a different object: one indexed on the dataset the policy was trained on, and not on the descriptor space the failures live in.

The framework therefore works, and it works for a reason that is not the reason the method originally advanced. The selector reasons about the failure in front of it and holds no representation of the dataset behind it, which is the limitation Aim 2 exists to remove. Section 6.1 lists the items of Aim-1 work that remain outstanding.

---

6. Project plan

6.1 Completed work

The first nine months of candidature delivered Aim 1 in full. The literature review covered interactive imitation learning and the query gates of the DAgger family, language and vision-language models as reasoners in robotics, structured environmental knowledge, and demonstration selection and curation, and it produced the statement of the gap in Section 3.1. The DISEIL framework was then specified and implemented as a single module with one entry point, so that one command runs one cell of the experimental matrix.

The experimental programme covers five tasks under two observation modalities, which is ten settings, against six comparison methods, with nine seeds on GridWorld and five seeds on each robot task. An ablation programme of eighteen studies, A1 to A18, was run on top of that matrix and is reported on the three ablation settings. The statistical analysis was carried out as a scripted pass over the results workbook and not by hand, and every figure and table in Section 5.1 is generated from the recorded run outputs. DISEIL attains the best mean success rate in all ten settings, with a mean margin of 3.71 points over the strongest baseline in each. Collapsed to five task means, the sweep holds at five from five, paired $t(4) = 4.15$, $p = 0.014$. The Aim-1 manuscript, *Demonstration Distillation for Sample-Efficient Imitation Learning*, was submitted in July 2026 and is under review. This report was drafted alongside it.

Four items of Aim-1 work remain outstanding and are scheduled before the author-response window rather than deferred. The cluster memory is re-run with a per-task kernel width, defined as a fraction of each task's reset range, which is the identified fix for the mis-scaling reported in Section 5.1.9. The compute and token-cost measurement is repeated at further seeds, because no cost figure in Section 5.1.7.5 carries cross-seed variance. The failure-count diagnostic is instrumented on one setting only and is extended to the three ablation settings. The prescription logs are inspected to resolve the disagreement between the account of bridging in Section 4.1.3 and the bridged share the diagnostics record. The kernel-width re-run is taken first, because it is the one defect whose fix is specified and whose outcome changes what the work is permitted to claim about its own memory.

6.2 Updated project plan table

Year 1, from November 2025 to August 2026, is complete. It carried the literature review, the specification and implementation of the framework, the full experimental matrix, the ablation programme, the Aim-1 manuscript, the compulsory higher-degree-research training and this report.

Year 2, from September 2026 to October 2027, is Aim 2. Problem formulation and the captioner begin in September 2026, while the Aim-1 review is still running, so that no development period waits on a review outcome. Implementation and the matched-information ablation of Section 4.2.4 run to the paper submission in late May 2027, and the mid-candidature progress review falls in the same month. Thesis writing begins in November 2027 and draws on material that has already been through peer review.

Year 3, from November 2027 to November 2028, is Aim 3 and the thesis. Ethics preparation begins in November 2027 and the application for the teaching study is lodged well before the experimentation window opens. The scripted-teacher validation of the demand model does not require approval and runs from February 2028, so the Aim-3 submission in late May 2028 does not rest on the approval date. The full thesis draft goes to the supervisors in September 2028, two months before submission.

Table 11 lists the milestones, the training items and the target venues, and Figure 19 draws the same schedule.

ID	Milestone	Venue	Date
	Year 1 (November 2025 to August 2026): Aim 1, completed		
M1	Candidature start		13 Nov 2025
T1	Research integrity training		2 Dec 2025
T2	Respectful behaviour module, higher-degree research		2 Dec 2025
T3	Research induction training		10 Dec 2025
M2	Aim-1 framework specified and implemented		Apr 2026
M3	Aim-1 experimental matrix complete, ten settings and six baselines		Jun 2026
T4	SSC900 Academic Writing and Communication, passed		Trimester 1, 2026
T5	Reproducibility and integrity audit of the Aim-1 results workbook		Jul 2026
M4	Aim-1 paper submitted	AAAI 2027, main track	Jul 2026
M5	**Confirmation of Candidature**		**13 Aug 2026**
	Year 2 (September 2026 to October 2027): Aim 2		
M6	Aim 1 complete, review outcome resolved	AAAI 2027	Feb 2027
T6	Reproducibility and integrity audit of the Aim-2 results		May 2027
M7	**Mid-candidature progress review**		**May 2027**
M8	Aim-2 paper submitted	CoRL 2027	late May 2027
M9	Aim 2 complete, paper presented and thesis chapter drafted	CoRL 2027	Nov 2027
	Year 3 (November 2027 to November 2028): Aim 3 and thesis		
T7	Human-research ethics training and application preparation		Nov 2027
M10	Human-research ethics approval, Aim-3 teaching study		Mar 2028, target
T8	Reproducibility and integrity audit of the Aim-3 results		May 2028
M11	Aim-3 paper submitted	CoRL 2028	late May 2028
M12	Full thesis draft to supervisors		Sep 2028
M13	Aim 3 complete, human study analysed and chapter final	CoRL 2028	Oct 2028
M14	**Thesis submission**		**Nov 2028**

Table 11. Updated project plan.

Two dates are fixed by the university, M5 and M14, and two are fixed by a venue, M8 and M11. M10 is a target and not a commitment, because the approval date is set by the ethics committee and not by the candidate; the schedule is built so that the Aim-3 submission survives a delay in it.

6.3 Thesis plan

The thesis is written from the three papers and from this report. Table 12 gives the chapter list in binding order and the year in which each chapter is drafted, updated and finalised. Chapters 1, 2 and 3 exist in draft, in the form of this report and the submitted Aim-1 manuscript.

Chapter	2026	2027	2028
1. Introduction	draft	update	final
2. Background and literature review	draft	update	final
3. Demonstration distillation under a fixed budget (Aim 1)	draft	update	final
4. Reverse vision-language-action, a coverage memory (Aim 2)		draft	final
5. Demonstration demand across tasks, embodiments and teachers (Aim 3)			draft, final
6. Discussion and conclusion			draft, final
7. References	draft	update	final
8. Appendices (ablation record, prompts, constraint stores, certificates)	draft	update	final

Table 12. Thesis plan. Each chapter is marked by the year in which its draft is written and the year in which it is finalised; the chapters drawn from Aim 1 and from this report already exist in draft, and the Aim-2 and Aim-3 chapters are written from the papers as those aims complete.

The same schedule is drawn as a chart in Figure 19.

6.4 Gantt chart

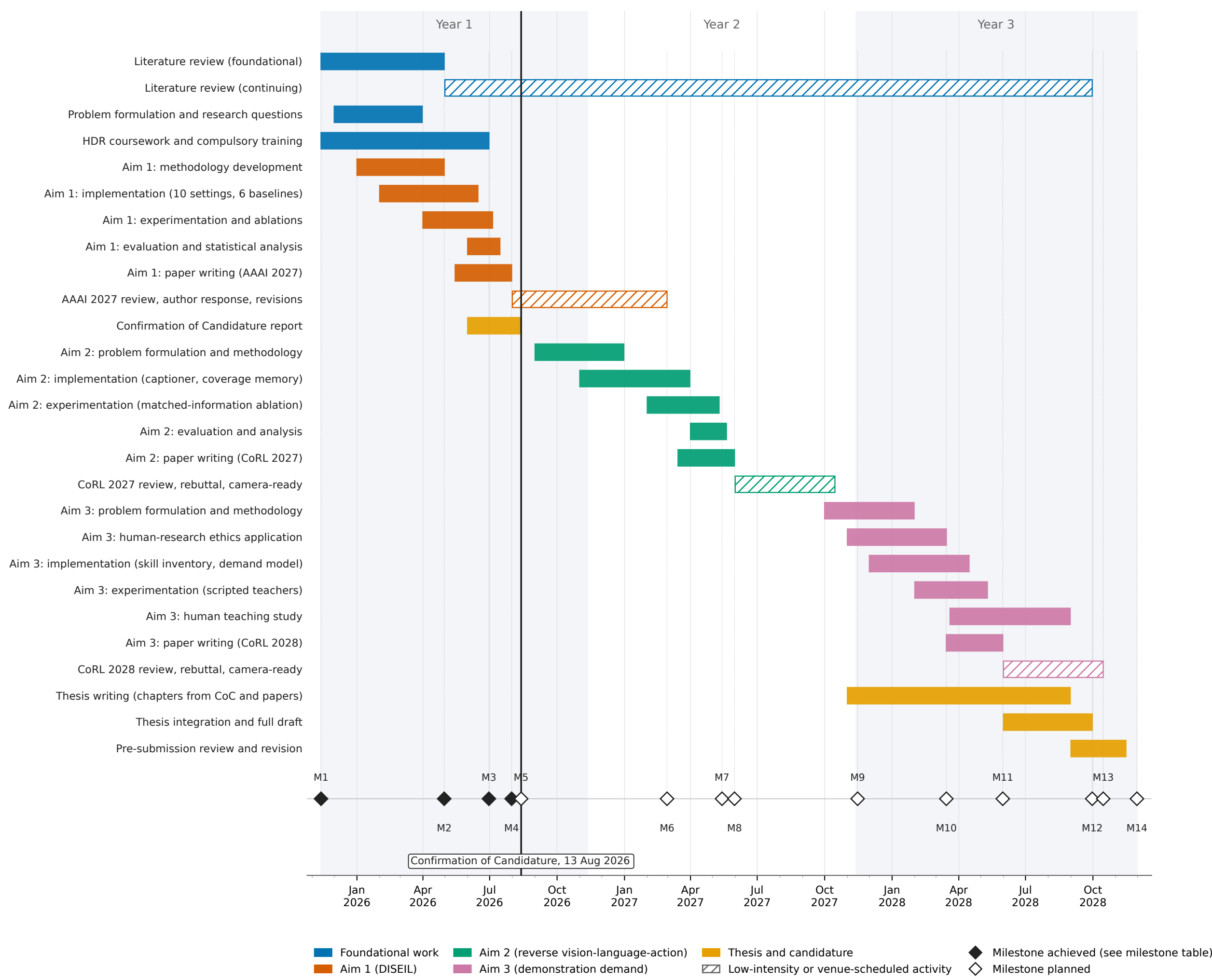


Figure 19. Project plan for the full candidature.

---

7. Ethical considerations

7.1 The position of the programme to date

No human participant has taken part in this research, and no human-subject data has been collected. Every experiment reported in Section 5.1 ran in simulation on a shared high-performance computing cluster, and the demonstrations that enter the training sets are produced either by a scripted or planner-based expert or, on the GridWorld task, by the candidate. No third party supplied a demonstration, and no personal information was recorded at any point.

The work carries no immediate deployment risk of its own. The framework decides which demonstration to collect next. It does not control a robot, and the language model is never in the path between an observation and an action at execution time. The policies trained in the evaluation are simulated manipulators and a grid agent, and none of them has been transferred to hardware.

7.2 The Aim-3 human study

Aim 3 is the point at which people enter the programme, and it does so in a bounded way. Non-expert participants will be asked to satisfy machine-generated demonstration requests in simulation, using a teleoperation interface. An application to the Deakin University human-research ethics committee will be prepared and approved before any participant is approached, and the target date for approval is recorded as milestone M10 in Table 11.

Four commitments constrain the design of that study, and they are stated here so that the panel can hold the eventual application to them.

Participation is voluntary and informed. Participants will be told what the demonstrations are used for, that the data trains a simulated policy, and that they may withdraw.

Data collection is minimal. The only data recorded is the demonstration itself, together with the time taken to produce it, because teacher time is the quantity Aim 3 measures. No personal information beyond what the consent process requires is collected, and demonstrations are stored against a participant identifier and not a name.

No request is issued that has not passed the feasibility check. The constraint store exists to reject prescriptions the environment cannot instantiate and the robot cannot reach, and with a person on the other end of the request an infeasible prescription is not a wasted round but wasted human time. The policy-solvability screen serves the same purpose, because a request the policy can already satisfy wastes a participant's time by construction.

Participants are not evaluated. The study measures the demand model and not the person. Demonstration quality is scored only to test the quality filter described in Section 4.3.2, and it is not reported per participant.

7.3 Data management, research integrity and the use of generative models

Primary data is retained and traceable to the claim it supports. Every figure and every table in Section 5.1 is generated from the recorded run outputs by script and not transcribed by hand, and the results workbook is the single source of every number in this report. Where a measurement was not taken, no placeholder is substituted for it, and the report says the measurement is missing. The items that remain open in the record are listed in Section 6.1 rather than resolved by assumption.

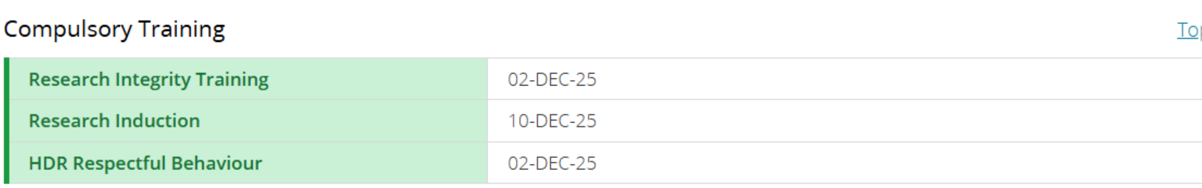
Two claims proposed during the study were not sustained by it, and both are retracted in the text where they arose: the halved-budget headline of Section 5.1.7.3, and the framing of the cluster memory as a principal driver of the framework's advantage.

The programme uses open-weight language and vision-language models as components [4, 94]. Their outputs are prescriptions and root-cause labels, and every prescription is verified against an explicit store of environmental constraints before it is acted on. The models are not used to generate the research claims, and the numbers in this report come from measured runs. The known failure mode of these models, the confident assertion of geometry they cannot perceive [15, 27], is the reason the framework hands them a computed descriptor and checks what they return.

Two wider issues are recorded, without overstating the programme's proximity to either. Automating the selection of what a person is asked to demonstrate is a form of task allocation to that person, and the participant on the other end of an Aim-3 request is being directed by a model. The mitigations are the ones above: the request is readable, it carries the reason it was issued, and it has been checked for feasibility before it is issued. The rationale the Aim-2 selector emits is what makes that direction auditable by the person who holds the budget, and it is scored as a deliverable. Separately, sample-efficient imitation learning lowers the cost of teaching a robot a task, and the tasks in this programme are manipulation benchmarks with no dual-use character. The framework is not task-specific and could in principle be applied to a task with a different character, which is a property it shares with imitation learning in general, and the programme develops no capability specific to a harmful application.

Appendix A. Higher-degree research training

The compulsory higher-degree-research training required by Deakin University in the first year of candidature is complete. All three modules were completed in December 2025, within the first month of the candidature, which began on 13 November 2025. Figure 20 reproduces the candidate's training record and is the evidence for that statement. The elective unit SSC900 Academic Writing and Communication was taken in trimester 1 of 2026 and passed. The certificates and the statement of results are reproduced below as A.1, A.2 and A.3. No certificate was issued for the Research Induction module in the documents available to the candidate, and the training record in Figure 20 is the evidence offered for that item.


Compulsory Training

Top

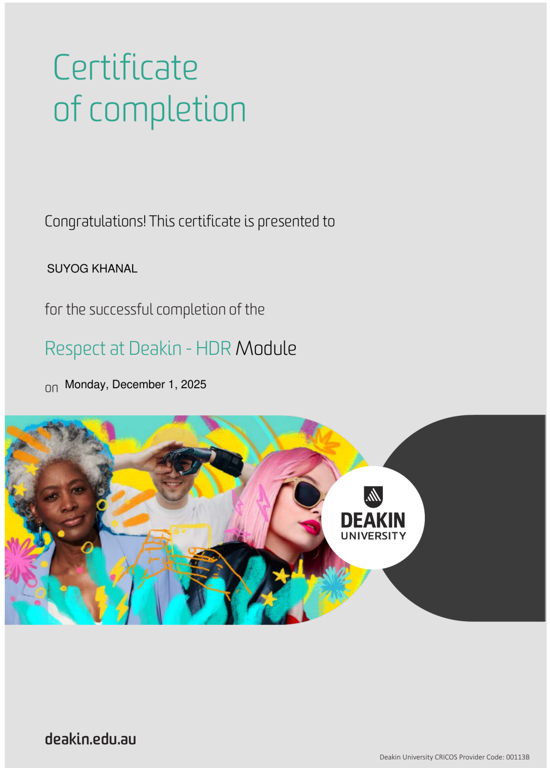
Research Integrity Training	02-DEC-25
Research Induction	10-DEC-25
HDR Respectful Behaviour	02-DEC-25

Figure 20. Status of the compulsory higher-degree-research training. The three modules required in the first year of candidature are recorded as complete.

A.1 Certificate of completion, Deakin Safety and Research Integrity Training

Figure A.1. Certificate of completion, Deakin Safety and Research Integrity Training, dated 1 December 2025. The training record in Figure 20 logs the same module as complete on 02-DEC-25.

A.2 Certificate of completion, Respect at Deakin, graduate research and supervision module


Certificate
of completion

Congratulations! This certificate is presented to

SUYOG KHANAL

for the successful completion of the

Respect at Deakin - HDR Module

on Monday, December 1, 2025

deakin.edu.au

Deakin University CRICOS Provider Code: 00113B

Figure A.2. Certificate of completion, Respect at Deakin, the higher-degree-research graduate research and supervision module, dated 1 December 2025.

A.3 Statement of results, SSC900 Academic Writing and Communication

Deakin University
Statement of Results

DISCLAIMER: This is an unofficial Statement of Results. It includes all results that are available on the day that this statement is generated. If formal documentation is required, please obtain an academic transcript.

Student Name: SUYOG KHANAL
Student ID: 226137394

Course: F975 DOCTOR OF PHILOSOPHY

UNIT CODE	UNIT NAME	UNIT PERIOD	MARK	GRADE
FAR972	PH D RESEARCH (2 CREDIT LOAD)	2025/HDR-Q4		CE
FAR972	PH D RESEARCH (2 CREDIT LOAD)	2026/HDR-Q1		CE
FAR972	PH D RESEARCH (2 CREDIT LOAD)	2026/HDR-Q2		CE
SSC900	ACADEMIC WRITING AND COMMUNICATION	2026/TRI-1		UP

Key to Results: deakin.edu.au/students/studying/assessment-and-results/results-key

Generated on: 13/07/2026 14:55:42

Figure A.3. Statement of results, recording SSC900 Academic Writing and Communication with the grade UP, the ungraded pass, and the FAR972 PhD Research enrolments with the grade CE for continuing enrolment. The document is a statement of results and is unofficial on its own terms; an academic transcript can be obtained if the panel requires formal evidence.

References

[1] Ahn, M.; Brohan, A.; Brown, N.; et al. 2022. Do As I Can, Not As I Say: Grounding Language in Robotic Affordances. In Proceedings of the 6th Conference on Robot Learning (CoRL), PMLR.

[2] Argall, B. D.; Chernova, S.; Veloso, M.; et al. 2009. A Survey of Robot Learning from Demonstration. Robotics and Autonomous Systems, 57(5), 469–483. doi:10.1016/j.robot.2008.10.024.

[3] Ash, J. T.; Zhang, C.; Krishnamurthy, A.; et al. 2020. Deep Batch Active Learning by Diverse, Uncertain Gradient Lower Bounds. In International Conference on Learning Representations (ICLR).

[4] Bai, S.; et al. 2025. Qwen3-VL Technical Report. arXiv:2511.21631.

[5] Bain, M.; Sammut, C. 1995. A Framework for Behavioural Cloning. In Machine Intelligence 15: Intelligent Agents, 103–129, Oxford University Press. https://academic.oup.com/book/53289/chapter/422019159.

[6] Belkhale, S.; Cui, Y.; Sadigh, D. 2023. Data Quality in Imitation Learning. arXiv:2306.02437.

[7] Belkhale, S.; Ding, T.; Xiao, T.; et al. 2024. RT-H: Action Hierarchies Using Language. In Proceedings of Robotics: Science and Systems (RSS). arXiv:2403.01823.

[8] Black, K.; Brown, N.; Driess, D.; et al. 2025. pi0: A Vision-Language-Action Flow Model for General Robot Control. In Proceedings of Robotics: Science and Systems (RSS). arXiv:2410.24164.

[9] Bousmalis, K.; Vezzani, G.; Rao, D.; et al. 2023. RoboCat: A Self-Improving Generalist Agent for Robotic Manipulation. Transactions on Machine Learning Research. arXiv:2306.11706.

[10] Brohan, A.; Brown, N.; Carbajal, J.; et al. 2022. RT-1: Robotics Transformer for Real-World Control at Scale. arXiv:2212.06817.

[11] Brohan, A.; Brown, N.; Carbajal, J.; et al. 2023. RT-2: Vision-Language-Action Models Transfer Web Knowledge to Robotic Control. In Proceedings of the 7th Conference on Robot Learning (CoRL), PMLR.

[12] Brown, D. S.; Goo, W.; Nagarajan, P.; et al. 2019. Extrapolating Beyond Suboptimal Demonstrations via Inverse Reinforcement Learning from Observations. In Proceedings of the 36th International Conference on Machine Learning (ICML), 97, PMLR. arXiv:1904.06387.

[13] Cazenavette, G.; Wang, T.; Torralba, A.; et al. 2022. Dataset Distillation by Matching Training Trajectories. In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR). arXiv:2203.11932.

[14] Celemin, C.; Pérez-Dattari, R.; Chisari, E.; et al. 2022. Interactive Imitation Learning in Robotics: A Survey. Foundations and Trends in Robotics, 10(1–2), 1–197. doi:10.1561/2300000072.

[15] Chen, B.; Xu, Z.; Kirmani, S.; et al. 2024. SpatialVLM: Endowing Vision-Language Models with Spatial Reasoning Capabilities. arXiv:2401.12168.

[16] Chen, Y.; Arkin, J.; Dawson, C.; et al. 2024. AutoTAMP: Autoregressive Task and Motion Planning with LLMs as Translators and Checkers. In 2024 IEEE International Conference on Robotics and Automation (ICRA), IEEE. arXiv:2306.06531.

[17] Chi, C.; Xu, Z.; Feng, S.; et al. 2023. Diffusion Policy: Visuomotor Policy Learning via Action Diffusion. In Proceedings of Robotics: Science and Systems (RSS).

[18] Christiano, P.; Leike, J.; Brown, T. B.; et al. 2017. Deep Reinforcement Learning from Human Preferences. arXiv:1706.03741.

[19] Cormen, T. H.; Leiserson, C. E.; Rivest, R. L.; et al. 2022. Introduction to Algorithms. 4th edition. MIT Press.

[20] Driess, D.; Xia, F.; Sajjadi, M. S. M.; et al. 2023. PaLM-E: An Embodied Multimodal Language Model. In Proceedings of the 40th International Conference on Machine Learning (ICML), 202, PMLR.

[21] Duan, J.; Pumacay, W.; Kumar, N.; et al. 2025. AHA: A Vision-Language-Model for Detecting and Reasoning Over Failures in Robotic Manipulation. In International Conference on Learning Representations (ICLR).

[22] Edge, D.; Trinh, H.; Cheng, N.; et al. 2024. From Local to Global: A Graph RAG Approach to Query-Focused Summarization. arXiv:2404.16130.

[23] Eldar, Y.; Lindenbaum, M.; Porat, M.; et al. 1997. The Farthest Point Strategy for Progressive Image Sampling. IEEE Transactions on Image Processing, 6(9), 1305–1315. doi:10.1109/83.623193.

[24] Eysenbach, B.; Gu, S.; Ibarz, J.; et al. 2018. Leave no Trace: Learning to Reset for Safe and Autonomous Reinforcement Learning. In International Conference on Learning Representations (ICLR).

[25] Florence, P.; Lynch, C.; Zeng, A.; et al. 2021. Implicit Behavioral Cloning. In Proceedings of the 5th Conference on Robot Learning (CoRL), 164, PMLR.

[26] Florensa, C.; Held, D.; Wulfmeier, M.; et al. 2017. Reverse Curriculum Generation for Reinforcement Learning. In Proceedings of the 1st Annual Conference on Robot Learning (CoRL), 78, PMLR.

[27] Fu, X.; Hu, Y.; Li, B.; et al. 2024. BLINK: Multimodal Large Language Models Can See but Not Perceive. In European Conference on Computer Vision (ECCV), Springer. arXiv:2404.12390.

[28] Gal, Y.; Ghahramani, Z. 2016. Dropout as a Bayesian Approximation: Representing Model Uncertainty in Deep Learning. In Proceedings of the 33rd International Conference on Machine Learning (ICML), 48, 1050–1059, PMLR.

[29] Gu, J.; Xiang, F.; Li, X.; et al. 2023. ManiSkill2: A Unified Benchmark for Generalizable Manipulation Skills. In International Conference on Learning Representations (ICLR).

[30] Hart, P. E.; Nilsson, N. J.; Raphael, B. 1968. A Formal Basis for the Heuristic Determination of Minimum Cost Paths. IEEE Transactions on Systems Science and Cybernetics, 4(2), 100–107. doi:10.1109/TSSC.1968.300136.

[31] Hejna, J.; Bhateja, C.; Jiang, Y.; et al. 2024. Re-Mix: Optimizing Data Mixtures for Large Scale Imitation Learning. arXiv:2408.14037.

[32] Ho, J.; Jain, A.; Abbeel, P. 2020. Denoising Diffusion Probabilistic Models. In Advances in Neural Information Processing Systems (NeurIPS), 33.

[33] Hoque, R.; Balakrishna, A.; Putterman, C.; et al. 2021. LazyDAgger: Reducing Context Switching in Interactive Imitation Learning. In 2021 IEEE 17th International Conference on Automation Science and Engineering (CASE), IEEE. doi:10.1109/CASE49439.2021.9551469.

[34] Hoque, R.; Balakrishna, A.; Novoseller, E.; et al. 2021. ThriftyDAgger: Budget-Aware Novelty and Risk Gating for Interactive Imitation Learning. In Proceedings of the 5th Conference on Robot Learning (CoRL), 164, PMLR.

[35] Houlsby, N.; Huszár, F.; Ghahramani, Z.; et al. 2011. Bayesian Active Learning for Classification and Preference Learning. arXiv:1112.5745.

[36] Huang, W.; Xia, F.; Xiao, T.; et al. 2022. Inner Monologue: Embodied Reasoning through Planning with Language Models. In Proceedings of the 6th Conference on Robot Learning (CoRL), PMLR.

[37] Huang, W.; Wang, C.; Zhang, R.; et al. 2023. VoxPoser: Composable 3D Value Maps for Robotic Manipulation with Language Models. In Proceedings of the 7th Conference on Robot Learning (CoRL), PMLR.

[38] James, S.; Ma, Z.; Arrojo, D. R.; et al. 2019. RLBench: The Robot Learning Benchmark & Learning Environment. arXiv:1909.12271.

[39] Janner, M.; Du, Y.; Tenenbaum, J. B.; et al. 2022. Planning with Diffusion for Flexible Behavior Synthesis. In Proceedings of the 39th International Conference on Machine Learning (ICML), 162, PMLR.

[40] Kelly, M.; Sidrane, C.; Driggs-Campbell, K.; et al. 2019. HG-DAgger: Interactive Imitation Learning with Human Experts. In 2019 International Conference on Robotics and Automation (ICRA), 8077–8083, IEEE. doi:10.1109/ICRA.2019.8793698.

[41] Khazatsky, A.; Pertsch, K.; Nair, S.; et al. 2024. DROID: A Large-Scale In-The-Wild Robot Manipulation Dataset. arXiv:2403.12945.

[42] Kim, M. J.; Pertsch, K.; Karamcheti, S.; et al. 2024. OpenVLA: An Open-Source Vision-Language-Action Model. In Proceedings of the 8th Conference on Robot Learning (CoRL), PMLR. arXiv:2406.09246.

[43] Krishna, R.; Hata, K.; Ren, F.; et al. 2017. Dense-Captioning Events in Videos. arXiv:1705.00754.

[44] Lakshminarayanan, B.; Pritzel, A.; Blundell, C. 2017. Simple and Scalable Predictive Uncertainty Estimation using Deep Ensembles. In Advances in Neural Information Processing Systems (NeurIPS), 30.

[45] Laskey, M.; Lee, J.; Fox, R.; et al. 2017. DART: Noise Injection for Robust Imitation Learning. In Proceedings of the 1st Annual Conference on Robot Learning (CoRL), 78, 143–156, PMLR.

[46] Lee, S. W.; Kang, X.; Kuo, Y. L. 2025. Diff-DAgger: Uncertainty Estimation with Diffusion Policy for Robotic Manipulation. In 2025 IEEE International Conference on Robotics and Automation (ICRA), IEEE. arXiv:2410.14868.

[47] Lewis, P.; Perez, E.; Piktus, A.; et al. 2020. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In Advances in Neural Information Processing Systems (NeurIPS), 33. arXiv:2005.11401.

[48] Liang, J.; Huang, W.; Xia, F.; et al. 2023. Code as Policies: Language Model Programs for Embodied Control. In 2023 IEEE International Conference on Robotics and Automation (ICRA), IEEE.

[49] Lin, F.; Hu, Y.; Sheng, P.; et al. 2024. Data Scaling Laws in Imitation Learning for Robotic Manipulation. arXiv:2410.18647.

[50] Liu, B.; Zhu, Y.; Gao, C.; et al. 2023. LIBERO: Benchmarking Knowledge Transfer for Lifelong Robot Learning. arXiv:2306.03310.

[51] Liu, B.; Jiang, Y.; Zhang, X.; et al. 2023. LLM+P: Empowering Large Language Models with Optimal Planning Proficiency. arXiv:2304.11477.

[52] Liu, H.; Nasiriany, S.; Zhang, L.; et al. 2023. Robot Learning on the Job: Human-in-the-Loop Autonomy and Learning During Deployment. In Proceedings of Robotics: Science and Systems (RSS). doi:10.15607/RSS.2023.XIX.005.

[53] Liu, Z.; Bahety, A.; Song, S. 2023. REFLECT: Summarizing Robot Experiences for Failure Explanation and Correction. In Proceedings of the 7th Conference on Robot Learning (CoRL), PMLR.

[54] Lloyd, S. P. 1982. Least Squares Quantization in PCM. IEEE Transactions on Information Theory, 28(2), 129–137. doi:10.1109/TIT.1982.1056489.

[55] Ma, Y. J.; Sodhani, S.; Jayaraman, D.; et al. 2023. VIP: Towards Universal Visual Reward and Representation via Value-Implicit Pre-Training. In International Conference on Learning Representations (ICLR).

[56] Ma, Y. J.; Liang, W.; Wang, G.; et al. 2024. Eureka: Human-Level Reward Design via Coding Large Language Models. In International Conference on Learning Representations (ICLR).

[57] Madaan, A.; Tandon, N.; Gupta, P.; et al. 2023. SELF-REFINE: Iterative Refinement with Self-Feedback. In Advances in Neural Information Processing Systems (NeurIPS), 36.

[58] Mandlekar, A.; Zhu, Y.; Garg, A.; et al. 2018. RoboTurk: A Crowdsourcing Platform for Robotic Skill Learning through Imitation. In Proceedings of the 2nd Conference on Robot Learning (CoRL), PMLR. arXiv:1811.02790.

[59] Mandlekar, A.; Xu, D.; Martín-Martín, R.; et al. 2020. Human-in-the-Loop Imitation Learning using Remote Teleoperation. arXiv:2012.06733.

[60] Mandlekar, A.; Xu, D.; Wong, J.; et al. 2021. What Matters in Learning from Offline Human Demonstrations for Robot Manipulation. In Proceedings of the 5th Conference on Robot Learning (CoRL), 164, PMLR.

[61] Mees, O.; Hermann, L.; Rosete-Beas, E.; et al. 2022. CALVIN: A Benchmark for Language-Conditioned Policy Learning for Long-Horizon Robot Manipulation Tasks. IEEE Robotics and Automation Letters. arXiv:2112.03227.

[62] Memmel, M.; Berg, J.; Chen, B.; et al. 2025. STRAP: Robot Sub-Trajectory Retrieval for Augmented Policy Learning. In International Conference on Learning Representations (ICLR). arXiv:2412.15182.

[63] Menda, K.; Driggs-Campbell, K.; Kochenderfer, M. J. 2017. DropoutDAgger: A Bayesian Approach to Safe Imitation Learning. arXiv:1709.06166.

[64] Menda, K.; Driggs-Campbell, K.; Kochenderfer, M. J. 2019. EnsembleDAgger: A Bayesian Approach to Safe Imitation Learning. In 2019 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), IEEE. doi:10.1109/IROS40897.2019.8968287.

[65] Mu, T.; Ling, Z.; Xiang, F.; et al. 2021. ManiSkill: Generalizable Manipulation Skill Benchmark with Large-Scale Demonstrations. In Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track, 34.

[66] Nair, S.; Rajeswaran, A.; Kumar, V.; et al. 2022. R3M: A Universal Visual Representation for Robot Manipulation. In Proceedings of the 6th Conference on Robot Learning (CoRL), PMLR.

[67] Octo Model Team; Ghosh, D.; Walke, H.; et al. 2024. Octo: An Open-Source Generalist Robot Policy. In Proceedings of Robotics: Science and Systems (RSS). arXiv:2405.12213.

[68] Open X-Embodiment Collaboration; O'Neill, A.; Rehman, A.; et al. 2023. Open X-Embodiment: Robotic Learning Datasets and RT-X Models. arXiv:2310.08864.

[69] Osa, T.; Pajarinen, J.; Neumann, G.; et al. 2018. An Algorithmic Perspective on Imitation Learning. Foundations and Trends in Robotics, 7(1–2), 1–179. arXiv:1811.06711.

[70] Pedregosa, F.; Varoquaux, G.; Gramfort, A.; et al. 2011. Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825–2830. arXiv:1201.0490.

[71] Pomerleau, D. A. 1988. ALVINN: An Autonomous Land Vehicle in a Neural Network. In Advances in Neural Information Processing Systems (NeurIPS), 1, 305–313, Morgan Kaufmann.

[72] Pomerleau, D. A. 1991. Efficient Training of Artificial Neural Networks for Autonomous Navigation. Neural Computation, 3(1), 88–97. doi:10.1162/neco.1991.3.1.88.

[73] Radosavovic, I.; Xiao, T.; James, S.; et al. 2022. Real-World Robot Learning with Masked Visual Pre-training. In Proceedings of the 6th Conference on Robot Learning (CoRL), PMLR.

[74] Reed, S.; Zolna, K.; Parisotto, E.; et al. 2022. A Generalist Agent. Transactions on Machine Learning Research. arXiv:2205.06175.

[75] Ren, A. Z.; Dixit, A.; Bodrova, A.; et al. 2023. Robots That Ask For Help: Uncertainty Alignment for Large Language Model Planners. In Proceedings of the 7th Conference on Robot Learning (CoRL), PMLR. arXiv:2307.01928.

[76] Ross, S.; Bagnell, J. A. 2010. Efficient Reductions for Imitation Learning. In Proceedings of the 13th International Conference on Artificial Intelligence and Statistics (AISTATS), 9, 661–668, PMLR. https://proceedings.mlr.press/v9/ross10a.html.

[77] Ross, S.; Gordon, G. J.; Bagnell, J. A. 2011. A Reduction of Imitation Learning and Structured Prediction to No-Regret Online Learning. In Proceedings of the 14th International Conference on Artificial Intelligence and Statistics (AISTATS), 15, 627–635, PMLR.

[78] Ross, S.; Bagnell, J. A. 2014. Reinforcement and Imitation Learning via Interactive No-Regret Learning. arXiv:1406.5979.

[79] Rousseeuw, P. J. 1987. Silhouettes: A Graphical Aid to the Interpretation and Validation of Cluster Analysis. Journal of Computational and Applied Mathematics, 20, 53–65. doi:10.1016/0377-0427(87)90125-7.

[80] Schulman, J.; Wolski, F.; Dhariwal, P.; et al. 2017. Proximal Policy Optimization Algorithms. arXiv:1707.06347.

[81] Sener, O.; Savarese, S. 2018. Active Learning for Convolutional Neural Networks: A Core-Set Approach. In International Conference on Learning Representations (ICLR).

[82] Settles, B. 2009. Active Learning Literature Survey. University of Wisconsin–Madison, Computer Sciences Technical Report 1648.

[83] Shimodaira, H. 2000. Improving Predictive Inference under Covariate Shift by Weighting the Log-Likelihood Function. Journal of Statistical Planning and Inference, 90(2), 227–244. doi:10.1016/S0378-3758(00)00115-4.

[84] Shinn, N.; Cassano, F.; Berman, E.; et al. 2023. Reflexion: Language Agents with Verbal Reinforcement Learning. In Advances in Neural Information Processing Systems (NeurIPS), 36.

[85] Singh, I.; Blukis, V.; Mousavian, A.; et al. 2023. ProgPrompt: Generating Situated Robot Task Plans using Large Language Models. In 2023 IEEE International Conference on Robotics and Automation (ICRA), IEEE.

[86] Sun, W.; Venkatraman, A.; Gordon, G. J.; et al. 2017. Deeply AggreVaTeD: Differentiable Imitation Learning for Sequential Prediction. In Proceedings of the 34th International Conference on Machine Learning (ICML), 70, 3309–3318, PMLR.

[87] Suzuki, K.; Shimizu, S.; Ogata, T. 2025. Proprioception Enhances Vision Language Model in Generating Captions and Subtask Segmentations for Robot Task. arXiv:2512.20876.

[88] Tao, S.; Xiang, F.; Shukla, A.; et al. 2024. ManiSkill3: GPU Parallelized Robotics Simulation and Rendering for Generalizable Embodied AI. arXiv:2410.00425.

[89] Tenorth, M.; Beetz, M. 2013. KnowRob: A Knowledge Processing Infrastructure for Cognition-Enabled Robots. The International Journal of Robotics Research, 32(5), 566–590. doi:10.1177/0278364913481635.

[90] Wang, G.; Xie, Y.; Jiang, Y.; et al. 2023. Voyager: An Open-Ended Embodied Agent with Large Language Models. arXiv:2305.16291.

[91] Ward, J. H. 1963. Hierarchical Grouping to Optimize an Objective Function. Journal of the American Statistical Association, 58(301), 236–244. doi:10.1080/01621459.1963.10500845.

[92] Wulff, T.; Maharjan, R. S.; Chi, X.; et al. 2025. Joint Action Language Modelling for Transparent Policy Execution. arXiv:2504.10055.

[93] Xu, S.; Liberatori, B.; Varol, G.; Rota, P. 2026. Dense Motion Captioning. In International Conference on 3D Vision (3DV). arXiv:2511.05369.

[94] Yang, A.; et al. 2025. Qwen3 Technical Report. arXiv:2505.09388.

[95] Yu, T.; Quillen, D.; He, Z.; et al. 2019. Meta-World: A Benchmark and Evaluation for Multi-Task and Meta Reinforcement Learning. In Proceedings of the 3rd Conference on Robot Learning (CoRL), PMLR. arXiv:1910.10897.

[96] Yu, W.; Gileadi, N.; Fu, C.; et al. 2023. Language to Rewards for Robotic Skill Synthesis. In Proceedings of the 7th Conference on Robot Learning (CoRL), PMLR.

[97] Yue, S.; Liu, J.; Hua, X.; et al. 2024. How to Leverage Diverse Demonstrations in Offline Imitation Learning. In Proceedings of the 41st International Conference on Machine Learning (ICML), PMLR. arXiv:2405.17476.

[98] Zawalski, M.; Chen, W.; Pertsch, K.; et al. 2024. Robotic Control via Embodied Chain-of-Thought Reasoning. In Proceedings of the 8th Conference on Robot Learning (CoRL), PMLR. arXiv:2407.08693.

[99] Zhang, J.; Cho, K. 2017. Query-Efficient Imitation Learning for End-to-End Simulated Driving. In Proceedings of the AAAI Conference on Artificial Intelligence, 31(1), AAAI Press. doi:10.1609/aaai.v31i1.10857.

[100] Zhu, Y.; Wong, J.; Mandlekar, A.; et al. 2020. robosuite: A Modular Simulation Framework and Benchmark for Robot Learning. arXiv:2009.12293.